# Predicción de Llegadas de Autobuses con Posiciones en Tiempo Real y Horarios Interpolados

Yeray Garrido

Bilbao, Bizkaia

## Nota del Autor

Este es un trabajo personal e independiente: no se ha presentado como tesis doctoral, no ha pasado una revisión por pares y ninguna universidad, revista ni institución lo ha evaluado. Bizkaibus+ es una aplicación sin relación con Bizkaibus, con la Diputación Foral de Bizkaia ni con los operadores del servicio, y usa solo los datos abiertos que se publican: el horario en formato GTFS y el feed de posiciones en formato SIRI.

El código del motor es privado. Este documento lo describe tal como estaba el 7 de noviembre de 2026, con cada fórmula, constante y regla transcrita del código y comprobada con vectores de prueba; la implementación de referencia, los vectores y el programa que los comprueba van adjuntos al PDF. Es el informe técnico del artículo que presenta el método en inglés, y ambos se publican, con los datos y el código de referencia, en https://github.com/Garridoparrayeray/bide-eta. Versión 2.1, 7 de noviembre de 2026. © 2026 Yeray Garrido.

**Resumen**

Bizkaibus, la red de autobuses interurbanos de Bizkaia, publica la posición GPS de cada vehículo y su próxima parada, pero no su retraso, y las horas intermedias de su horario oficial están interpoladas a velocidad constante: de 169 308 pasos reales por parada, el 85 % de los que ocurren pasada la primera hora de viaje llega más de dos minutos antes de la hora publicada. Esta memoria describe, para que pueda replicarse, el método con el que la aplicación Bizkaibus+ predice las llegadas. El método trabaja con diferencias de horario en lugar de horas absolutas, aprende el tiempo real de cada tramo por franja horaria con medianas contraídas, y organiza la predicción en tres capas con reglas para abstenerse: la posición proyectada sobre el recorrido, la última parada registrada y el horario corregido viaje a viaje. Se recalibra cada día y solo adopta cambios que reducen el error. En 2,21 millones de predicciones a menos de veinte minutos, de 17 días, contrastadas con el paso real, la llegada mostrada cae dentro de $\pm 2$ minutos en el 96 % a menos de tres minutos y en el 67 % entre diez y veinte minutos, frente al 24 % y el 25 % del horario publicado. Evaluados por días, un filtro de Kalman sobre el ritmo del propio autobús y una red neuronal reducen el error medio de las sumas de tiempos de tramo un 4 % y entre un 6 y un 7 %. Se incluyen vectores de prueba y una implementación de referencia que los reproduce.

*Palabras clave:* predicción de llegadas, transporte público, posicionamiento GPS, GTFS, SIRI, estimación robusta, contracción, validación temporal

**Abstract**

Bizkaibus, the interurban bus network of Biscay, publishes the GPS position and the next stop of every vehicle but not its delay, and the intermediate times of its official timetable are interpolated at constant speed: of 169,308 actual stop passages, 85 % of those made after the first hour of travel come more than two minutes before the published time. This report describes, so that it can be reimplemented in any language, the method the Bizkaibus+ app uses to predict arrivals. The method works with timetable differences instead of absolute times, learns the actual travel time of every segment between stops by time band with medians shrunk towards the all-day median of the segment, and organizes the prediction in three layers with explicit abstention rules: the position projected on the route, the last stop registered by the feed and a timetable corrected trip by trip. It recalibrates itself daily and only adopts changes that reduce the measured error. On 2.21 million predictions within twenty minutes, from 17 days, checked against the actual passage, the displayed arrival is within $\pm 2$ minutes 96 % of the time when the bus is under three minutes away and 67 % at ten to twenty minutes, against 24 % and 25 % for the published timetable. Under a day-by-day evaluation protocol, a Kalman filter on the bus's own pace and a neural network reduce the mean error of sums of segment times by 4 % and by 6 to 7 %. Test vectors and a reference implementation that reproduces them are included.

*Keywords:* arrival time prediction, public transport, GPS positioning, GTFS, SIRI, robust estimation, shrinkage, temporal validation

**Laburpena**

Bizkaibusek, Bizkaiko hiriarteko autobus sareak, ibilgailu bakoitzaren GPS kokapena eta hurrengo geralekua argitaratzen ditu, baina ez atzerapena, eta ordutegi ofizialeko tarteko orduak abiadura konstantean interpolatuta daude: geralekuetako 169 308 benetako igaroalditan, bidaiaren lehen orduaren ondoren, igaroaldien %85 argitaratutako ordua baino bi minutu baino lehenago igarotzen da. Memoria honek Bizkaibus+ aplikazioak iritsierak aurreikusteko erabiltzen duen metodoa deskribatzen du, edozein lengoaiatan berriro egiteko behar den xehetasunarekin. Metodoak ordutegiaren diferentziak erabiltzen ditu, ez ordu absolutuak; geralekuen arteko tarte bakoitzaren benetako denbora ordu-tarteka ikasten du, tartearen egun osoko medianarantz hurbildutako medianekin; eta aurreikuspena hiru geruzatan antolatzen du, abstentzio-arau esplizituekin: ibilbidean proiektatutako kokapena, jarioak erregistratutako azken geralekua eta bidaiaz bidaia zuzendutako ordutegia. Egunero birkalibratzen da jarioaren harrapaketekin, eta neurtutako errorea jaisten duten aldaketak bakarrik onartzen ditu. Benetako igaroaldiarekin alderatutako 17 eguneko 2,21 milioi aurreikuspenetan (hogei minutu barrukoak), erakutsitako iritsiera $\pm 2$ minuturen barruan dago kasuen %96an autobusa hiru minutu baino gutxiagora dagoenean, eta %67an hamar eta hogei minutu artean; argitaratutako ordutegiak %24 eta %25 asmatzen ditu. Egunez eguneko ebaluazio-protokoloarekin, autobusaren beraren erritmoaren gaineko Kalman iragazki batek eta sare neuronal batek tarte-denboren baturen batez besteko errorea %4 eta %6-7 murrizten dute. Memoriak formulak, konstanteak, sasikodea, proba-bektoreak eta horiek erreproduzitzen dituen erreferentziazko inplementazio bat ditu.

*Gako-hitzak:* iritsieren aurreikuspena, garraio publikoa, GPS kokapena, GTFS, SIRI, estimazio sendoa, hurbilketa, denborazko balidazioa

## Contenido

Resumen . . . . . 2
Abstract . . . . . 3
Laburpena . . . . . 4
Lista de Figuras . . . . . 13
Lista de Tablas . . . . . 14
Acrónimos . . . . . 16
Introducción . . . . . 17
Motivación . . . . . 17
Esperar un Autobús Interurbano . . . . . 17
Lo que Cuesta No Saber Cuándo Llega . . . . . 17
El Horario Publicado Se Desvía en los Dos Sentidos . . . . . 18
Este Trabajo . . . . . 20
El Problema . . . . . 20
Objetivos . . . . . 21
Contribuciones . . . . . 21
Visión General del Sistema . . . . . 22
Convenciones . . . . . 22
Estructura del Documento . . . . . 24
Marco Teórico y Estado del Arte . . . . . 25
Predicción de la Hora de Llegada . . . . . 25
Medias Históricas . . . . . 25
Regresión y Modelos de Series Temporales . . . . . 25
Filtros de Kalman . . . . . 25
Redes Neuronales . . . . . 25
Trabajos Recientes . . . . . 26

Datos de Transporte: GTFS y SIRI . . . . . 27
Ajuste al Mapa . . . . . 27
Estadística Robusta . . . . . 28
Contracción y Modelos Jerárquicos . . . . . 28
Validación . . . . . 29
Fuentes y Modelo de Datos . . . . . 30
El Horario Programado . . . . . 30
Trazados que No Casan con sus Paradas . . . . . 31
Fusión de Variantes y Alias de Viaje . . . . . 32
Por Qué el Horario No Sirve Como Referencia Absoluta . . . . . 34
El Feed en Tiempo Real . . . . . 34
Referencia del Viaje . . . . . 35
Horas . . . . . 35
Retraso Declarado . . . . . 36
Lo que el Feed No Dice . . . . . 36
Notación . . . . . 37
Emparejamiento y Localización . . . . . 40
Emparejamiento con el Horario . . . . . 40
Emparejamiento Exacto . . . . . 40
Emparejamiento de Respaldo . . . . . 40
Proyección de un Punto Sobre un Segmento . . . . . 40
Plano Local . . . . . 41
Proyección . . . . . 41
Error de la Aproximación . . . . . 41
Ventana de Búsqueda . . . . . 42
Con Trazado . . . . . 43
Sin Trazado . . . . . 44

Condiciones Para Usar el GPS . . . . . 45
En la Cabecera, Antes de Salir . . . . . 46
Modelo del Tiempo de Recorrido . . . . . 48
Por Qué Diferencias de Horario . . . . . 48
La Unidad de Aprendizaje: El Tramo . . . . . 49
Estimador del Tiempo de un Tramo . . . . . 50
Las Muestras . . . . . 50
Un Valor por Pista . . . . . 51
Mediana y Contracción Hacia el Tramo a Cualquier Hora . . . . . 51
Consulta . . . . . 54
Respaldo por Longitud del Tramo . . . . . 54
Cubos de Longitud . . . . . 55
Proporción Real/Horario por Cubo y Franja . . . . . 55
Consulta . . . . . 56
Factor de Ritmo . . . . . 56
Consulta . . . . . 57
Estimación . . . . . 57
Tiempo Esperado Entre Dos Instantes del Horario . . . . . 58
La Cascada Completa . . . . . 61
Predicción en Tres Capas . . . . . 63
Entradas, Salidas y Estados . . . . . 63
Capa 1: Llegada Desde el GPS . . . . . 65
Capa 1 en la Cabecera . . . . . 65
Capa 2: Llegada Desde la Parada Registrada . . . . . 66
El Registro . . . . . 66
La Llegada . . . . . 67
Contraste Entre el GPS y la Parada Registrada . . . . . 68

Capa 3: Horario Corregido . . . . . 69
Corrección por Viaje . . . . . 70
Sin Conexión . . . . . 71
Paradas Ya Pasadas . . . . . 72
Llegando sin Confirmar el Paso . . . . . 72
Registro de Pasos . . . . . 73
Autobuses que Faltan en una Lectura del Feed . . . . . 73
Viajes que Pasan de las 24:00 . . . . . 75
La Decisión Completa . . . . . 76
El Tope de 45 Minutos . . . . . 76
El Margen de 120 Segundos . . . . . 76
Retraso y Minutos que Se Muestran . . . . . 77
Después de la Última Salida del Día . . . . . 78
Ejemplo Completo . . . . . 78
Escenario 1 . . . . . 78
Escenario 2 . . . . . 78
Escenarios 3 y 4 . . . . . 79
Escenario 5 . . . . . 79
Escenario 6 . . . . . 79
Escenario 8 . . . . . 79
Escenario 9 . . . . . 79
Escenarios 10 y 11 . . . . . 80
Escenarios 12 y 13 . . . . . 80
Aprendizaje Continuo . . . . . 83
El Ciclo. . . . . 83
Captura del Feed. . . . . 84
Pistas . . . . . 85

El Instante Real de Paso por una Parada. . . . . 85
Muestras de Tramo . . . . . 86
Muestras de Ritmo . . . . . 86
Por Qué Exigir la Ventana. . . . . 87
Aclarado . . . . . 87
Acumulación y Retención. . . . . 88
Recalibración Diaria . . . . . 89
Pliegues . . . . . 90
Factor de Ritmo . . . . . 90
Tiempos por Tramo y Proporciones . . . . . 90
Corrección por Viaje . . . . . 91
Qué Garantiza y Qué No . . . . . 92
Curva de Aprendizaje . . . . . 93
Resultados . . . . . 95
Medida del Error. . . . . 95
La Verdad de Referencia. . . . . 96
Por Posición. . . . . 96
Por Cambio de Parada . . . . . 96
Resultados . . . . . 96
Por Horizonte. . . . . 97
Por Capa . . . . . 97
Por Cobertura de Tramos Aprendidos . . . . . 98
Evaluación a lo Largo de Varios Días . . . . . 98
Evaluación por Días de la Contracción . . . . . 100
Corrección por Viaje de la Capa 3 . . . . . 101
Corrección y Partes del Recorrido . . . . . 102
La Media o un Percentil . . . . . 102

Tipo de Día, Festivos y Temporada . . . 105
Comparación con Otros Métodos . . . 105
Trazados del Derecho y Espera en la Cabecera . . . 108
Fiabilidad de lo que Dice la Aplicación . . . 110
Ideas Probadas y Descartadas . . . 111
Discusión . . . 116
Interpretación de los Resultados . . . 116
El Tramo en que Está el Autobús . . . 116
La Corrección del Horario . . . 116
La Cobertura de Tramos Aprendidos . . . 117
El Error y la Distancia . . . 117
Relación con Otros Enfoques . . . 117
Limitaciones . . . 118
Límites de la Infraestructura . . . 120
Aplicabilidad a Otras Redes . . . 122
Requisitos . . . 122
Qué Hay que Adaptar . . . 123
El Mismo Motor en Otra Red de Autobús . . . 124
Redes que Publican Retraso o Predicción . . . 124
Pasos Para Otra Ciudad . . . 127
Conclusiones y Líneas Futuras . . . 128
Resumen . . . 128
Resultados . . . 128
Lecciones . . . 129
Lo Único Vivo Es la Posición . . . 129
Revisar los Datos de Entrada . . . 129
Medir Antes de Adoptar . . . 129

Abstenerse Es Parte de la Precisión. . . . . 130
Lo Sencillo y Auditable Compite . . . . . 130
Líneas Futuras . . . . . 130
Reproducibilidad . . . . . 132
Referencias . . . . . 133
Apéndice A. Constantes . . . . . 138
Apéndice B. Pseudocódigo . . . . . 145
Apéndice C. Formatos de Datos . . . . . 153
Base de Datos del Horario . . . . . 153
Tiempos por Tramo: `segment-times.json`. . . . . 154
Factores de Ritmo: `pace-factors.json` . . . . . 154
Corrección por Viaje: `trip-offsets.json`. . . . . 155
Muestras Acumuladas (JSON Lines). . . . . 155
Registro de Pasos . . . . . 156
El Feed SIRI-VM . . . . . 156
Apéndice D. Vectores de Prueba . . . . . 158
Funciones Básicas. . . . . 158
Calibración . . . . . 159
Tiempos por Tramo . . . . . 159
Factor de Ritmo . . . . . 160
Paso Real . . . . . 161
Aclarado . . . . . 161
Corrección por Viaje . . . . . 161
Consulta . . . . . 161
Calibración . . . . . 162
El Emparejador Completo. . . . . 163

Del Feed a la Pantalla . . . . . 164
Lectura del Feed . . . . . 164
Emparejado . . . . . 165
Buses que Faltan . . . . . 165
Viajes que Pasan de Medianoche . . . . . 165
Registro de Pasos . . . . . 166
El Retraso Mostrado . . . . . 166
Elección del Percentil . . . . . 166
Apéndice E. Implementación de Referencia . . . . . 167
Cómo Comprobarla . . . . . 167
Lo que Hay que Respetar al Portar . . . . . 168
Código . . . . . 169
Apéndice F. Glosario . . . . . 198

## Lista de Figuras

Figura 1. Desvío del Paso Real Según el Tiempo Programado Desde la Salida . . . . . 19

Figura 2. Recorrido de un Dato Desde el Feed Hasta la Pantalla . . . . . 23

Figura 3. Distancia y Tiempo de un Viaje Ilustrativo . . . . . 34

Figura 4. Proyección del Autobús Sobre un Segmento: Fracción $f$ y Distancia Perpendicular $d_{\perp}$ . . . . . 42

Figura 5. Ventana de Búsqueda Alrededor de la Próxima Parada. . . . . 44

Figura 6. Estado y Llegada de una Fila (Motor de Autobús). . . . . 81

Figura 7. Error por Tramo Según el Número de Autobuses . . . . . 94

Figura B1. Proyección de un Punto Sobre un Segmento . . . . . 145

Figura B2. Posición del Autobús en el Horario (Capa 1, sin el Contraste). . . . . 146

Figura B3. Tiempo de un Tramo y Tiempo Esperado Entre Dos Horas del Horario. . . . . 147

Figura B4. Registro (Capa 2), Llegada Desde el Registro y Hora de Paso . . . . . 148

Figura B5. Posición con Contraste (Se Evalúa Antes que Cualquier Uso del Registro) . . . . . 149

Figura B6. Capa 3: Horario Corregido, Corrección por Viaje y Parte Aprendida . . . . . 150

Figura B7. Paso Real por una Parada y Aclarado de Muestras . . . . . 151

Figura B8. Ajuste de los Tiempos por Tramo . . . . . 151

Figura B9. Ajuste del Factor de Ritmo. . . . . 152

Figura B10. Validación Cruzada y Escritura de los Tiempos por Tramo . . . . . 152

## Lista de Tablas

Tabla 1. Desvío del Paso Real Respecto a la Hora Publicada, Según el Tiempo Desde la Salida . . . . . 18

Tabla 2. Tablas de la Base de Datos del Horario que Usa el Motor . . . . . 30

Tabla 3. Llegadas de Bizkaibus Según los Metros de las Paradas . . . . . 33

Tabla 4. Campos que el Motor Lee del Feed SIRI-VM . . . . . 35

Tabla 5. Proporción Entre Tiempo Real y Tiempo Programado por Cubo de Longitud . . . . . 56

Tabla 6. Cascada de Estimación del Tiempo de un Tramo . . . . . 61

Tabla 7. Estados de una Fila y lo que Muestra la Aplicación . . . . . 64

Tabla 8. Llegadas de Autobuses que Faltaban en el Feed, con y sin Guardar su Último Dato . 74

Tabla 9. Estado y Llegada Estimada en Cada Parada del Viaje Sintético . . . . . 82

Tabla 10. Error por Tramo con el Tope por Tramo y con el Tope por Tramo y Tipo de Día. . . 89

Tabla 11. Curva de Aprendizaje con la Primera Sesión de Captura . . . . . 93

Tabla 12. Precisión de la Llegada Mostrada con GPS y del Horario Publicado, por Horizonte . . . . . 97

Tabla 13. Aciertos Dentro de $\pm 2$ Minutos por Capa, en las Mismas Llegadas . . . . . 97

Tabla 14. Aciertos de la Capa 1 Según la Cobertura del Camino . . . . . 98

Tabla 15. Precisión de Todas las Llegadas Mostradas en 17 Días, por Horizonte . . . . . 99

Tabla 16. Aciertos Dentro de $\pm 2$ Minutos en 17 Días, por Origen del Dato y por Cobertura. . 100

Tabla 17. Evaluación por Días de la Contracción Hacia la Mediana del Tramo . . . . . 102

Tabla 18. Error de la Hora sin Datos del Autobús, con y sin Corrección . . . . . 103

Tabla 19. Media o Percentil en la Corrección por Viaje . . . . . 104

Tabla 20. Error Medio de la Capa 3 Corregida Según el Tipo de Día . . . . . 105

Tabla 21. Comparación con Otros Métodos con el Protocolo por Días . . . . . 107

Tabla 22. Aciertos Dentro de ±2 Minutos con los Trazados Corregidos y la Cabecera, por Horizonte . . . . . 109

Tabla 23. Lo que Afirma Cada Estado y Cuántas Veces Fue Cierto . . . . . 110

Tabla 24. Ideas Probadas y No Adoptadas . . . . . 111

Tabla A1. Constantes del Método . . . . . 138

Tabla D1. Mediana Baja y Percentiles . . . . . 158

Tabla D2. Franja de una Fecha y un Instante . . . . . 158

Tabla D3. Factor de Ritmo . . . . . 159

Tabla D4. Proyección Sobre un Segmento de Prueba . . . . . 159

Tabla D5. Cubos de Longitud . . . . . 159

Tabla D6. Respaldo por Longitud . . . . . 160

## Acrónimos

**APC** *Automatic Passenger Counting*, conteo automático de viajeros.

**AVL** *Automatic Vehicle Location*, localización automática de vehículos.

**CEN** Comité Europeo de Normalización.

**CRC** *Cyclic Redundancy Check*, código de redundancia cíclica.

**ECM** Error cuadrático medio.

**ETA** *Estimated Time of Arrival*, hora estimada de llegada.

**GPS** *Global Positioning System*, sistema de posicionamiento global.

**GTFS** *General Transit Feed Specification*, formato abierto de horarios de transporte público.

**GTFS-RT** GTFS Realtime, extensión de GTFS para datos en tiempo real.

**JSON** *JavaScript Object Notation*, formato de intercambio de datos.

**NAP** Punto de Acceso Nacional de transporte multimodal.

**SIRI** *Service Interface for Real Time Information*, norma europea EN 15531.

**SIRI-VM** Servicio *Vehicle Monitoring* de SIRI.

**UTC** Tiempo universal coordinado.

**WGS84** *World Geodetic System 1984*, sistema de referencia de las coordenadas GPS.

# Introducción

## Motivación

Quien espera un autobús en una parada no necesita saber a qué hora *debería* pasar, sino cuánto falta para que pase *de verdad*. La diferencia entre ambas cosas es el objeto de esta memoria. En una red interurbana como Bizkaibus, con líneas que combinan calles congestionadas, carreteras y autopistas, esa diferencia puede ser de muchos minutos y cambia con la hora del día, el día de la semana y el tramo concreto del recorrido.

### *Esperar un Autobús Interurbano*

En una línea urbana que pasa cada cinco minutos, el viajero no necesita saber a qué hora llega el autobús: va a la parada y espera, en promedio, la mitad del intervalo. En una red interurbana eso no funciona, porque los intervalos son largos. En el horario de Bizkaibus de un miércoles laborable (28 de octubre de 2026), entre las 7:00 y las 21:00, la mitad de las combinaciones de línea y parada tienen, como mucho, un autobús por hora: el intervalo mediano es de 60 minutos, el 68 % de las combinaciones tiene un intervalo mediano de una hora o más, el 87 % de media hora o más, y solo el 3 % pasa cada cuarto de hora o con más frecuencia.[1]

Con esos intervalos, el viajero planifica: decide a qué hora salir de casa a partir de la hora a la que cree que pasará el autobús. Si llega tarde, aunque sea por un minuto, pierde el autobús y espera el siguiente, a menudo una hora. Si llega con mucho margen, espera en una parada que con frecuencia está en el arcén de una carretera, sin marquesina ni información. Cuanto menos fiable es la hora que tiene, más margen deja, y más tiempo pierde.

### *Lo que Cuesta No Saber Cuándo Llega*

La literatura sobre transporte público ha medido el efecto de esta incertidumbre. Los viajeros que esperan sin información perciben su espera como más larga de lo que es (Mishalani et al., 2006). En Seattle, los usuarios de una aplicación con llegadas en tiempo real no percibían su espera como más larga que la medida, a diferencia de quienes no la usaban, y además

[1] Calculado sobre las 4 932 combinaciones de línea y parada con servicio ese día, con el intervalo mediano entre salidas consecutivas de la misma línea en la misma parada.

esperaban menos (Watkins et al., 2011). Una revisión de la literatura sobre los beneficios de la información en tiempo real recoge, entre otros, menos tiempo de espera, menos tiempo total de viaje, más satisfacción, más sensación de seguridad y, en algunos estudios, más uso del transporte público (Brakewood & Watkins, 2019).

Estos beneficios dependen de que la información sea *precisa*. Una hora que se equivoca no solo no ayuda: lleva a perder el autobús o a esperar más que sin ella.

### *El Horario Publicado Se Desvía en los Dos Sentidos*

Las horas de paso por las paradas intermedias que publica el horario de Bizkaibus son, como se explica en la sección «El Problema», una interpolación a velocidad constante entre cabeceras. Son las que muestra cualquier aplicación que use el horario oficial en formato GTFS sin otra corrección. Para saber cuánto se desvían de la realidad se compararon con el paso real de los autobuses por las paradas, reconstruido con su GPS (sección «El Instante Real de Paso por una Parada»): 169 308 pasos por parada de autobuses observados entre el 6 y el 31 de octubre de 2026, con el horario vigente el 30 de octubre.[2] La Tabla 1 y la Figura 1 muestran el resultado según el tiempo transcurrido desde la salida de cabecera.

**Tabla 1**

*Desvío del Paso Real Respecto a la Hora Publicada, Según el Tiempo Desde la Salida*

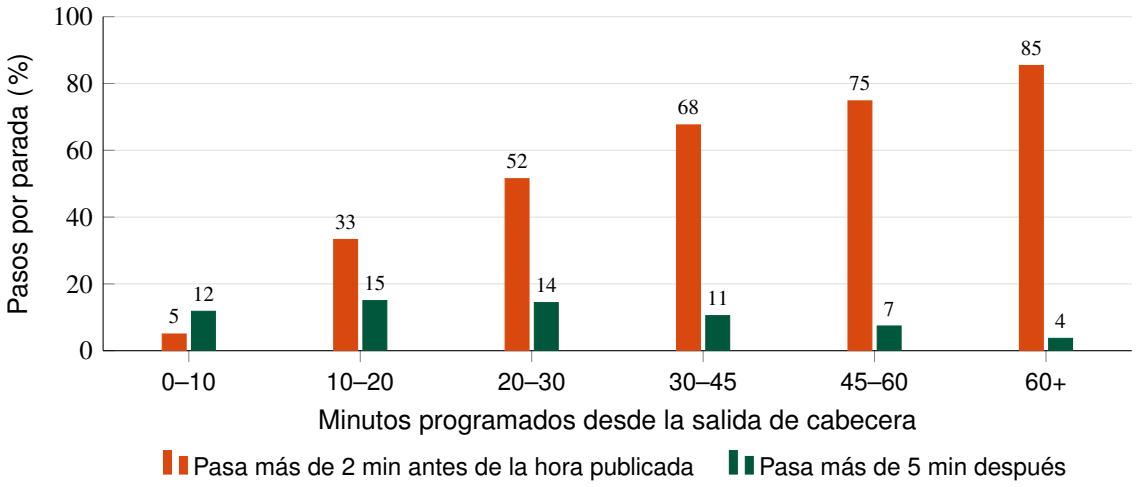
| Desde la cabecera | Pasos | Desvío mediano | Más de 2 min antes | Más de 5 min después | Dentro de ±2 min |
|---|---|---|---|---|---|
| 0–10 min | 29 115 | +1,3 min | 5,0 % | 11,8 % | 57,0 % |
| 10–20 min | 27 408 | −0,2 min | 33,3 % | 15,0 % | 34,9 % |
| 20–30 min | 28 449 | −2,2 min | 51,5 % | 14,4 % | 23,6 % |
| 30–45 min | 35 560 | −5,6 min | 67,6 % | 10,5 % | 14,6 % |
| 45–60 min | 25 931 | −8,6 min | 74,8 % | 7,4 % | 11,6 % |
| 60 min o más | 22 845 | −13,6 min | 85,4 % | 3,7 % | 8,0 % |
| Todos | 169 308 | −2,4 min | 52,1 % | 10,7 % | 25,3 % |

*Nota.* El tiempo se cuenta en minutos programados desde la salida de cabecera. Un desvío negativo significa que el autobús pasa *antes* de la hora publicada.

[2] Se descartan 521 pasos (el 0,3 %) con más de 45 minutos de diferencia, que suelen indicar un cambio de horario entre la fecha de la observación y la del horario usado. El cálculo está en `referencia/desvio-horario.py`.

**Figura 1**

*Desvío del Paso Real Según el Tiempo Programado Desde la Salida*




*Nota.* Cerca de la cabecera el autobús va algo retrasado; a medida que avanza, pasa cada vez más antes de la hora publicada, porque el horario reparte el tiempo a velocidad constante y el autobús recorre los tramos de carretera más deprisa.

El resultado es el que predice la interpolación. Cerca de la cabecera el autobús va algo retrasado (desvío mediano de $+1{,}3$ min en los diez primeros minutos). A partir de ahí se adelanta cada vez más al horario: a media hora de la salida, la mitad de los pasos ocurre más de dos minutos antes de la hora publicada, y pasada la hora, el 85 %, con un adelanto mediano de casi catorce minutos. En fin de semana el adelanto es mayor (se adelantan más de dos minutos el 60 % de los pasos, frente al 50 % en laborables), y de las 69 líneas con al menos 500 pasos observados, en 39 más de la mitad de los pasos se adelantan más de dos minutos. Solo una cuarta parte de los pasos cae dentro de $\pm 2$ minutos de la hora publicada.

Para el viajero, eso significa dos riesgos de signo contrario:

- **Perder el autobús.** Quien se guiara por la hora publicada y llegara a la parada dos minutos antes encontraría que el autobús ya ha pasado en el 52 % de los casos. Con un intervalo típico de una hora, cada autobús perdido es una hora de espera.
- **Esperar de más.** Cuando el autobús todavía no ha pasado, esa misma persona espera una

mediana de 3,8 minutos, pero una de cada diez veces más de diez minutos, y el 10,7 % de los pasos llega más de cinco minutos tarde.

Estas cifras no son un juicio sobre la puntualidad del servicio: las horas intermedias del GTFS no son un compromiso del operador (sección «El Problema»), sino una aproximación. Son, sin embargo, las horas que ve quien consulta el horario en una aplicación que no las corrige, y por eso el problema merece una solución.

### *Este Trabajo*

Bizkaibus+ es una aplicación web independiente que muestra, para cada parada, las próximas salidas con su hora estimada. Este documento describe el algoritmo que calcula esas horas: de dónde salen los datos, qué se hace con ellos, por qué se hace así y cómo se ha comprobado que funciona. El objetivo es que el método pueda estudiarse, criticarse y replicarse en cualquier otra ciudad y en cualquier lenguaje de programación.

## El Problema

Los datos disponibles condicionan todo el diseño. Bizkaibus publica dos fuentes abiertas, que son las únicas que usa este trabajo:

1. Un **horario programado** en formato GTFS (MobilityData, s.f.-b), publicado en el Punto de Acceso Nacional de transporte multimodal (Ministerio de Transportes y Movilidad Sostenible, s.f.): líneas, viajes, paradas y la hora de paso de cada viaje por cada parada, junto con el trazado geográfico (*shape*) de cada recorrido.

2. Un **feed en tiempo real** en formato SIRI Vehicle Monitoring (Comité Europeo de Normalización [CEN], 2022), actualizado una vez por minuto, que para cada autobús en servicio indica el viaje que realiza, su posición GPS, la hora a la que se tomó esa posición y el número de orden de su próxima parada.

Dos características de estas fuentes descartan el enfoque habitual de sumar un retraso al horario:

- **El feed no publica retrasos.** El elemento `Delay` de SIRI no aparece, y el feed alternativo de actualizaciones de viaje declara siempre un retraso nulo (`PT0S`) con una marca de tiempo inválida.

- **El horario intermedio es una interpolación.** Las paradas intermedias del GTFS llevan `timepoint=0`: sus horas se obtienen repartiendo el tiempo entre cabeceras a velocidad constante. Un autobús que recorre 18 km de autopista en 15 minutos puede tener, para ese tramo, entre 34 y 49 minutos programados. En mitad del recorrido, el horario absoluto puede ir quince o veinte minutos por detrás del autobús sin que este se haya adelantado en ningún sentido operativo.

Así, ni existe un retraso que sumar ni el horario absoluto es una referencia fiable. Lo único vivo y fiable es la posición.

**Objetivos**

1. Estimar, para cada viaje en servicio y cada parada que le quede por recorrer, la hora de llegada con el menor error posible, a partir de la posición del vehículo.

2. Seguir dando una estimación útil cuando el GPS falla o no existe, sin inventar datos.

3. Aprender de forma automática el comportamiento real de la red, sin reglas escritas a mano sobre la velocidad de los autobuses.

4. Medir la precisión contra el paso real de los autobuses por las paradas, y adoptar un cambio en el modelo solo si esa medida mejora.

5. Documentarlo todo con la precisión suficiente para que otra persona pueda reconstruirlo sin leer el código.

**Contribuciones**

- Un método de predicción por **diferencias de horario** que neutraliza el error de forma de un horario interpolado (capítulo «Modelo del Tiempo de Recorrido», proposición 2).

- Un procedimiento de **localización** del GPS sobre el trazado oficial robusto ante recorridos que vuelven sobre sí mismos (capítulo «Emparejamiento y Localización»).
- Un **modelo de tiempos por tramo** con estimadores robustos, contracción hacia la mediana del tramo y una cascada de respaldo para los tramos sin datos (capítulo «Modelo del Tiempo de Recorrido»).
- Una arquitectura de **tres capas con abstención**, que elige en cada predicción la mejor fuente fiable y nunca afirma un retraso que no puede sostener, con una **corrección por viaje** para cuando no hay ningún dato del vehículo (capítulo «Predicción en Tres Capas»).
- Una **revisión de los datos de entrada** en la construcción del horario: los trazados dibujados al revés se dan la vuelta y, en las redes que no publican los metros de cada parada, estos se miden sobre el trazado (capítulo «Fuentes y Modelo de Datos»).
- Un ciclo de **aprendizaje continuo** con validación cruzada y reglas de escritura conservadoras (capítulo «Aprendizaje Continuo»), un protocolo de **evaluación** por días que reproduce lo que ve el usuario, y una **comparación** con filtros de Kalman y redes neuronales hecha con ese mismo protocolo (capítulo «Resultados»).
- **Vectores de prueba** generados con el código de producción y una **implementación de referencia** independiente que los reproduce exactamente (Apéndices D y E).

### Visión General del Sistema

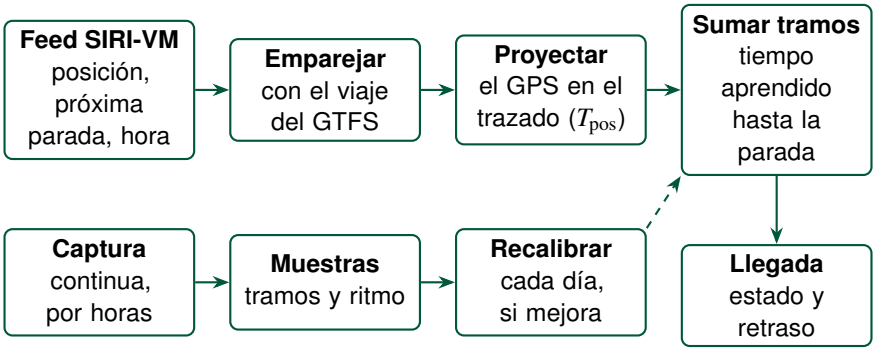
La Figura 2 resume el recorrido de un dato desde el feed hasta la pantalla. Cada paso tiene su capítulo.

### Convenciones

- **Tiempo.** Todas las horas son enteros en segundos desde la medianoche de la zona `Europe/Madrid` del día de servicio. Siguiendo a GTFS, pueden superar 86 400: las 25:10 son 90 600 s. El feed y el reloj dan horas del día en curso; para un viaje del día anterior se

**Figura 2**

*Recorrido de un Dato Desde el Feed Hasta la Pantalla*




*Nota.* Arriba, la predicción en cada petición (capítulos «Emparejamiento y Localización» a «Predicción en Tres Capas»). Abajo, el aprendizaje fuera de línea que alimenta los tiempos por tramo (capítulo «Aprendizaje Continuo»).

les suman 86 400 s (sección «Viajes que Pasan de las 24:00»). Las duraciones se expresan en segundos salvo que se diga otra cosa.

- **Distancia.** En metros. Las coordenadas geográficas, en grados decimales (WGS84).
- **Redondeo.** $\mathrm{rnd}(x)$ redondea al entero más próximo con las mitades *alejándose del cero* ($\mathrm{rnd}(2{,}5) = 3$, $\mathrm{rnd}(-2{,}5) = -3$). Es el comportamiento de PHP y difiere del «redondeo del banquero» de otros lenguajes; replicarlo es necesario para obtener los mismos resultados.
- **Mediana.** Salvo que se indique otra cosa, med es la *mediana baja*: con los valores ordenados $x_{(0)} \le \cdots \le x_{(n-1)}$, $\mathrm{med}(x) = x_{(\lfloor (n-1)/2 \rfloor)}$. En general, el percentil $p$ es $P_p(x) = x_{(\lfloor p\,(n-1) \rfloor)}$, sin interpolar.
- **Acotación.** $\mathrm{clamp}(x,a,b) = \mathrm{máx}(a, \mathrm{mín}(b,x))$.
- **Nombres en el código.** Cuando conviene, se indica entre paréntesis y en `monoespaciada` el nombre de la constante o función del código, para facilitar la correspondencia.

**Estructura del Documento**

El documento sigue la estructura habitual de un trabajo de investigación (introducción, método, resultados y discusión) y el formato de las normas APA en su séptima edición. Este capítulo y el capítulo «Marco Teórico y Estado del Arte» sitúan el trabajo respecto a la literatura de predicción de llegadas y a las herramientas estadísticas que usa. Los cinco siguientes describen el método completo: las fuentes de datos (capítulo «Fuentes y Modelo de Datos»), la localización del vehículo (capítulo «Emparejamiento y Localización»), el modelo del tiempo de recorrido (capítulo «Modelo del Tiempo de Recorrido»), la predicción en tres capas (capítulo «Predicción en Tres Capas») y el aprendizaje de los parámetros (capítulo «Aprendizaje Continuo»). El capítulo «Resultados» presenta la precisión medida, en una muestra con las tres capas, a lo largo de 17 días y frente a otros métodos; el capítulo «Discusión» la interpreta y discute sus limitaciones y su aplicación a otras redes, y el capítulo «Conclusiones y Líneas Futuras» cierra con las conclusiones y las líneas futuras. Tras las referencias, los apéndices reúnen las constantes, el pseudocódigo completo, los formatos de datos, los vectores de prueba, la implementación de referencia y un glosario.

# Marco Teórico y Estado del Arte

Este capítulo sitúa el método en el contexto de la predicción de llegadas de transporte público y presenta las herramientas estadísticas en que se apoya. Se centra en los trabajos y las técnicas que explican de dónde vienen las ideas del método y por qué se eligieron unas y no otras.

## Predicción de la Hora de Llegada

Predecir cuándo llegará un vehículo a un punto de su recorrido es un problema clásico de los sistemas inteligentes de transporte. Los enfoques de la literatura pueden agruparse en cuatro familias.

### *Medias Históricas*

Se estima el tiempo de recorrido entre dos puntos con el valor típico observado en el pasado para esa hora y ese día. Son sencillas, estables y fáciles de explicar, pero no reaccionan a lo que le ocurre al vehículo concreto.

### *Regresión y Modelos de Series Temporales*

Relacionan el tiempo de recorrido con variables explicativas (hora, distancia, número de paradas, demanda) o con su propia historia reciente. Exigen elegir y mantener esas variables.

### *Filtros de Kalman*

Combinan una predicción a priori con la observación más reciente, ponderando cada una por su incertidumbre (Kalman, 1960). Se han usado para predecir llegadas y salidas a partir de datos de localización automática de vehículos (AVL) y de conteo de pasajeros (Shalaby & Farhan, 2004).

### *Redes Neuronales*

Aprenden la relación entre la posición y el tiempo restante a partir de muchos datos (Chien et al., 2002; R. Jeong & Rilett, 2005). Suelen ser precisas cuando hay abundancia de datos homogéneos, pero son difíciles de auditar y de explicar a quien usa el sistema.

### *Trabajos Recientes*

Dos revisiones recientes recogen el estado del campo: Kumar et al. (2025), de punta a punta (datos, modelos, discretización del recorrido y medidas de evaluación), y Singh y Kumar (2022), para los modelos de inteligencia artificial. Partir el viaje en tramos entre paradas consecutivas y predecir cada uno es habitual (Gal et al., 2017), y la variabilidad del tiempo de recorrido depende mucho de la hora del día (Mazloumi et al., 2010), lo que justifica aprender por franjas. Lin y Zeng (1999) compararon algoritmos con GPS que usan el horario para un servicio rural con frecuencias bajas, un caso parecido al de los autobuses interurbanos. Entre los filtros de Kalman, Cathey y Dailey (2003) propusieron el esquema de seguimiento, filtrado y predicción para datos AVL, y Vanajakshi et al. (2009) lo aplicaron a autobuses en tráfico heterogéneo. Entre las redes neuronales se han usado redes recurrentes (Pang et al., 2019), redes LSTM convolucionales que predicen a la vez todos los enlaces de una línea (Petersen et al., 2019), redes de grafos con atención para datos escasos (Ma et al., 2022) y transformadores entrenados con datos abiertos (S. Jeong et al., 2024). En la predicción del tráfico, el aprendizaje profundo no siempre es la mejor técnica, y las referencias sencillas bien ajustadas importan (Manibardo et al., 2022).

Otros trabajos parten, como este, de los datos que publican los operadores. Wessel et al. (2017) reconstruyen el horario realmente cumplido a partir de un feed público de posiciones, como aquí la verdad de referencia. Cats y Loutos (2016) evaluaron las predicciones que ya se mostraban a los viajeros en Estocolmo y encontraron que subestimaban la espera, y García-Mauriño et al. (2024) corrigen los errores sistemáticos de las predicciones de los operadores de Madrid y París solo con datos de sus API públicas. Woodard et al. (2017) modelan, para viajes en coche, un efecto propio de cada viaje y recurren a parámetros agrupados cuando un enlace tiene pocas observaciones, un análogo de la corrección por viaje y de la contracción de este método.

El método de esta memoria pertenece a la primera familia en su forma de estimar tiempos (valores típicos por tramo y franja) y la combina con la observación en tiempo real de la posición,

que hace de «reinicio» de la predicción en cada actualización del feed. La elección es deliberada: con un dato de posición por minuto y sin retrasos publicados, un modelo explicable, robusto y que se puede medir tramo a tramo resultó más útil que uno de caja negra. Las ideas que se probaron y no mejoraron la medida (entre ellas, corregir con la velocidad del último minuto del vehículo, la forma más simple de filtrar) se recogen en la sección «Ideas Probadas y Descartadas». Un filtro de Kalman sobre la desviación del propio autobús en los tramos ya terminados, suavizada y olvidada con el tiempo, sí reduce un 4 % el error de las sumas de tramos (sección «Comparación con Otros Métodos»): lo que decide es qué se filtra y cuánto se recuerda (sección «Relación con Otros Enfoques»).

**Datos de Transporte: GTFS y SIRI**

GTFS (MobilityData, s.f.-b) es el formato de facto para publicar horarios programados. Describe agencias, líneas (`routes`), viajes (`trips`), paradas (`stops`), horas de paso (`stop_times`), calendarios (`calendar`, `calendar_dates`) y trazados (`shapes`). Dos campos son centrales aquí: `shape_dist_traveled`, la distancia recorrida a lo largo del trazado hasta cada parada y cada punto del trazado, y `timepoint`, que indica si la hora de paso es exacta (`1`) o aproximada por interpolación (`0`).

GTFS Realtime (MobilityData, s.f.-a) y SIRI (CEN, 2022) son los formatos más usados para datos en tiempo real. SIRI Vehicle Monitoring (SIRI-VM) publica, para cada vehículo, la actividad de su viaje: referencia del viaje (`VehicleJourneyRef`), posición (`VehicleLocation`), hora de la posición (`LocationRecordedAtTime`) y la llamada a la próxima parada (`MonitoredCall`), con su orden (`Order` o `VisitNumber`).

**Ajuste al Mapa**

Situar una posición GPS sobre una red o un recorrido se conoce como *map matching* (Quddus et al., 2007). Los métodos van desde la proyección geométrica sobre el segmento más próximo hasta modelos probabilísticos que tienen en cuenta la topología y la trayectoria. En este problema el recorrido es conocido de antemano (es el trazado del viaje que el propio feed declara), lo que reduce el problema a una proyección sobre una polilínea. La dificultad que queda

es la ambigüedad cuando el recorrido pasa dos veces por la misma calle; el capítulo «Emparejamiento y Localización» la resuelve con una ventana alrededor de la próxima parada y una regla de desempate.

Para distancias de unos pocos kilómetros basta una proyección equirrectangular local (Snyder, 1987), que trata la superficie terrestre como un plano alrededor del punto de interés.

**Estadística Robusta**

La media de una muestra es muy sensible a valores extremos: un único autobús parado veinte minutos por un accidente desplaza la media de un tramo tanto como muchos autobuses normales. Los estimadores *robustos* (Huber, 1964; Tukey, 1977) limitan esa influencia. La mediana es el más sencillo: su punto de ruptura es del 50 %, es decir, la mitad de la muestra tendría que estar contaminada para llevarla a un valor arbitrario. El método usa medianas en todos sus agregados, y además toma un solo valor por vehículo y viaje, para que un vehículo lento visto muchas veces no pese más que uno visto una vez.

**Contracción y Modelos Jerárquicos**

Cuando se estiman muchas medias a la vez (un tiempo para cada tramo y cada franja horaria), algunas se apoyan en muy pocas observaciones y son ruidosas. James y Stein (1961) demostraron que, al estimar a la vez tres o más medias de distribuciones normales, un estimador que contrae las medias observadas hacia un punto fijo tiene un error cuadrático total esperado menor que el de usar cada media por separado. Efron y Morris (1975) mostraron su utilidad con datos reales, contrayendo cada media hacia la media común del grupo. En términos bayesianos (Gelman et al., 2013), la contracción equivale a un modelo jerárquico con *agrupamiento parcial*: cada grupo tiene su propio parámetro, pero estos se parecen entre sí, y la estimación de cada grupo es una media ponderada entre su dato propio y el valor común, con un peso que crece con el número de observaciones del grupo.

El método aplica esta idea en dos sitios: el tiempo de un tramo en una franja se contrae hacia el tiempo del tramo a cualquier hora (sección «Mediana y Contracción Hacia el Tramo a Cualquier Hora»), y el factor de ritmo se contrae hacia 1 (sección «Factor de Ritmo»).

**Validación**

Medir el error con los mismos datos con que se aprendió sobreestima la precisión. La *validación cruzada* (Hastie et al., 2009; Kohavi, 1995) separa los datos en particiones, aprende con unas y mide con otras. Cuando los datos tienen orden temporal, lo honesto es aprender con el pasado y medir con el futuro (*validación con origen móvil*, Hyndman y Athanasopoulos, 2021). El método usa ambas: una partición por recorridos (un autobús en un viaje de un día) para decidir si se escribe una nueva calibración y una partición por días para comparar reglas. La primera comparte los días entre aprendizaje y prueba, y con ellos el tráfico y la meteorología de cada día, por lo que las reglas nuevas se deciden con la segunda. Hyndman y Koehler (2006) distinguen entre las medidas de error que dependen de la escala de los datos, como el error absoluto medio, y las que no: las primeras sirven para comparar métodos sobre unos mismos datos, pero no entre conjuntos de datos con escalas distintas. Aquí todas las predicciones se miden en la misma unidad, segundos, de modo que se usan errores absolutos y porcentajes de aciertos dentro de umbrales que tienen sentido para quien espera el autobús.

## Fuentes y Modelo de Datos

### El Horario Programado

El horario oficial se publica en GTFS y se transforma, una vez al día, en una base de datos SQLite de solo lectura por red. La Tabla 2 resume las tablas que usa el motor; el esquema completo está en el Apéndice C.

**Tabla 2**

*Tablas de la Base de Datos del Horario que Usa el Motor*

| Tabla | Contenido relevante para el motor |
|---|---|
| `stops` | Paradas: identificador, nombre, latitud y longitud. |
| `service_journeys` | Viajes: línea, recorrido (`journey_pattern_id`), número de viaje, calendario, hora de salida de cabecera y trazado (`shape_id`). |
| `passing_times` | Para cada viaje y cada parada en orden (`seq_order`, desde 1): llegada y salida programadas en segundos y `dist_m`, la distancia a lo largo del trazado. |
| `shapes` | Trazado de cada recorrido como lista JSON de puntos $[\varphi, \lambda, \sigma]$: latitud, longitud y distancia recorrida. |
| `trip_aliases` | Correspondencia entre cada `trip_id` original del GTFS y el viaje fusionado al que pertenece. |
| `journey_patterns` | Recorridos: secuencias de paradas compartidas por varios viajes. |

La columna `dist_m` de `passing_times` es el campo `shape_dist_traveled` de `stop_times.txt`, y la tercera componente de cada punto del trazado es el `shape_dist_traveled` de `shapes.txt`. Ambas se miden sobre el mismo trazado y en la misma unidad (o se corrigen al construir la base hasta que lo hacen; sección «Trazados que No Casan con sus Paradas»), lo que permite pasar de un punto del trazado a la parada que tiene delante o detrás (sección «Con Trazado»). Los puntos del trazado se guardan con 7 decimales en las coordenadas y 2 en la distancia. Si la red no tiene trazados, la base no tiene la tabla `shapes` y el motor trata `dist_m` como desconocida; esto tiene consecuencias en el respaldo por longitud (sección «Respaldo por Longitud del Tramo»).

### Trazados que No Casan con sus Paradas

Lo anterior supone que el trazado y las distancias de las paradas casan, y en el GTFS de Bizkaibus no siempre es así. Con el horario del 3 de noviembre de 2026, 55 de los 440 trazados que usan los viajes están dibujados del final al principio: la primera parada del viaje queda junto al último punto del trazado y la última junto al primero. Afectan a 670 de los 6 130 viajes fusionados, repartidos en 25 líneas. En otros trazados el sentido es el bueno, pero los metros de `shape_dist_traveled` no caen sobre la parada: unos van desplazados 200 o 300 m, y en otros la diferencia crece hasta varios kilómetros al final del viaje. En total, 1 417 viajes tenían alguna parada a más de 200 m del punto del trazado que marcan sus metros.

El efecto en el motor es grave. La ventana de búsqueda (sección «Ventana de Búsqueda») se define con los metros de las paradas, así que, con el trazado al revés, el autobús se busca en la otra punta del recorrido: un autobús a 9 m de su parada queda «a kilómetros» y el pasillo de 50 m lo descarta. Esos autobuses no se situaban con el GPS y, como el aprendizaje sale de las posiciones, tampoco aprendían: los viajes con el trazado al revés tenían el 47 % de sus tramos con tiempo aprendido, frente al 95 % del resto.

La construcción de la base corrige las dos cosas antes de publicarla (el programa, `scripts/build/shapes.php`, es parte del código privado de la aplicación; el algoritmo completo es este):

1. *Sentido*. Un trazado está al revés si, en la mayoría de sus viajes, la suma de las distancias de la primera parada al último punto y de la última al primero es menor que la de cada parada a su extremo. Se le da la vuelta: los puntos en orden inverso y $\sigma' = \sigma_{\text{máx}} - \sigma$.

2. *Metros de las paradas*. Si el trazado se ha dado la vuelta, o si alguna parada cae a más de 100 m del punto del trazado que marcan sus metros, se miden de nuevo las de ese viaje: cada parada se proyecta sobre el trazado a partir de la anterior y, como mucho, hasta tres veces la distancia en línea recta entre las dos más 500 m. Entre dos pasos del autobús por la misma zona gana el primero, salvo que el otro quede 30 m más cerca.

Con el horario de ese día se dieron la vuelta los 55 trazados y se midieron de nuevo las paradas de 2 905 viajes; los viajes con alguna parada a más de 200 m de sus metros bajaron de 1 417 a 26. La corrección es general: se aplica al construir cualquier red, porque nada en el formato GTFS impide estos errores.

Tampoco obliga a publicar los metros. El GTFS de Lurraldebus, Bilbobus, Tuvisa y La Unión no trae los de las paradas, y el de Lurraldebus, La Unión y Bilbobus tampoco los del trazado (el de Bilbobus los deja a 0 en todos los puntos menos el último). Sin metros, el motor sitúa al autobús con la recta entre paradas (sección «Sin Trazado»). Por eso la construcción los mide: los del trazado, sumando la distancia de cada punto al siguiente, y los de las paradas, como en el paso 2.

Los feeds de Lurraldebus y Bilbobus, las dos redes con motor, no traían vehículos el 3 de noviembre de 2026, así que la medida se hizo con Bizkaibus: se le quitaron los metros del GTFS y se compararon tres versiones del mismo horario con las mismas capturas, 36 bloques de una hora del 2 y el 3 de noviembre con 2,08 millones de llegadas de paso conocido (tabla 3). Los metros medidos caen a una mediana de 21 m de los del GTFS, el 95 % a menos de 103 m, y aciertan casi como ellos. La recta entre paradas sitúa con el GPS un 5,6 % menos de llegadas y acierta 1,8 puntos menos dentro de $\pm 2$ min. Sin tiempos aprendidos, como en una red recién activada, las tres versiones quedan a menos de medio punto: entonces lo que falta es aprender, no situar.

En las cuatro redes la medida es coherente: los metros nunca bajan de una parada a la siguiente y tienen todas sus paradas a menos de 100 m de su punto del trazado el 99,5 % de los viajes de Bilbobus, el 95,9 % de los de Lurraldebus, el 99,6 % de los de Tuvisa y el 98,9 % de los de La Unión. Medirlos alarga la construcción de cada red entre 0 y 9 s.

**Fusión de Variantes y Alias de Viaje**

Un mismo servicio aparece en el GTFS como varios `trip_id` que solo se diferencian en el calendario (por ejemplo, uno para los lunes y otro para el resto de laborables). Para no mostrar salidas duplicadas, la construcción de la base agrupa esos `trip_id` en un único viaje. Dos

**Tabla 3**

*Llegadas de Bizkaibus Según los Metros de las Paradas*

| | Con tiempos aprendidos | | | | Sin aprender | |
|---|---|---|---|---|---|---|
| Metros de las paradas | Con GPS | ±1 min | ±2 min | Error | ±2 min | Error |
| Del GTFS | 97,4 % | 55,6 % | 78,4 % | 92 s | 51,5 % | 192 s |
| Medidos sobre el trazado | 97,3 % | 55,3 % | 78,2 % | 93 s | 51,6 % | 192 s |
| Ninguno (recta entre paradas) | 91,9 % | 54,0 % | 76,6 % | 105 s | 51,2 % | 194 s |

*Nota.* Las mismas 2,08 millones de llegadas en las tres versiones. «Del GTFS»: los metros del GTFS con la corrección de la sección anterior. «Con GPS»: las que el motor sitúa con la posición del autobús (capa 1). «Error»: error absoluto medio de la hora de llegada. «Sin aprender»: sin tiempos por tramo, factores de ritmo ni correcciones por viaje.

`trip_id` se fusionan cuando coinciden en:

1. la línea (`route_id`) y el número de viaje;
2. la secuencia de paradas (la clave del recorrido es un resumen MD5 de los identificadores de parada en orden);
3. el grupo de calendario;
4. la *forma temporal*: para cada parada, la llegada relativa a la salida de cabecera y el tiempo de espera en la parada (un resumen MD5 de esa lista);

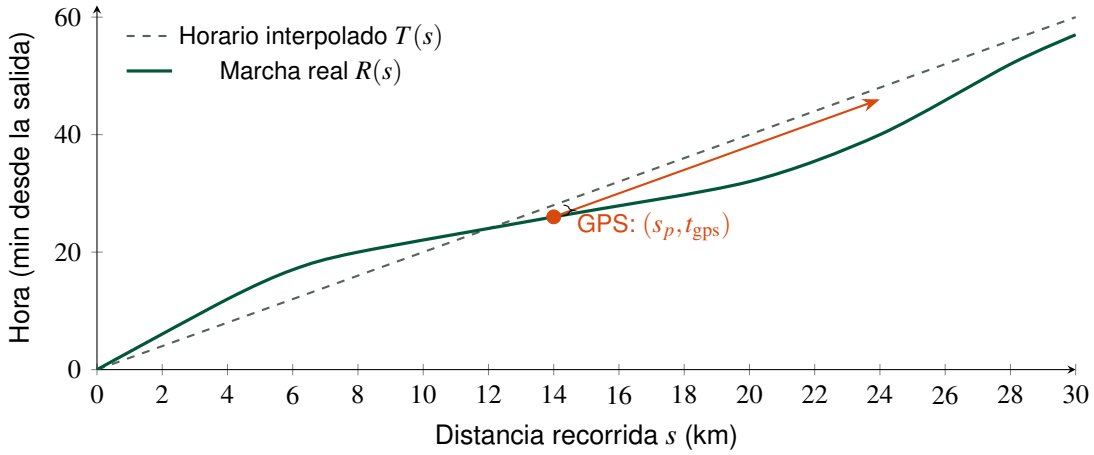
y, además, sus salidas de cabecera son consecutivas con una separación no mayor de 90 s (`iDepartureClusterGapSeconds`). El viaje resultante reúne los días de sus miembros (unión de las máscaras de días de la semana y de las fechas incluidas y excluidas). Cada `trip_id` miembro queda registrado en `trip_aliases` apuntando al viaje fusionado. Esa tabla es la que permite emparejar exactamente el feed en tiempo real con el horario (sección «Emparejamiento con el Horario»): el feed usa los `trip_id` originales.

### Por Qué el Horario No Sirve Como Referencia Absoluta

En el GTFS de Bizkaibus las paradas intermedias llevan `timepoint=0`. Sus horas no son compromisos del operador, sino una interpolación lineal en la distancia entre las horas de las cabeceras. La Figura 3 muestra la consecuencia. El horario es la recta $T(s)$: una velocidad constante. La marcha real $R(s)$ es más lenta en la ciudad y mucho más rápida en la autopista. A mitad de recorrido, $R$ y $T$ pueden estar separadas muchos minutos sin que el autobús vaya «adelantado» en ningún sentido operativo.

**Figura 3**

*Distancia y Tiempo de un Viaje Ilustrativo*




*Nota.* La recta discontinua es el horario interpolado; la curva, la marcha real. Desde el punto GPS, el método traza una paralela al horario (flecha) y la corrige tramo a tramo con los tiempos aprendidos.

### El Feed en Tiempo Real

El feed SIRI-VM de Bizkaibus es un documento XML público que se regenera aproximadamente una vez por minuto y contiene unos 250 vehículos en hora punta. El ritmo ha variado: en octubre de 2026 salía una versión cada 55 s, y el 5 de noviembre, cada 64 s de mediana, con cada posición 66 s después de tomarse (sección «Llegando sin Confirmar el Paso»). El motor lo descarga con un tiempo máximo de 8 s y lo guarda en caché 25 s. Para cada

`VehicleActivity` extrae los campos de la Tabla 4.

**Tabla 4**

*Campos que el Motor Lee del Feed SIRI-VM*

| Símbolo | Elemento SIRI | Uso |
|---|---|---|
| $\rho$ | `MonitoredVehicleJourney/VehicleJourneyRef` | Referencia del viaje; igual al `trip_id` del GTFS. |
| $\ell,\ \nu,\ d_0^{\text{feed}}$ | (extraídos de $\rho$) | Línea (sin ceros a la izquierda, como en el horario), número de viaje y salida de cabecera, por la expresión (1). |
| $v$ | `VehicleRef` | Identificador del vehículo. |
| $n$ | `MonitoredCall/Order` (o `VisitNumber`) | Orden de la próxima parada en el viaje. |
| $(\varphi, \lambda)$ | `VehicleLocation/Latitude`, `Longitude` | Posición GPS. |
| $t_{\text{gps}}$ | `LocationRecordedAtTime` (o `RecordedAtTime`) | Hora de la posición. |
| $t_{\text{rec}}$ | `VehicleActivity/RecordedAtTime` | Hora del registro del vehículo. |
| $\delta_{\text{decl}}$ | `Delay` (duración ISO 8601) | Retraso declarado; ausente en Bizkaibus (0). |

***Referencia del Viaje***

Solo se aceptan las actividades cuya referencia encaja con

```
^trp_[A-Za-z]+(\d+)_(\d+)_[A-Za-z0-9]+_(\d+)
```
(1)

cuyos tres grupos son la línea $\ell$, el número de viaje $\nu$ y la salida de cabecera $d_0^{\text{feed}}$ en segundos. Por ejemplo, `trp_A3414_808_OP9LSEPT_54000_...` da $\ell = 3414$, $\nu = 808$ y $d_0^{\text{feed}} = 54\,000$ (15:00:00). Las demás se ignoran. La línea se guarda sin ceros a la izquierda, como el `route_id` del horario: `trp_A0651_737_...` da $\ell = 651$, no `0651` (el `0651` es el relleno del nombre público `A0651`, que el horario guarda aparte en `route_short_name`).

***Horas***

Las horas del feed son fechas ISO 8601 con zona. Se convierten a `Europe/Madrid` y a segundos desde medianoche, $3600H + 60M + S$. Si la fecha convertida no es la fecha de hoy, el

valor se descarta: una posición de ayer no dice nada del viaje de hoy. Si la hora de la posición no es válida, también se descartan la latitud y la longitud. Una entrada sin ninguna hora de hoy, ni $t_{\text{gps}}$ ni $t_{\text{rec}}$, no se usa: ni en la predicción, ni en el registro de pasos, ni en el mapa (`RealtimeMatcher::hasTime`). Mientras el feed se actualiza, todas la traen: 1 471 de 1 471 lecturas del 6 de noviembre de 2026 entre las 23:00 y las 23:59, la última hora antes de que se detenga. De 23:58 a 05:00, en cambio, el feed de Bizkaibus no se actualiza: repite la lectura de las 23:58 con sus horas de entonces, aunque haya autobuses nocturnos en servicio (capturas de las noches del 31 de octubre y del 6 de noviembre de 2026). Pasada la medianoche, ninguna de esas entradas tiene hora de hoy (sección «Viajes que Pasan de las 24:00»).

***Retraso Declarado***

El retraso, cuando existe, es una duración $\pm$`PT`$h$`H`$m$`M`$s$`S`; se convierte a $\pm(3600h + 60m + s)$. En Bizkaibus no aparece y el motor de autobús no lo usa (sección «Redes que Publican Retraso o Predicción»).

Las entradas se agrupan por la clave «$\ell|v$», porque puede haber varios vehículos con la misma línea y número de viaje en distintos días de calendario.

***Lo que el Feed No Dice***

El feed solo trae posiciones: no publica cancelaciones, ni qué vehículo hará cada viaje, ni refuerzos fuera del horario. Con las capturas del 3 de noviembre de 2026 se midieron cuatro consecuencias:

- *Viajes sin autobús.* En cuatro momentos del día, 947 viajes debían estar en marcha según el horario y 187 (el 20 %) no tenían autobús en el feed. Siguiendo esos viajes en las capturas, la mayoría no falta: el 12 % del total ya había llegado al final antes de su hora (el horario deja margen al final del recorrido), un 2 % aparece después con unos 15 min de retraso (mediana), otro 2 % desaparece un rato y vuelve, y otro 2 % desaparece a mitad y no vuelve. Solo el 2 % no aparece en todo el día; desde fuera no se distingue un autobús sin GPS de un viaje que no se hizo.

- *El número de viaje no es el vehículo.* De los viajes seguidos de una misma clave «$\ell|v$», solo

el 11 % los hizo el mismo autobús, y el GTFS no trae `block_id`. No se puede saber de antemano qué autobús hará el próximo viaje, ni arrastrar a él el retraso del anterior (sección «Ideas Probadas y Descartadas»).

- *Orden y parada.* El orden $n$ y la parada `StopPointRef` coinciden en el 97,9 % de los casos. En unas pocas líneas (3129, 2610, 2611 y 652) el operador cuenta una o varias paradas más que el GTFS; el motor usa $n$, y fiarse de la parada no mejoró la medida.

- *Dónde está el autobús.* El feed no dice si un autobús está en una parada. `StopPointRef` es siempre la próxima: mientras espera en la primera parada de su viaje, apunta a la segunda (359 de 360 casos en nueve capturas del 3 de noviembre). `ArrivalStatus` vale `arrived` en los 2 142 autobuses mirados, parados o en marcha, y no hay `PreviousCalls`. Solo el GPS lo dice (sección «En la Cabecera, Antes de Salir»).

**Notación**

**Definición 1** (Viaje)**.** *Un viaje $J$ es una secuencia de $N \geq 2$ paradas $p_1, \ldots, p_N$. Para cada $i$ se conocen la llegada y la salida programadas $a_i \leq d_i$, las coordenadas $(\varphi_i, \lambda_i)$, el identificador $u_i$ de la parada y, opcionalmente, la distancia $s_i$ a lo largo del trazado, con $s_1 \leq s_2 \leq \cdots \leq s_N$. El orden de la parada $p_i$ es $i$ (`seq_order`). La salida de cabecera es $d_1$. El trazado, si existe, es una polilínea $\Sigma = \big((\varphi_j, \lambda_j, \sigma_j)\big)_{j=1}^{M}$ con $\sigma_j$ no decreciente.*

**Definición 2** (Tramo)**.** *El tramo $i$ de $J$ es el par de paradas consecutivas $(p_i, p_{i+1})$. Su identificador es «$u_i > u_{i+1}$», su duración programada es $a_{i+1} - a_i$ y su longitud, si se conocen las distancias, es $s_{i+1} - s_i$. Dos viajes de líneas distintas que pasan por las mismas dos paradas comparten el tramo.*

**Definición 3** (Entrada del feed)**.** *Una entrada es la tupla $e = (\rho, \ell, \nu, d_0^{\text{feed}}, v, n, \varphi, \lambda, t_{\text{gps}}, t_{\text{rec}})$ de la Tabla 4, donde cualquiera de $n, \varphi, \lambda, t_{\text{gps}}, t_{\text{rec}}$ puede faltar.*

**Definición 4** (Franja)**.** *La franja de una fecha $D$ y un instante $t$ es el par*

$$\text{franja}(D,t) = \big(\mathit{tipo}(D),\ h(t)\big),$$

$$\mathit{tipo}(D) = \begin{cases} \texttt{fin_semana} & \text{si } D \text{ es sábado o domingo}, \\ \texttt{laborable} & \text{en otro caso,} \end{cases}$$

*con* $h = \lfloor t/3600 \rfloor \bmod 24$ *y*

$$h(t) = \begin{cases} \texttt{madrugada} & 0 \le h < 6 \\ \texttt{punta_manana} & 6 \le h < 9 \\ \texttt{manana} & 9 \le h < 13 \\ \texttt{mediodia} & 13 \le h < 16 \\ \texttt{punta_tarde} & 16 \le h < 20 \\ \texttt{noche} & 20 \le h < 24. \end{cases}$$

*Se escribe como texto «*`tipo.nombre`*», por ejemplo* `laborable.punta_manana`*. Hay, por tanto, doce franjas.*

**Observación 1.** *La fecha que se usa es la del día de servicio, no la del reloj. Un instante de 25:00 (*$90\,000$ *s) de un miércoles es la franja* `laborable.madrugada` *del miércoles, aunque en el calendario ya sea jueves. En los tiempos por tramo, los festivos entre semana se tratan como laborables: se comprobó que su tráfico se parece más al de un laborable que al de un fin de semana (sección «Ideas Probadas y Descartadas»). La capa 3 es la excepción: para ella el festivo cuenta como fin de semana en todo (franja de los tramos, factor de ritmo, parte aprendida y corrección por viaje), porque así se midió mejor (sección «Capa 3: Horario Corregido»).*

**Definición 5** (Hora actual)**.** $t_{\text{now}}$ *es la hora del servidor en el momento de responder, en segundos desde la medianoche de Madrid del día actual. Para un viaje del día anterior se usa* $t_{\text{now}} + 86\,400$*,*

*en las horas de su horario (sección «Viajes que Pasan de las 24:00»).*

## Emparejamiento y Localización

Antes de predecir nada hay que responder dos preguntas: qué viaje del horario está haciendo cada autobús del feed, y en qué punto de ese viaje se encuentra. La respuesta a la segunda es una hora programada, $T_{\text{pos}}$: la hora a la que el horario sitúa al autobús en el lugar donde el GPS dice que está.

### Emparejamiento con el Horario

#### *Emparejamiento Exacto*

La referencia $\rho$ del feed es un `trip_id` del GTFS. Con la tabla `trip_aliases` (sección «Fusión de Variantes y Alias de Viaje») se obtiene directamente el viaje fusionado $J = \text{alias}(\rho)$. Al construir el emparejador se consultan a la vez los alias de todas las referencias del feed; para cada viaje $J$ se guarda la *primera* entrada del feed cuyo alias es $J$. En las pruebas con el feed real, el 100 % de los autobuses se emparejó así.

#### *Emparejamiento de Respaldo*

Si la base de datos no tiene la tabla de alias, o el viaje no aparece en ella, se buscan las entradas con la misma clave «$\ell|\nu$» que el viaje y se elige la de salida de cabecera más próxima:

$$e^{*} = \arg\min_{e \in C} \left| d_0^{\text{feed}}(e) - d_1(J) \right| \quad \text{sujeto a} \quad \left| d_0^{\text{feed}}(e) - d_1(J) \right| \leq 180 \text{ s}, \tag{2}$$

donde $C$ excluye las entradas que ya tienen un emparejamiento exacto con otro viaje. Si hay empate, gana la primera encontrada; si ninguna cumple la tolerancia (`MATCH_TOLERANCE_SECONDS`), el viaje no tiene datos en vivo.

### Proyección de un Punto Sobre un Segmento

Toda la localización se basa en una operación: proyectar la posición del autobús sobre un segmento rectilíneo $AB$ (entre dos puntos del trazado, o entre dos paradas).

***Plano Local***

Se usa una proyección equirrectangular centrada en el autobús, que queda en el origen. Un punto $Q = (\varphi_Q, \lambda_Q)$ pasa a

$$x_Q = (\lambda_Q - \lambda) \cdot \cos\varphi \cdot 111\,320, \qquad y_Q = (\varphi_Q - \varphi) \cdot 111\,320, \tag{3}$$

en metros, con $(\varphi, \lambda)$ la posición del autobús (`METERS_PER_DEGREE`).

***Proyección***

Sean $\mathbf{a} = (x_A, y_A)$ y $\mathbf{b} = (x_B, y_B)$ y $\mathbf{u} = \mathbf{b} - \mathbf{a}$.

$$f = \begin{cases} \operatorname{clamp}\left(-\dfrac{\mathbf{a}\cdot\mathbf{u}}{\|\mathbf{u}\|^2}, 0, 1\right) & \text{si } \|\mathbf{u}\|^2 > 0 \\ 0 & \text{si } \|\mathbf{u}\|^2 = 0 \end{cases} \qquad d_\perp = \|\mathbf{a} + f\,\mathbf{u}\|, \qquad L = \|\mathbf{u}\|. \tag{4}$$

$f \in [0,1]$ es la fracción del segmento hasta el punto más próximo al autobús, $d_\perp$ es la distancia del autobús a ese punto y $L$ la longitud del segmento.

**Proposición 1.** *El punto* $\mathbf{a} + f\,\mathbf{u}$ *con* $f$ *dado por* (4) *es el punto del segmento más próximo al origen.*

*Demostración.* Si $\|\mathbf{u}\| = 0$ el segmento es un punto. Si no, $g(t) = \|\mathbf{a} + t\mathbf{u}\|^2 = \|\mathbf{a}\|^2 + 2t\,\mathbf{a}\cdot\mathbf{u} + t^2\|\mathbf{u}\|^2$ es una parábola convexa con mínimo en $t^* = -\mathbf{a}\cdot\mathbf{u}/\|\mathbf{u}\|^2$. Restringida a $[0,1]$, su mínimo es $\operatorname{clamp}(t^*, 0, 1)$, porque $g$ es decreciente a la izquierda de $t^*$ y creciente a la derecha. □

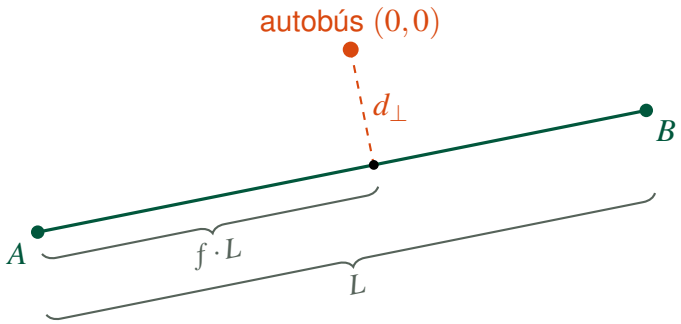
***Error de la Aproximación***

La proyección (3) usa $\cos\varphi$ en la latitud del autobús. En un punto a $\Delta\varphi$ radianes de distancia norte–sur, el factor correcto sería $\cos(\varphi + \Delta\varphi) \approx \cos\varphi\,(1 - \Delta\varphi\tan\varphi)$. A la latitud de Bilbao ($\varphi \approx 43{,}26°$, $\tan\varphi \approx 0{,}94$) y a 3 km ($\Delta\varphi \approx 4{,}7 \cdot 10^{-4}$ rad), el error relativo es del orden del 0,04 %, poco más de un metro. El factor de 111 320 m por grado sobrestima en torno a un 0,2 % la longitud de un grado de latitud a esa altura, unos 6 m en 3 km. Ambos errores son

despreciables frente a los pasillos de tolerancia (50 y 150 m). Además, la hora programada se obtiene de la *fracción* $f$ y de las distancias del propio GTFS, no de distancias medidas en el plano, así que un error de escala uniforme no altera $T_{\text{pos}}$. El de este plano no es del todo uniforme (frente al elipsoide WGS84, la longitud queda un 0,16 % corta y la latitud un 0,2 % larga), pero esa diferencia entre los ejes mueve el punto proyectado menos de un 0,4 % de la longitud del segmento: también despreciable.

**Figura 4**

*Proyección del Autobús Sobre un Segmento: Fracción $f$ y Distancia Perpendicular $d_\perp$*




**Ventana de Búsqueda**

El feed indica el orden $n$ de la próxima parada. Esa información acota dónde puede estar el autobús: en algún punto poco antes de $p_n$. La búsqueda se limita a los tramos

$$i_{\text{mín}} = \text{máx}(1, n-3), \qquad i_{\text{máx}} = \text{mín}(N-1, n+2), \tag{5}$$

es decir, del tramo $n-3$ al $n+2$: los tres que llevan hasta la próxima parada y los tres que salen de ella o de las siguientes (`SEGMENTS_BEHIND_NEXT_STOP` y `SEGMENTS_AHEAD_OF_NEXT_STOP`, 3 y 2, son los desplazamientos respecto a $n$). La ventana cubre las paradas $p_{i_{\text{mín}}}, \ldots, p_{i_{\text{máx}}+1}$, de $p_{n-3}$ a $p_{n+3}$ cuando el viaje las tiene. Mirar hacia atrás tolera que el feed cambie de próxima parada con algo de retraso; mirar hacia delante, que lo haga con antelación. Si $n$ no es el orden de ninguna parada del viaje, no hay posición.

**Con Trazado**

Se usa este método cuando el viaje tiene trazado y se conocen las distancias $s_i$ de todas las paradas de la ventana. Sean $S_{\text{lo}} = s_{i_{\text{mín}}}$ y $S_{\text{hi}} = s_{i_{\text{máx}}+1}$.

1. **Candidatos.** Para cada par de puntos consecutivos del trazado $A = \Sigma_j$, $B = \Sigma_{j+1}$ que no quedan fuera de la ventana (es decir, salvo si $\sigma_B < S_{\text{lo}}$ o $\sigma_A > S_{\text{hi}}$), se proyecta el autobús con (4) y se calcula su posición a lo largo del trazado, $s^* = \sigma_A + f\,(\sigma_B - \sigma_A)$.

2. **Dos grupos.** El candidato es *esperado* si $s^* \leq s_n + 100$ m (`BEYOND_NEXT_STOP_SLACK_METERS`) y *más allá* en otro caso. En cada grupo se queda el de menor $d_\perp$; con empate, el primero.

3. **Desempate.** Se elige el mejor esperado salvo que el mejor de más allá esté al menos 20 m más cerca del recorrido (`AMBIGUITY_MARGIN_METERS`):

$$\text{elegido} = \begin{cases} \text{más allá} & \text{si existe y } \left(\text{no hay esperado o } d_\perp^{\text{más allá}} + 20 < d_\perp^{\text{esperado}}\right) \\ \text{esperado} & \text{en otro caso.} \end{cases}$$

4. **Pasillo.** La posición es válida si $d_\perp \leq 50$ m (`MAX_DISTANCE_FROM_SHAPE_METERS`).

5. **Hora programada.** Se acota $s^* \leftarrow \text{clamp}(s^*, S_{\text{lo}}, S_{\text{hi}})$ y se busca el primer tramo $i \in [i_{\text{mín}}, i_{\text{máx}}]$ con $s^* \leq s_{i+1}$ (o el último de la ventana). Entonces

$$f_i = \begin{cases} \text{clamp}\Big(\dfrac{s^* - s_i}{s_{i+1} - s_i}, 0, 1\Big) & \text{si } s_{i+1} > s_i \\ 0 & \text{en otro caso} \end{cases} \qquad T_{\text{pos}} = \text{rnd}\left(d_i + f_i\,(a_{i+1} - d_i)\right). \quad (6)$$

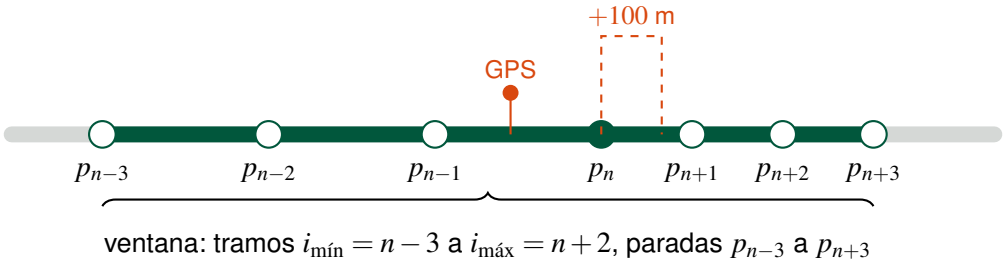
La hora programada se interpola entre la *salida* de la parada de inicio del tramo y la *llegada* a la de fin: dentro del tramo el autobús está en marcha, y la espera en la parada no forma parte de él. Como los segmentos del trazado que tocan el borde de la ventana entran como candidatos y $s^*$ se acota a $[S_{\text{lo}}, S_{\text{hi}}]$, un GPS algo más allá de un extremo queda en ese extremo:

con un trazado que solo tuviera puntos en las paradas, un autobús 400 m después de $p_{i_{\text{máx}}+1}$ y dentro del pasillo toma $T_{\text{pos}} = a_{i_{\text{máx}}+1}$.

Hacen falta dos grupos porque hay recorridos que vuelven sobre la misma calle: una ida y vuelta por el mismo eje, o una rotonda. Un punto GPS en esa calle tiene dos proyecciones casi igual de cercanas. El grupo esperado prefiere la que es coherente con la próxima parada que declara el feed. Solo se acepta una proyección más adelantada si es claramente mejor (20 m), lo que cubre el caso en que el feed tarda en actualizar la próxima parada. Los 100 m de margen tras la próxima parada absorben que el feed cambia de parada con retraso.

**Figura 5**

*Ventana de Búsqueda Alrededor de la Próxima Parada*




*Nota.* Se marca el límite de los candidatos «esperados» alrededor de la próxima parada $p_n$.

### Sin Trazado

Si el viaje no tiene trazado, o falta la distancia de alguna parada de la ventana, se proyecta sobre las rectas entre paradas. Para cada tramo $i$ de la ventana, en orden, se proyecta el autobús sobre el segmento $p_i p_{i+1}$ (con las coordenadas de las paradas) y se conserva el de menor $d_\perp$ (con empate, el primero). Para ese tramo se calculan

$$c_i = \text{máx}\left(150,\ 0{,}15\,L_i\right), \qquad T_{\text{pos}} = \text{rnd}\left(d_i + f\left(a_{i+1} - d_i\right)\right), \tag{7}$$

y la posición es válida si $d_\perp \le c_i$ (`MAX_DISTANCE_FROM_ROUTE_METERS`, `MAX_DISTANCE_FROM_ROUTE_SEGMENT_RATIO`). El pasillo es más ancho porque la calle real se aparta de la recta entre dos paradas, tanto más cuanto más largo es el tramo. Sin trazado no hay

dos grupos ni desempate: se queda el tramo más cercano, y en un recorrido que vuelve sobre sí mismo la única protección es la ventana.

Desde que la construcción mide los metros de las redes que no los publican (sección «Trazados que No Casan con sus Paradas»), este caso solo queda para los viajes que no tienen ningún trazado: el resto se sitúa con el pasillo de 50 m.

**Condiciones Para Usar el GPS**

La posición de una entrada $e$ en el viaje $J$ existe solo si se cumplen todas estas condiciones:

1. $e$ tiene latitud, longitud, hora de la posición $t_{\text{gps}}$ y orden $n$ de la próxima parada;
2. el dato es reciente: $t_{\text{now}} - t_{\text{gps}} \leq 600$ s (`MAX_LOCATION_AGE_SECONDS`);
3. el viaje tiene al menos dos paradas;
4. el viaje ha empezado o está a punto: $t_{\text{gps}} \geq d_1 - 60$ s (`NOT_STARTED_GRACE_SECONDS`); si no, solo cabe la posición en la cabecera de la sección «En la Cabecera, Antes de Salir»;
5. $n$ es el orden de una parada del viaje;
6. la proyección es válida (secciones «Con Trazado» o «Sin Trazado»);
7. el GPS no contradice a la parada registrada (sección «Contraste Entre el GPS y la Parada Registrada»).

Si se cumplen, la *posición* es la cuádrupla $\left(T_{\text{pos}},\ t_{\text{gps}},\ d_{\perp},\ \text{método}\right)$ y el *desfase en la posición* es

$$\delta_{\text{pos}} = t_{\text{gps}} - T_{\text{pos}}. \tag{8}$$

Un valor positivo indica que el autobús llega a ese punto más tarde de lo que dice el horario. Como el horario está interpolado, este desfase *no* se muestra al usuario tal cual (sección «Retraso y Minutos que Se Muestran»).

**Ejemplo 1.** Considérese el viaje sintético del Apéndice D: cinco paradas en línea recta hacia el este a lo largo del paralelo 43,26°, a 0, 400, 1 000, 2 500 y 3 200 m, con horas 8:00:00, 8:01:30, 8:03:30, 8:08:30 y 8:10:30 (28 800, 28 890, 29 010, 29 310 y 29 430 s). Un autobús declara como próxima parada $p_3$ ($n = 3$) y su GPS está a 700 m del origen y 0,0001° al norte ($\approx 11$ m), con $t_{\text{gps}} = 28\,950$.

La ventana es $i_{\text{mín}} = \text{máx}(1,0) = 1$, $i_{\text{máx}} = \text{mín}(4,5) = 4$. La mejor proyección cae en el tramo $p_2p_3$, con $d_\perp = 11{,}132$ m $\leq 50$ y $s^* = 700$ m. Entonces $f_2 = (700-400)/(1000-400) = 0{,}5$ y $T_{\text{pos}} = \text{rnd}(28\,890 + 0{,}5 \cdot (29\,010 - 28\,890)) = 28\,950$. El desfase en la posición es $\delta_{\text{pos}} = 0$: el autobús está donde el horario dice. Sin trazado se obtiene el mismo $T_{\text{pos}}$, por proyección sobre la recta $p_2p_3$.

**En la Cabecera, Antes de Salir**

Un autobús asignado a un viaje que aún no ha salido no cumple la condición 4. Muchos de ellos esperan en la primera parada: en la captura del 3 de noviembre de 2026 a las 14:47, 36 de los 259 autobuses estaban antes de su hora y 34 de ellos a menos de 50 m de su primera parada. Sin posición, la parada de cabecera y las siguientes salían solo con el horario aunque el autobús estuviera allí. Por eso, cuando $t_{\text{gps}} < d_1 - 60$, la entrada tiene la posición

$$T_{\text{pos}} = \text{mín}(a_1, d_1), \qquad t'_{\text{gps}} = d_1, \qquad \text{método «cabecera»}, \tag{9}$$

si el autobús está a 150 m o menos de la primera parada (`MAX_DISTANCE_FROM_ROUTE_METERS`) y su posición se tomó como mucho 60 min antes de su salida, $d_1 - t_{\text{gps}} \leq 3\,600$ s (`MAX_WAIT_AT_ORIGIN_SECONDS`). Más lejos puede estar acabando otro viaje, y más de una hora antes la asignación es poco segura. La hora de la posición pasa a ser la de salida, $d_1$: el autobús no ha recorrido nada y saldrá, como pronto, a su hora.

Con esta posición la fila sale «en directo», pero su llegada no se calcula desde $t'_{\text{gps}}$: es la de la capa 3 (sección «Capa 1 en la Cabecera»). Tampoco cuenta la primera parada como pasada, aunque el feed ya apunte a la segunda ($n = 2$) mientras el autobús espera. La aplicación lo dice

como «En la parada», pero solo en la fila de la primera parada, decidida por su orden en el recorrido y no por su hora: dos paradas seguidas pueden tener la misma hora publicada, y en un viaje circular la primera y la última son la misma. Las posiciones en la cabecera no entran en las medidas del aprendizaje, porque no describen ningún recorrido. El 97 % de las veces que una entrada se sitúa así, ese autobús hace después el viaje desde allí (sección «Fiabilidad de lo que Dice la Aplicación»).

## Modelo del Tiempo de Recorrido

El capítulo anterior sitúa al autobús en el horario: da la hora $T_{\text{pos}}$ a la que el horario lo pone en el punto donde está. Falta saber cuánto tardará en llegar desde ahí hasta una parada. Este capítulo explica por qué se trabaja con diferencias de horario, qué unidad de tiempo se aprende (el tramo entre paradas, por franja), cómo se estima cada tiempo, qué se hace cuando un tramo no tiene datos y cómo se suman los tramos para obtener el tiempo esperado entre dos puntos del recorrido.

### Por Qué Diferencias de Horario

Sea $J$ un viaje y $s \in [s_1, s_N]$ un punto de su recorrido, medido en metros desde la cabecera. Se definen tres funciones:

- $T(s)$, la hora a la que el *horario* pone al autobús en $s$. En las paradas, $T(s_i) = a_i$; entre ellas, se interpola linealmente (ecuación (6)).
- $R(s)$, la hora *real* a la que el autobús pasa por $s$. Es desconocida salvo en el punto donde se ha visto al autobús.
- $\Delta(s) = R(s) - T(s)$, el *desfase* en $s$.

El autobús se ha visto en $s_p$ a la hora $t_{\text{gps}} = R(s_p)$, y la parada de interés está en $s_d > s_p$, con hora publicada $a_{\text{dest}} = T(s_d)$. La cantidad que se quiere predecir es $R(s_d)$.

**Proposición 2** (El error de las diferencias)**.** *Sean dos estimadores de $R(s_d)$:*

$$\widehat{R}_{\text{hor}} = T(s_d) \qquad \textit{(solo horario)}, \qquad \widehat{R}_{\text{dif}} = t_{\text{gps}} + \big(T(s_d) - T(s_p)\big) \qquad \textit{(diferencias)}.$$

*Sus errores son*

$$R(s_d) - \widehat{R}_{\text{hor}} = \Delta(s_d), \qquad R(s_d) - \widehat{R}_{\text{dif}} = \Delta(s_d) - \Delta(s_p).$$

*Más en general, si $\widehat{E}$ es cualquier estimación del tiempo real entre $s_p$ y $s_d$ y $\widehat{R} = t_{\text{gps}} + \widehat{E}$, el error es $\big(R(s_d) - R(s_p)\big) - \widehat{E}$, que no depende de $\Delta(s_p)$.*

*Demostración.* La primera igualdad es la definición de $\Delta$. Para la segunda, como $t_{\text{gps}} = R(s_p) = T(s_p) + \Delta(s_p)$,

$$\begin{aligned} R(s_d) - t_{\text{gps}} - T(s_d) + T(s_p) &= \big(T(s_d) + \Delta(s_d)\big) - \big(T(s_p) + \Delta(s_p)\big) - T(s_d) + T(s_p) \\ &= \Delta(s_d) - \Delta(s_p). \end{aligned}$$

La tercera es inmediata: $R(s_d) - t_{\text{gps}} - \widehat{E} = \big(R(s_d) - R(s_p)\big) - \widehat{E}$. □

La proposición tiene tres consecuencias que guían todo el diseño.

1. **El retraso acumulado no importa.** Un autobús que va veinte minutos tarde desde la cabecera tiene $\Delta(s_p) \approx \Delta(s_d) \approx 1200$ s; el horario solo yerra en 20 minutos, las diferencias en casi nada. La predicción no necesita saber cuánto retraso lleva el autobús, solo dónde está.

2. **El error de forma del horario sí importa.** Como el horario intermedio es una interpolación a velocidad constante (sección «Por Qué el Horario No Sirve Como Referencia Absoluta»), $T(s_d) - T(s_p)$ puede estar muy lejos del tiempo real entre esos dos puntos: en un tramo de autopista, el doble. Las diferencias de horario eliminan el desfase acumulado, pero no este error. Por eso se sustituye $T(s_d) - T(s_p)$ por una estimación aprendida $\widehat{E}$ del tiempo real (el resto de este capítulo).

3. **El error crece con la distancia.** El error $\big(R(s_d) - R(s_p)\big) - \widehat{E}$ suma la variación de todos los tramos entre el autobús y la parada. Si los tiempos de los tramos fueran independientes, su varianza sería la suma de las varianzas de cada tramo y crecería aproximadamente de forma lineal con el número de tramos. Es lo que se observa: la precisión cae de forma regular con el horizonte (capítulo «Resultados»).

**La Unidad de Aprendizaje: El Tramo**

El error de forma del horario se produce tramo a tramo: en cada tramo de autopista el horario se queda corto y en cada tramo urbano congestionado se queda largo. Si se aprende el

tiempo real de cada tramo, el error de forma desaparece al sumar. Esta es la razón de aprender *por tramo* y no por línea:

- un factor único por línea (por ejemplo, «la línea 3414 tarda un 4 % menos que su horario») no puede corregir un viaje que combina ciudad y autopista, porque la corrección tiene signos distintos en cada parte;
- el tramo se identifica por sus dos paradas (definición de tramo, sección «Notación»), de modo que las medidas de todas las líneas que lo recorren se suman y el modelo aprende más deprisa;
- el tiempo de un tramo es una magnitud física estable (depende de la calle, los semáforos y la demanda de esa parada) y fácil de auditar: se puede comparar con lo que tarda uno mismo en recorrerlo.

El tiempo de un tramo cambia con la hora y el día. Se aprende por *franja* (definición 4): doce combinaciones de tipo de día (laborable, fin de semana) y periodo del día (madrugada, punta de la mañana, mañana, mediodía, punta de la tarde, noche). Las fronteras de las franjas siguen los periodos de demanda habituales en el transporte urbano. Con los datos de octubre de 2026, los tiempos de un mismo tramo difieren entre franjas un 8–13 % de media, y la noche es un 12 % más rápida que la punta de la tarde.

**Estimador del Tiempo de un Tramo**

***Las Muestras***

El capítulo «Aprendizaje Continuo» explica cómo se reconstruye, a partir de las capturas del feed, el instante real en que un autobús pasa por cada parada. Con dos pasos consecutivos se obtiene una *muestra de tramo*:

**Definición 6** (Muestra de tramo)**.** *Una muestra de tramo es una tupla* $x = (\kappa,\ j,\ b,\ y)$*. El primer elemento,* $\kappa = u_i{>}u_{i+1}$*, es el identificador del tramo.* $j$ *es la* pista*: el par vehículo y viaje en una fecha concreta, que identifica un recorrido físico de un autobús.* $b$ *es la franja del instante en que el autobús pasó por* $p_i$*,* $e$ $y > 0$ *es el tiempo real, en segundos, que tardó en ir de* $p_i$ *a* $p_{i+1}$*.*

***Un Valor por Pista***

Para cada tramo $\kappa$ y cada *grupo* $g$ (una franja concreta, o el grupo especial `all` que reúne todas las franjas) se forma el conjunto de pistas que tienen alguna muestra de $\kappa$ en $g$. Cada pista aporta *un solo valor*: si una pista tuviera varias muestras del mismo tramo y grupo, se queda la *última* en el orden del fichero de muestras.[1] Sea $Y_{\kappa,g}$ la lista de esos valores y $n_{\kappa,g} = |Y_{\kappa,g}|$ el número de pistas distintas.

Un valor por pista evita que un autobús visto muchas veces pese más que otro visto una vez, y es el sentido en que el estimador es *robusto*: un autobús que se queda parado veinte minutos aporta un valor extremo, pero uno solo.

***Mediana y Contracción Hacia el Tramo a Cualquier Hora***

Para cada tramo $\kappa$ se calcula la mediana baja a cualquier hora,

$$m_\kappa = \operatorname{med}\left(Y_{\kappa,\texttt{all}}\right), \tag{10}$$

y, para cada franja $b$ con $n_{\kappa,b} \geq 2$ pistas, la mediana de la franja $m_{\kappa,b} = \operatorname{med}(Y_{\kappa,b})$ contraída hacia $m_\kappa$:

$$\boxed{\tau_{\kappa,b} = \operatorname{rnd}\left(\frac{n_{\kappa,b}\, m_{\kappa,b} + n_0\, m_\kappa}{n_{\kappa,b} + n_0}\right), \qquad n_0 = 5} \tag{11}$$

(`SEGMENT_BAND_PRIOR_TRACKS`). El valor a cualquier hora se guarda como $\tau_{\kappa,\texttt{all}} = \operatorname{rnd}(m_\kappa)$, pero solo si el tramo tiene muestras en al menos dos franjas distintas. Las reglas completas de qué se guarda son:

1. Un tramo se considera solo si $n_{\kappa,\texttt{all}} \geq 2$ (`MIN_TRACKS_PER_SEGMENT`).
2. Una franja $b$ se guarda solo si $n_{\kappa,b} \geq 2$.
3. El grupo `all` se guarda solo si hay muestras de al menos dos franjas distintas, aunque alguna de ellas tenga una sola pista. Con una sola franja, el valor a cualquier hora sería el

[1] En la práctica una pista recorre cada tramo una sola vez. La regla de «la última» importa solo para reproducir exactamente el resultado con datos duplicados; los vectores de prueba la comprueban (Apéndice D).

de esa franja con otro nombre, y se usaría en otras franjas para las que no hay ninguna medida: lo medido a mediodía acabaría usándose en hora punta.

4. El tramo se guarda si se guardó al menos un grupo. Se guarda también $n_{\kappa,\texttt{all}}$ como información (clave `n`).

Todos los valores se guardan como enteros (segundos, con rnd).

La ecuación (11) es una media ponderada entre lo que dice la franja y lo que dice el tramo a cualquier hora, con pesos $n_{\kappa,b}$ y $n_0$. Es exactamente la forma de la media a posteriori de un modelo normal jerárquico (Gelman et al., 2013): si las pistas de la franja $b$ se distribuyen alrededor de su media $\mu_b$ con varianza $\sigma^2$, y las medias de las franjas se distribuyen alrededor de la media del tramo $\mu$ con varianza $\sigma_0^2$, la media a posteriori de $\mu_b$ es

$$\mathbb{E}[\mu_b \mid \text{datos}] = \frac{n\,\bar{y}_b + (\sigma^2/\sigma_0^2)\,\mu}{n + \sigma^2/\sigma_0^2}.$$

La constante $n_0$ representa el cociente $\sigma^2/\sigma_0^2$: cuántas pistas de la franja hacen falta para que su propia mediana pese lo mismo que la del tramo. Con pocas pistas manda el tramo; con muchas, la franja. El método sustituye las medias por medianas para conservar la robustez, y fija $n_0$ por validación (no por una estimación de las varianzas), lo que es más sencillo y se comprobó suficiente.

**Proposición 3** (Por qué contraer reduce el error)**.** *Sea $\hat{\theta}_b$ un estimador insesgado de $\theta_b$ con varianza $\sigma^2/n$, y $\mu$ un valor fijo con $\theta_b - \mu = \beta$. Para el estimador contraído $\tilde{\theta}_b = w\hat{\theta}_b + (1-w)\mu$, con $w = n/(n+n_0)$, el error cuadrático medio es*

$$\mathrm{ECM}(\tilde{\theta}_b) = w^2\frac{\sigma^2}{n} + (1-w)^2\beta^2,$$

*que es menor que el de $\hat{\theta}_b$ ($\sigma^2/n$) siempre que $\beta^2 < \sigma^2\,(2n+n_0)/(n\,n_0)$. En particular, la ganancia es mayor cuanto menor es n.*

*Demostración.* $\tilde{\theta}_b - \theta_b = w(\hat{\theta}_b - \theta_b) - (1-w)\beta$; el primer término tiene media cero y varianza

$w^2\sigma^2/n$ y el segundo es constante, de donde sale la expresión. La desigualdad $w^2\sigma^2/n + (1-w)^2\beta^2 < \sigma^2/n$ equivale a $(1-w)^2\beta^2 < (1-w^2)\sigma^2/n$, es decir, $\beta^2 < \frac{1+w}{1-w}\cdot\frac{\sigma^2}{n}$, y con $w = n/(n+n_0)$ se tiene $\frac{1+w}{1-w} = \frac{2n+n_0}{n_0}$. □

La proposición 3 explica el comportamiento medido (sección «Evaluación por Días de la Contracción»): la contracción mejora las franjas con pocas pistas, donde la mediana es ruidosa, y apenas cambia las que tienen muchas. Un $n_0$ demasiado grande sesga las franjas que de verdad son distintas (el término $\beta^2$), y por eso el valor 10 resultó peor que el 5 a 12–20 minutos. La proposición supone un estimador insesgado y un $\mu$ fijo; el método usa medianas bajas y estima $\mu$ ($m_\kappa$) con datos que incluyen las pistas de la propia franja, así que es una justificación heurística, no una garantía: el valor de $n_0$ lo decide la medida.

**Ejemplo 2.** Un tramo $A{>}B$ tiene dos pistas en `laborable.punta_manana` (90 y 100 s) y seis en `laborable.mediodia` (60, 58, 62, 64, 59 y 61 s). Las ocho juntas, ordenadas, son 58, 59, 60, 61, 62, 64, 90, 100, y su mediana baja es el elemento $\lfloor 7/2 \rfloor = 3$: $m_{AB} = 61$. Entonces

$$\tau_{AB,\texttt{punta_manana}} = \mathrm{rnd}\big(\tfrac{2\cdot 90+5\cdot 61}{7}\big) = \mathrm{rnd}(69{,}29) = 69,$$
$$\tau_{AB,\texttt{mediodia}} = \mathrm{rnd}\big(\tfrac{6\cdot 60+5\cdot 61}{11}\big) = \mathrm{rnd}(60{,}45) = 60,$$

y $\tau_{AB,\texttt{all}} = 61$ porque hay dos franjas. Sin contracción, la punta de la mañana valdría 90 s, apoyada solo en dos autobuses. Un tramo $C{>}D$ con tres pistas, todas de noche (200, 210 y 190 s), guarda solo `laborable.noche` $= 200$ (la contracción hacia sí mismo no cambia nada) y ningún valor `all`.

***Consulta***

Dado un tramo $\kappa$ y una franja $b$, el tiempo aprendido es

$$\tau(\kappa, b) = \begin{cases} \tau_{\kappa,b} & \text{si existe,} \\ \tau_{\kappa,\texttt{all}} & \text{si no, y si existe,} \\ \text{indefinido} & \text{en otro caso.} \end{cases} \tag{12}$$

Se dice que el tramo está *aprendido* en la franja $b$ si $\tau(\kappa, b)$ está definido. Obsérvese que un tramo con datos solo de noche está aprendido de noche y no lo está a mediodía: entonces se usa el respaldo de la sección siguiente, no el valor nocturno.

**Respaldo por Longitud del Tramo**

Un tramo sin tiempo aprendido no se puede dejar sin estimar, porque la llegada suma todos los tramos del camino. La primera idea fue usar el horario multiplicado por un factor de ritmo de la línea (sección «Factor de Ritmo»), pero eso reproduce el error de forma del horario interpolado. La observación que lleva al respaldo es que ese error depende sobre todo de la *longitud* del tramo: los tramos cortos son urbanos (el autobús tarda lo que dice el horario o algo más), y los largos son interurbanos y de autopista (el autobús tarda bastante menos de lo programado, porque la interpolación reparte el tiempo a velocidad constante).

***Cubos de Longitud***

La longitud de un tramo es $L = s_{i+1} - s_i$, a partir de `shape_dist_traveled`. Se clasifica en seis cubos (`LENGTH_CUTS_METERS`):

$$c(L) = \begin{cases} 0 & L < 300 \\ 1 & 300 \le L < 600 \\ 2 & 600 \le L < 1000 \\ 3 & 1000 \le L < 1500 \\ 4 & 1500 \le L < 3000 \\ 5 & L \ge 3000 \end{cases} \tag{13}$$

Los cortes son cerrados por abajo: un tramo de exactamente 300 m está en el cubo 1.

***Proporción Real/Horario por Cubo y Franja***

Para cada muestra de tramo con duración programada $h = a_{i+1} - a_i > 0$ y longitud conocida $L > 0$, se calcula la proporción $q = y/h$. Para cada cubo $c$ y grupo $g$ (franja o `all`) se toma la mediana baja de *todas* las proporciones del grupo (aquí sin reducir a un valor por pista, porque cada proporción ya es adimensional y compara tramos distintos):

$$r_{c,g} = \mathrm{rnd}_3\left(\mathrm{med}\{q\}\right), \tag{14}$$

siempre que el grupo tenga al menos 20 proporciones (`MIN_SAMPLES_PER_LENGTH_RATIO`); si no, $r_{c,g}$ no existe. Aquí $\mathrm{rnd}_3(x) = \mathrm{rnd}(1000x)/1000$ redondea a tres decimales con la misma regla de las mitades. La longitud $L$ de un tramo, al calibrar, es la mediana baja de las longitudes $s_{i+1} - s_i$ de ese par de paradas en todos los viajes del horario que lo recorren (puede variar algo entre viajes si los trazados difieren).

***Consulta***

Al predecir, para un tramo no aprendido con duración programada $h$ y longitud $L$ del viaje concreto,

$$\tau^{\mathrm{L}}(h,L,b) = \mathrm{clamp}\left(\hat{r},\ 0{,}3,\ 1{,}5\right)\cdot h, \qquad \hat{r} = \begin{cases} r_{c(L),b} & \text{si existe,} \\ r_{c(L),\texttt{all}} & \text{si no.} \end{cases} \tag{15}$$

Si no existe ninguna de las dos proporciones, si $L$ no se conoce o si $L \leq 0$, $\tau^{\mathrm{L}}$ está indefinido. Las cotas $[0{,}3;\ 1{,}5]$ impiden que un dato corrupto (una proporción de 0,1 o de 2) se traslade a la predicción. Si el viaje no tiene distancias (sin `shape_dist_traveled`), $L$ no se conoce y este respaldo no se aplica.

La Tabla 5 muestra las proporciones calibradas el 31 de octubre de 2026. En la columna `all` la tendencia es clara y monótona: los tramos de menos de 300 m se recorren en el tiempo programado y los de más de 3 km en algo más de la mitad. En las franjas, los cubos 3 y 4 quedan casi iguales.

**Tabla 5**

*Proporción Entre Tiempo Real y Tiempo Programado por Cubo de Longitud*

| Cubo | Longitud | $r_{c,\texttt{all}}$ | Laborable punta mañana | Laborable mediodía | Fin de semana mañana |
|---|---|---|---|---|---|
| 0 | $<$ 300 m | 0,987 | 1,016 | 1,052 | 0,894 |
| 1 | 300–600 m | 0,907 | 0,937 | 0,949 | 0,823 |
| 2 | 600–1 000 m | 0,769 | 0,811 | 0,793 | 0,716 |
| 3 | 1 000–1 500 m | 0,647 | 0,665 | 0,659 | 0,606 |
| 4 | 1 500–3 000 m | 0,640 | 0,668 | 0,669 | 0,600 |
| 5 | $\geq$ 3 000 m | 0,534 | 0,565 | 0,524 | 0,523 |

*Nota.* Columna $r_{c,\texttt{all}}$: grupo `all`; las demás, la franja de su título. Muestras acumuladas a 31 de octubre de 2026.

**Factor de Ritmo**

El factor de ritmo $k$ es el último recurso de la cascada y, además, el único modelo cuando no existe ninguna tabla de tiempos por tramo (por ejemplo, al poner el método en marcha en una red nueva). Entonces las capas 1 y 2 usan $E = k\,(t_1 - t_0)$, la capa 3 no se aplica (las filas sin datos

del autobús salen con el horario publicado) y el autobús que espera en la cabecera se calcula con la ecuación (22). Mide cuánto tarda de verdad un autobús frente a lo que dice el horario, de forma global por franja y, con datos suficientes, por línea y franja.

***Consulta***

$$k(\ell, D, t) = \operatorname{clamp}\left(\hat{k},\ 0{,}6,\ 1{,}2\right), \qquad \hat{k} = \begin{cases} k_{\ell,b} & \text{si existe,} \\ k_b & \text{si no, y si existe,} \\ 1 & \text{en otro caso,} \end{cases} \qquad b = \text{franja}(D,t), \tag{16}$$

donde $\ell$ es la línea del viaje (`MIN_FACTOR`, `MAX_FACTOR`). El número de línea se escribe siempre como en el horario, sin ceros a la izquierda: el feed pone `A0651` y se lee como `651`, y una tabla guardada con la otra forma vale igual (`PaceFactors::lineKey`).

***Estimación***

Las *muestras de ritmo* (capítulo «Aprendizaje Continuo») son tuplas $(j,\ \ell,\ v,\ b,\ H,\ Y)$: la pista $j$, la línea $\ell$, el vehículo $v$, la franja $b$ del instante de la observación, el tiempo que según el horario faltaba para llegar a una parada, $H = a_{\text{dest}} - T_{\text{pos}} \geq 120$ s, y el tiempo que de verdad faltaba, $Y = t_{\text{real}} - t_{\text{gps}} > 0$.

Para un conjunto de pistas $\mathscr{J}$ (las de una franja, o las de una línea en una franja), se calcula para cada pista la proporción de sumas

$$\rho_j = \frac{\sum Y}{\sum H} \qquad \text{(sumas sobre las muestras de la pista } j\text{)}, \tag{17}$$

que pondera cada muestra por su duración (una predicción a 15 minutos pesa más que una a 2), y después

$$\boxed{k = \operatorname{rnd}_3\left(\operatorname{clamp}\left(1 + \left(\tilde{\rho} - 1\right)\frac{m}{m+10},\ 0{,}6,\ 1{,}2\right)\right), \qquad \tilde{\rho} = \operatorname{med}_j \rho_j, \quad m = |\,\mathscr{J}\,|} \tag{18}$$

(`SHRINKAGE_TRACKS`). Es una contracción hacia $k = 1$ (el horario tal cual) con peso equivalente a diez pistas: la misma idea que la sección «Mediana y Contracción Hacia el Tramo a Cualquier Hora», con el horario como valor común.

Se estima $k_b$ si la franja tiene al menos $m \geq 8$ pistas (`MIN_TRACKS_PER_BAND`), y $k_{\ell,b}$ si la línea en esa franja tiene al menos 25 pistas y 5 vehículos distintos (`MIN_TRACKS_PER_LINE`, `MIN_VEHICLES_PER_LINE`). La exigencia de vehículos distintos evita que el factor de una línea refleje a un solo conductor o a un autobús averiado.

**Ejemplo 3.** Ocho pistas con proporciones 0,80, 0,82, …, 0,94 (en pasos de 0,02) tienen mediana baja $\tilde{\rho} = 0{,}86$ (el elemento $\lfloor 3{,}5 \rfloor = 3$). Entonces $k = 1 - 0{,}14 \cdot 8/18 = 0{,}93778$, que se redondea a 0,938. Con 40 pistas de proporción 0,85, $k = 1 - 0{,}15 \cdot 40/50 = 0{,}88$. Con diez pistas de proporciones 1,6, 1,7, …, 2,5, la mediana baja es 2,0 y la fórmula da $1 + 1{,}0 \cdot 10/20 = 1{,}5$, que queda en la cota: $k = 1{,}2$.

Con los datos de octubre de 2026 los factores globales van de 0,756 (fin de semana, punta de la mañana) a 0,928 (laborable, punta de la tarde), y 54 líneas tienen factor propio en alguna franja.

**Tiempo Esperado Entre Dos Instantes del Horario**

Con los tres niveles anteriores se define la función central del método: el tiempo real que se espera que tarde el autobús en ir desde el punto donde el horario lo pone a la hora $t_0$ hasta el punto donde lo pone a la hora $t_1$. Los dos extremos se dan como *horas del horario*, no como distancias, porque es lo que proporciona la localización ($T_{\text{pos}}$) y lo que define a cada parada ($a_{\text{dest}}$).

**Definición 7** (Tiempo de un tramo)**.** *Para el tramo i de J, con duración programada* $h_i = a_{i+1} - a_i$,

*longitud $L_i = s_{i+1} - s_i$ (si se conoce), identificador $\kappa_i$, franja $b$ y factor $k$:*

$$\tau_i = \begin{cases} \tau(\kappa_i, b) & \text{si está definido (tiempo aprendido)}, \\ \tau^{\mathrm{L}}(h_i, L_i, b) & \text{si no, y si está definido (respaldo por longitud)}, \\ k \cdot h_i & \text{en otro caso (factor de ritmo).} \end{cases} \tag{19}$$

**Definición 8** (Tiempo esperado)**.** *Sea $J$ un viaje con llegadas $a_1 \le \cdots \le a_N$, y sean $t_0 < t_1$ dos horas del horario, una franja $b$ y un factor $k$. Si no existe ninguna tabla de tiempos por tramo (la tabla* `segments` *está vacía),*

$$E(J, t_0, t_1, b, k) = k\,(t_1 - t_0).$$

*En otro caso,*

$$\boxed{E(J, t_0, t_1, b, k) = \sum_{i \in I(t_0, t_1)} \max(0, w_i)\, \tau_i} \tag{20}$$

*con*

$$w_i = \begin{cases} \dfrac{\min(a_{i+1}, t_1) - \max(a_i, t_0)}{a_{i+1} - a_i} & \text{si } a_{i+1} > a_i, \\ 1 & \text{si } a_{i+1} = a_i, \end{cases} \tag{21}$$

*donde el conjunto de tramos $I(t_0, t_1)$ se obtiene recorriendo $i = 1, \ldots, N-1$ en orden y aplicando dos reglas: se* salta *el tramo si $a_{i+1} \le t_0$ (ya está detrás) y se* termina *el recorrido en el primer tramo con $a_i \ge t_1$ (ya está después del destino).*

El peso $w_i$ es la fracción de la duración programada del tramo $i$ que queda entre $t_0$ y $t_1$: vale 1 en los tramos completos y una fracción en el tramo donde está el autobús. Equivale a suponer que, *dentro de un tramo*, el autobús avanza al ritmo del horario; entre tramos, en cambio, el ritmo es el aprendido. Se probó una curva no lineal dentro del tramo y no mejoró (sección «Ideas Probadas y Descartadas»).

Varias propiedades de la definición son importantes para replicarla exactamente:

**Observación 2** (Se recorre por horas, no por índices)**.** *Las reglas de salto y de fin se evalúan*

*sobre las horas $a_i$, no sobre los índices de parada. Si dos paradas consecutivas $p_j$ y $p_{j+1}$ tienen la misma hora programada, el destino $t_1 = a_{j+1} = a_j$ termina el recorrido en el tramo $j$ (porque $a_j \geq t_1$), de modo que ambas paradas reciben la misma llegada estimada.*

**Observación 3** (Tramos de duración nula)**.** *Un tramo con $a_{i+1} = a_i$ (frecuente en el GTFS, donde dos paradas próximas comparten minuto) se incluye con peso 1, y aporta su tiempo aprendido completo, solo si $t_0 < a_i$ y $a_i < t_1$. Por eso el horario corregido de la capa 3 (sección «Capa 3: Horario Corregido») empieza en $t_0 = d_1 - 1$ y no en $d_1$: así se incluyen los tramos de duración nula del principio del viaje. Por la misma razón, $E$ no es exactamente aditiva: si $t_0 < t_m < t_1$ y hay un tramo de duración nula con $a_i = t_m$, $E(t_0, t_1)$ lo cuenta y $E(t_0, t_m) + E(t_m, t_1)$ no.*

**Observación 4** (La espera en la parada)**.** *La duración de un tramo, $h_i = a_{i+1} - a_i$, incluye la espera programada en $p_i$, mientras que $T_{\text{pos}}$ se interpola entre $d_i$ y $a_{i+1}$ (sección «Con Trazado»). Si $d_i > a_i$, un autobús que acaba de salir de $p_i$ ($T_{\text{pos}} = d_i$) recibe $w_i = (a_{i+1} - d_i)/h_i < 1$ y no cuenta la parte del tiempo aprendido que corresponde a esa espera. En la base de Bizkaibus todas las paradas tienen $d_i = a_i + 2$ s, así que el efecto es de unos segundos; los vectores de prueba, con $a_i = d_i$, no lo ejercitan.*

**Observación 5** (Franja única)**.** *Todo el camino se evalúa con una sola franja $b$, la del instante de la observación, aunque el autobús vaya a cruzar a otra franja antes de llegar. Calcular la franja de cada tramo a la hora prevista de paso por él se probó y no mejoró (sección «Ideas Probadas y Descartadas»).*

**Ejemplo 4.** En el viaje sintético del ejemplo 1, con franja `laborable.punta_manana` y $k = 0{,}9$, los tramos aprendidos son $S0{>}S1$ (110 s en esa franja) y $S1{>}S2$ (140 s), y las proporciones del cubo 2 y del cubo 4 en esa franja son 0,811 y 0,668. Desde $t_0 = T_{\text{pos}} = 28\,950$ (a mitad del tramo $S1{>}S2$):

- hasta $S2$ ($t_1 = 29\,010$): $w = (29\,010 - 28\,950)/120 = 0{,}5$ en $S1{>}S2$, luego $E = 0{,}5 \cdot 140 = 70$ s;

- hasta $S3$ ($t_1 = 29\,310$): se añade $S2{>}S3$, no aprendido, de 1 500 m (cubo 4) y 300 s programados: $0{,}668 \cdot 300 = 200{,}4$ s, luego $E = 270{,}4$ s;
- hasta $S4$ ($t_1 = 29\,430$): se añade $S3{>}S4$, de 700 m (cubo 2) y 120 s: $0{,}811 \cdot 120 = 97{,}32$ s, luego $E = 367{,}72$ s.

Sin distancias (viaje sin trazado), los dos últimos tramos caen al factor de ritmo: $0{,}9 \cdot 300 = 270$ y $0{,}9 \cdot 120 = 108$ s, y $E$ vale 70, 340 y 448 s.

**La Cascada Completa**

La Tabla 6 resume qué nivel del modelo se usa en cada caso. Todos los niveles devuelven un tiempo en segundos para un tramo; el tiempo esperado de un camino es la suma ponderada de la ecuación (20), y cada tramo del camino puede salir de un nivel distinto.

**Tabla 6**

*Cascada de Estimación del Tiempo de un Tramo*

| Nivel | Se usa si | Valor | Aprende de |
|---|---|---|---|
| 1 | El tramo tiene valor en la franja | $\tau_{\kappa,b}$, contraído hacia el tramo | Pasos reales de $\geq 2$ pistas en la franja |
| 2 | El tramo tiene valor a cualquier hora (y muestras de $\geq 2$ franjas) | $\tau_{\kappa,\texttt{all}}$ | Pasos reales de $\geq 2$ pistas |
| 3 | Se conoce la longitud y hay proporción para su cubo | $\text{clamp}(\hat{r}, 0{,}3, 1{,}5) \cdot h$, con $\hat{r} = r_{c,b}$ o, si no, $r_{c,\texttt{all}}$ | $\geq 20$ proporciones del cubo en el grupo |
| 4 | Siempre | $k \cdot h$, con $k \in [0{,}6;\ 1{,}2]$ | $\geq 8$ pistas (franja) o $\geq 25$ y 5 vehículos (línea) |

El diseño en cascada responde a un compromiso entre sesgo y varianza: cada nivel es menos específico que el anterior (más sesgo) pero se apoya en más datos (menos varianza). El nivel que se usa es siempre el más específico que tiene datos suficientes, y los umbrales de «suficiente» se han fijado por validación cruzada (capítulos «Aprendizaje Continuo» y «Resultados»). Con los datos de octubre de 2026, la tabla tiene 2 454 tramos aprendidos, el 86 % de las predicciones tiene todos los tramos de su camino en los niveles 1 o 2, y, como los seis

cubos tienen proporción a cualquier hora, el nivel 4 solo actúa en la práctica en viajes sin distancias a lo largo del trazado.

## Predicción en Tres Capas

Este capítulo combina la localización (capítulo «Emparejamiento y Localización») y el modelo de tiempos (capítulo «Modelo del Tiempo de Recorrido») en la predicción que ve el usuario. La idea organizadora es una jerarquía de fuentes, cada una con una regla explícita para abstenerse:

**Capa 1 · GPS.** La posición proyectada sobre el recorrido. Es la fuente más precisa y la que permite afirmar un adelanto o un retraso.

**Capa 2 · Parada registrada.** Cuando el GPS no es utilizable pero el feed dice cuál es la próxima parada y a qué hora se tomó el dato, se supone al autobús a mitad del tramo que lleva a esa parada.

**Capa 3 · Horario corregido.** Cuando no hay ningún dato del vehículo, se recalcula el horario desde la cabecera con los tiempos aprendidos.

Si ninguna capa puede sostener una estimación, se muestra el horario publicado, marcado como tal. Ninguna capa inventa: cada una se usa solo en las condiciones en que se ha medido que mejora a la siguiente.

### Entradas, Salidas y Estados

La unidad de trabajo es una *fila*: una salida concreta de una parada. Una fila identifica un viaje $J$ (su línea, número de viaje y salida de cabecera $d_1$) y una parada de ese viaje, la parada de *destino* de la predicción, con su orden $q$ y su hora publicada $a_{\text{dest}}$. Para cada fila el motor devuelve:

- un **estado**, de la Tabla 7;
- una **llegada estimada** ETA, en segundos desde medianoche;
- un **retraso** $\delta$ en segundos, que puede ser *nulo* (no se afirma retraso alguno);

- el vehículo y su parada actual, si se conocen.

**Tabla 7**

*Estados de una Fila y lo que Muestra la Aplicación*

| Estado | Significado | Texto en la aplicación |
|---|---|---|
| `live` | Capa 1: el GPS sitúa al autobús en el recorrido. | «En hora», «Retraso $+N$m» o «Adelantado $-N$m»; distintivo «En ruta», «Llegando» o «En la parada» (también mientras espera en la cabecera). |
| `estimated` | Capa 2 o capa 3: estimación sin seguimiento en directo; retraso 0. | «Estimado con lo que tardan de verdad los buses»; distintivo «Estimado». |
| `departed` | El autobús ya pasó por la parada. | «Ya salió»; distintivo «Saliendo». |
| `likely_departed` | Sin datos del autobús, el horario corregido indica que ya debió pasar; o, con GPS, la llegada se cumplió hace más de 3 min sin que el feed ni el GPS confirmen el paso. | «Sin datos del bus: puede que ya haya pasado»; distintivo «Sin datos del bus»; se ordena al final. |
| `scheduled` | Ninguna capa puede sostener una estimación. | «Horario, sin seguimiento en vivo»; distintivo «Solo horario». Si el directo trae autobuses con hora de hoy y el viaje aún no ha llegado a su hora, la ficha dice que aún no tiene autobús asignado. |

La fila se empareja con una entrada del feed como se explicó en la sección «Emparejamiento con el Horario»: primero de forma exacta por los alias de viaje y, si no, por línea, número de viaje y salida de cabecera con una tolerancia de 180 s. Si no hay entrada, la fila va directamente a la capa 3.

En todo el capítulo, $D$ es la fecha de servicio del viaje (de madrugada, la de ayer para los viajes que pasan de las 24:00; sección «Viajes que Pasan de las 24:00»), $\ell$ la línea, $n$ el orden de la próxima parada que declara el feed y $E(\cdot)$ el tiempo esperado de la definición 8 aplicada al viaje $J$.

**Capa 1: Llegada Desde el GPS**

Si existe la posición $(T_{\text{pos}}, t_{\text{gps}})$ (sección «Condiciones Para Usar el GPS»), la franja y el factor se toman en el instante de la posición,

$$b = \text{franja}(D, t_{\text{gps}}), \qquad k = k(\ell, D, t_{\text{gps}}),$$

y la llegada a la parada de destino es

$$\boxed{\text{ETA} = \text{rnd}\left(t_{\text{gps}} + E(J, T_{\text{pos}}, a_{\text{dest}}, b, k)\right), \qquad \delta = \text{ETA} - a_{\text{dest}} \qquad (a_{\text{dest}} \geq T_{\text{pos}}).} \tag{22}$$

Si $a_{\text{dest}} < T_{\text{pos}}$, la parada ya ha quedado atrás según el horario y

$$\text{ETA} = t_{\text{gps}} - (T_{\text{pos}} - a_{\text{dest}}), \qquad \delta = \delta_{\text{pos}} = t_{\text{gps}} - T_{\text{pos}}, \tag{23}$$

que es la hora a la que se estima que pasó: la distancia programada entre la parada y el autobús, contada hacia atrás desde la hora de la posición.

La llegada parte de $t_{\text{gps}}$, no de $t_{\text{now}}$. Cada posición llega al motor un minuto o más después de tomarla (66 s de mediana y 92 s en el percentil 90 el 5 de noviembre de 2026; sección «Llegando sin Confirmar el Paso»); sumar el tiempo restante a $t_{\text{now}}$ desplazaría todas las llegadas ese retraso. Partiendo de $t_{\text{gps}}$, la antigüedad del dato no sesga la predicción, solo la hace algo menos precisa. Una consecuencia es que ETA puede ser anterior a $t_{\text{now}}$ si el autobús estaba muy cerca de la parada cuando se tomó la posición.

**Capa 1 en la Cabecera**

Con la posición en la cabecera (9), la ecuación (22) daría $d_1 + E(J, a_1, a_{\text{dest}}, b, k)$: supone que el autobús sale a su hora en punto. Los autobuses no salen así, y la medida lo confirmó: con esa suposición, las llegadas a 10-20 min vista se adelantaban 1 min de media y los aciertos dentro de $\pm 2$ min bajaban del 67,0 al 64,0 % (sección «Trazados del Derecho y Espera en la Cabecera»).

La llegada es entonces la de la capa 3,

$$\text{ETA} = \text{máx}\left(C(J, a_{\text{dest}}),\ t_{\text{now}}\right), \qquad \delta = \text{ETA} - a_{\text{dest}} \qquad (a_{\text{dest}} > d_1), \tag{24}$$

con $C$ el horario corregido (30), que ya incluye la corrección por viaje $\Delta_v$ (34): los dos recogen lo tarde que salen de verdad. La fila es «en directo» porque se sabe que el autobús está allí; si no hay tiempos aprendidos, se usa la ecuación (22). La parada de cabecera ($a_{\text{dest}} \leq d_1$) sale a la hora de salida, $d_1$.

**Capa 2: Llegada Desde la Parada Registrada**

Muchas entradas del feed traen la próxima parada y la hora del registro, pero no una posición utilizable: sin coordenadas, con un GPS fuera del pasillo o con una hora de posición que no es de hoy. La capa 2 aprovecha lo que sí es un dato real: el autobús ha dejado atrás la parada $p_{n-1}$ y va hacia $p_n$, y eso se sabía a la hora $t_{\text{ref}}$.

***El Registro***

Se define el *registro* de una entrada si se cumplen todas estas condiciones:

1. hay viaje, orden $n$ y el par (viaje, vehículo) no está marcado como no fiable (sección «Contraste Entre el GPS y la Parada Registrada»);
2. $n \geq 2$: hay una parada anterior;
3. existe la hora de referencia $t_{\text{ref}}$, que es $t_{\text{gps}}$ si existe y, si no, $t_{\text{rec}}$, y es reciente: $t_{\text{now}} - t_{\text{ref}} \leq 600$ s;
4. el viaje ha empezado: $t_{\text{ref}} \geq d_1 - 60$ s;
5. el viaje tiene paradas de orden $n-1$ y $n$;
6. el tramo es corto: $a_n - a_{n-1} \leq 1200$ s (`REGISTRATION_MAX_SEGMENT_SECONDS`).

Entonces, con $b = \text{franja}(D, t_{\text{ref}})$ y $k = k(\ell, D, t_{\text{ref}})$,

$$\theta = \text{rnd}\left(\tfrac{1}{2} E(J, a_{n-1}, a_n, b, k)\right) \qquad \text{(lo que le falta para llegar a } p_n\text{)}, \tag{25}$$

$$\delta_{\text{reg}} = t_{\text{ref}} + \theta - a_n \qquad \text{(retraso estimado en } p_n\text{)}, \tag{26}$$

$$g = a_n - a_{n-1} \qquad \text{(duración programada del tramo)}. \tag{27}$$

Si el tramo anterior tiene duración programada nula ($a_{n-1} = a_n$, frecuente cuando dos paradas comparten minuto), $E(J, a_{n-1}, a_n, b, k) = 0$ por la regla de los tramos nulos (sección «Tiempo Esperado Entre Dos Instantes del Horario»): $\theta = 0$ y $g = 0$, el autobús se supone ya en $p_n$ y el contraste de la sección «Contraste Entre el GPS y la Parada Registrada» se hace con $\delta_{\text{reg}} = t_{\text{ref}} - a_n$, aunque ese tramo tenga tiempo aprendido.

**Por Qué la Mitad.** El feed no dice en qué punto del tramo está el autobús. Si el instante de la observación es independiente de la posición, el autobús está en cualquier punto del tramo con la misma probabilidad, y el valor esperado del tiempo que le falta es la mitad del tiempo del tramo. Se midieron repartos de 0,35 a 0,65 y el óptimo fue 0,5 (sección «Ideas Probadas y Descartadas»).

**Por Qué Tramos de Hasta 20 Minutos.** En tramos largos (salidas de cabecera, regulaciones) la incertidumbre de «a mitad del tramo» es de muchos minutos. Contrastada con el GPS en 201 autobuses, la parada registrada coincide con mediana de 0,7 min y percentil 90 de 1,7 min en tramos de hasta 5 min, pero se desvía 10 min de mediana en tramos de más de 10 min. Con el umbral de 20 min la capa 2 mejora al horario en todas las longitudes donde se usa; por encima, no.

***La Llegada***

Para una parada de destino con orden $q \geq n$,

$$\boxed{\text{ETA} = \text{rnd}\left(t_{\text{ref}} + \theta + E(J, a_n, a_{\text{dest}}, b, k)\right)} \tag{28}$$

donde el último término es cero si $a_{\text{dest}} \leq a_n$. Para $q < n$ el registro no da llegada (la parada ya ha quedado atrás; véase la sección «Paradas Ya Pasadas»).

**Contraste Entre el GPS y la Parada Registrada**

Cuando una entrada tiene a la vez posición y registro, las dos fuentes deberían contar la misma historia. Si no lo hacen, una de ellas está equivocada (un GPS que pertenece a otro vehículo, un orden de parada que no se actualizó) y no hay forma de saber cuál. La regla es que el par (viaje, vehículo) deja de ser fiable si

$$g \leq 360 \text{ s} \qquad \text{y} \qquad \left|\delta_{\text{pos}} - \delta_{\text{reg}}\right| > 360 \text{ s} \tag{29}$$

Los dos umbrales son las constantes `REGISTRATION_CROSSCHECK_MAX_SEGMENT_SECONDS` y `MAX_GPS_VS_STOP_DISAGREEMENT_SECONDS`. Un par no fiable no tiene posición ni registro durante el resto de la petición, de modo que sus filas pasan a la capa 3. El contraste solo se hace en tramos de hasta 6 minutos, porque en los largos el registro es demasiado impreciso para contradecir a nadie.

Para que el resultado sea reproducible, la posición (con su contraste) debe evaluarse antes que cualquier uso del registro. En el código, cada consulta del estado de una fila empieza por la posición; el registro que se usa dentro del contraste se calcula antes de marcar el par, y todas las consultas posteriores al marcado lo ven como inexistente.

**Ejemplo 5.** En el viaje sintético, un autobús declara $n = 3$ (va hacia $S2$, a 1 000 m), pero su GPS está a 2 800 m, entre $S3$ y $S4$, a las 29 400 s. La proyección elige el candidato «más allá» (a 11 m del trazado, frente a 1 800 m del mejor «esperado») y da $T_{\text{pos}} = 29\,361$, luego $\delta_{\text{pos}} = 39$ s. El registro da $\theta = 70$ y $\delta_{\text{reg}} = 29\,400 + 70 - 29\,010 = 460$ s. Como $g = 120 \leq 360$ y $|39 - 460| = 421 > 360$, el par se marca como no fiable y todas las filas de ese viaje se resuelven sin datos del vehículo.

**Capa 3: Horario Corregido**

Sin ningún dato fiable del vehículo, la mejor estimación de su llegada es la de un autobús que sale de la cabecera a su hora y recorre cada tramo en el tiempo aprendido. Si la red tiene tabla de tiempos por tramo, el viaje tiene paradas y $a_{\text{dest}} > d_1$, se toma la franja y el factor a mitad de camino,

$$t_{\text{mid}} = \left\lfloor \frac{d_1 + a_{\text{dest}}}{2} \right\rfloor, \qquad b = \text{franja}(D', t_{\text{mid}}), \qquad k = k(\ell, D', t_{\text{mid}}),$$

donde $D'$ es la fecha de servicio $D$, salvo en un festivo entre semana, en que es el domingo de esa semana. Las franjas solo miran el día de la semana, así que el festivo usa las de fin de semana. El *horario corregido* es

$$\boxed{C = d_1 + 60 + \text{rnd}\left(E(J, d_1 - 1, a_{\text{dest}}, b, k)\right) + \text{rnd}(\Delta_v).} \tag{30}$$

Los 60 s (`SCHEDULE_ORIGIN_OFFSET_SECONDS`) recogen que los autobuses salen de cabecera algo después de la hora publicada; el resumen de cada captura vigila que este sesgo siga siendo el medido. El inicio en $d_1 - 1$ incluye los tramos de duración nula del principio del viaje (sección «Tiempo Esperado Entre Dos Instantes del Horario»). El último término, $\Delta_v$, es la corrección por viaje de la sección «Corrección por Viaje»; sin tabla de correcciones vale 0.

La decisión es:

1. Si $C \geq t_{\text{now}}$: estado `estimated`, ETA $= C$, $\delta = 0$. El autobús todavía no debería haber pasado y $C$ es una estimación mejor que el horario publicado.

2. Si no, y además

$$C < t_{\text{now}} - 600, \qquad a_{\text{dest}} - C \geq 600, \qquad \lambda \geq 0{,}8, \tag{31}$$

el estado es `likely_departed` y ETA $= a_{\text{dest}}$ (`WRONG_SCHEDULE_PASSED_SECONDS`,

`WRONG_SCHEDULE_GAP_SECONDS`, `WRONG_SCHEDULE_LEARNED_SHARE`).

3. En otro caso, estado `scheduled`, ETA $= a_{\text{dest}}$, $\delta = 0$.

La *parte aprendida* $\lambda$ del camino es la fracción de su duración programada que recorre tramos aprendidos en la franja $b$:

$$\lambda = \frac{\sum_{i \in I(d_1-1,\, a_{\text{dest}})} \mathbf{1}\left[\tau(\kappa_i, b) \text{ definido}\right] h_i}{\sum_{i \in I(d_1-1,\, a_{\text{dest}})} h_i}, \qquad \lambda = 0 \text{ si el denominador es } 0, \tag{32}$$

con el mismo conjunto de tramos $I$ de la definición 8 y las duraciones programadas completas $h_i$ (sin los pesos $w_i$).

Las tres condiciones de (31), que llevan al estado «probablemente ya ha pasado», identifican un caso concreto y frecuente: un horario publicado mucho más lento que la realidad (típicamente, un viaje con un tramo largo de autopista), en el que el horario corregido dice que el autobús pasó hace más de diez minutos, la hora publicada todavía no ha llegado o acaba de pasar, y la corrección se apoya en tramos medidos en al menos el 80 % del camino. Mostrar la hora publicada como si el autobús fuera a venir haría esperar a alguien por un autobús que ya se fue. No se oculta porque no hay ningún dato del vehículo que lo confirme: se muestra al final de la lista, con su hora publicada y el texto «Sin datos del bus: puede que ya haya pasado», y las alertas de llegada lo tratan como pasado. El texto dice lo que se sabe: en las capturas del 3 de noviembre de 2026, el 97,8 % de los viajes con este estado no apareció en el feed en todo el día (sección «Fiabilidad de lo que Dice la Aplicación»).

***Corrección por Viaje***

Sin la corrección, el horario corregido llegaba de media 131 s antes que el autobús (sección «Corrección por Viaje de la Capa 3»). Hay dos razones. La suma de las medianas de los tramos queda por debajo de la media del recorrido, porque el tiempo de cada tramo tiene una cola larga hacia arriba. Además, algunos viajes salen tarde o se retrasan en el mismo sitio casi todos los días. La corrección $\Delta_v$ recoge cuánto se aparta de costumbre cada viaje del horario corregido.

Un viaje se reconoce por la clave $v = (\ell,\, p_1,\, p_N,\, d_1, N)$: su línea, su primera y su última

parada, su salida de cabecera y su número de paradas. A diferencia del identificador del GTFS, esta clave no cambia cuando el operador publica un horario nuevo con los mismos viajes. Como en el factor de ritmo (sección «Factor de Ritmo»), la línea de la clave y de la tabla por línea se compara sin ceros a la izquierda. El tipo de día $\theta$ es laborable o fin de semana según $D'$, de modo que un festivo entre semana cuenta como fin de semana. La parada de destino, en la posición $i$ del recorrido (0 es la cabecera), cae en el tercio

$$p = \text{mín}\left(2,\ \lfloor 3i/N \rfloor\right). \tag{33}$$

La corrección se busca de lo más fino a lo más general:

$$\Delta_v = \begin{cases} \Delta(v, \theta, p) & \text{si el viaje tiene valor para ese tipo de día,} \\ l(\ell, b_1) & \text{si no, y su línea lo tiene en la franja } b_1 = \text{franja}(D', d_1), \\ g(b_1) & \text{si no, y la franja lo tiene,} \\ 0 & \text{en otro caso.} \end{cases} \tag{34}$$

Las tres tablas se aprenden cada día (sección «Recalibración Diaria»). Un día es festivo entre semana si el horario hace circular ese día, por excepción, algún calendario de solo domingo (máscara de días 64), que es como publica Bizkaibus sus festivos. La corrección solo afecta a la capa 3: las capas 1 y 2 parten de dónde está el autobús y ya recogen lo que le pasa ese día.

***Sin Conexión***

La aplicación puede guardar en el móvil el horario completo de cada red para usarlo sin cobertura. En Bizkaibus, cada viaje lleva además su horario corregido (30), ya con $\Delta_v$, calculado una vez para un laborable y otra para un fin de semana. El móvil usa el de fin de semana en sábado, en domingo y en los festivos, que reconoce con la misma regla de los calendarios de solo domingo. Aplica las reglas de estado del servidor, con la parte aprendida $\lambda$ calculada sobre el recorrido entero: `estimated` si $C \geq t_{\text{now}}$ y `likely_departed` con las condiciones (31). En 1 200

paradas al azar de un laborable, un sábado y un festivo, las horas del móvil y del servidor difieren como mucho 6 s, por el orden de los redondeos y por cómo se cuenta el primer tramo cuando la salida de cabecera va unos segundos después de su llegada.

**Paradas Ya Pasadas**

Antes de calcular una llegada se comprueba si el autobús ya pasó por la parada de destino. La hora de paso $t_{\text{paso}}$ se calcula así:

- Con posición: si $a_{\text{dest}} < T_{\text{pos}}$ y además el feed ya ha dejado atrás esa parada ($q < n$) o el GPS está a más de 150 m de ella (`PASSED_DISTANCE_METERS`), $t_{\text{paso}} = t_{\text{gps}} - (T_{\text{pos}} - a_{\text{dest}})$ (ecuación (23)); si no, pero $q < n$ (el feed ya ha dejado atrás esa parada aunque la proyección ponga al autobús antes), $t_{\text{paso}} = t_{\text{gps}}$; si no, no ha pasado.
- Sin posición, con registro y $q < n$: $t_{\text{paso}} = t_{\text{ref}} - \text{máx}(0,\ a_{n-1} - a_{\text{dest}})$.

Si hay hora de paso, la fila sale como `departed` con ETA $= \text{mín}(t_{\text{paso}}, t_{\text{now}})$ y $\delta = 0$.

**Llegando sin Confirmar el Paso**

El feed de Bizkaibus publica una versión cada 64 s de mediana (de 58 a 72 s) y cada posición llega 66 s después de tomarla (mediana; 92 s en el percentil 90), según las capturas del 5 de noviembre de 2026 de 7:00 a 9:00. La posición que se usa es, por tanto, de hace un minuto o más. Si el autobús va algo más lento de lo esperado, la llegada calculada desde esa posición, $t_{\text{gps}} + R$, puede quedar ya en el pasado sin que el autobús haya llegado a la parada. Antes, la fila salía en directo con esa hora ya pasada y la lista la retiraba al quedar atrás (sección «Retraso y Minutos que Se Muestran»): en esas dos horas, en 170 de las 5 041 llegadas a parada que el GPS ve a menos de 40 m (el 3,4 %), el autobús desaparecía de la lista antes de llegar.

Con posición y sin hora de paso, si ETA $< t_{\text{now}}$ la fila sale con ETA $= t_{\text{now}}$ (en la aplicación, «0 min») y un retraso de al menos $t_{\text{now}} - a_{\text{dest}}$, hasta que el feed o el GPS confirmen el paso. Si $t_{\text{now}} -$ ETA pasa de 180 s (`ARRIVING_MAX_SECONDS`) sin confirmación, la fila sale como `likely_departed` («puede que ya haya pasado») con ETA $= t_{\text{now}}$, en vez de desaparecer. Con las mismas capturas, el autobús desaparece antes de llegar en 22 de las 5 041 llegadas (el 0,4 %;

recorridos que pasan dos veces cerca de la parada o un feed que salta paradas), y a cambio deja de salir algo más tarde después de irse: 104 s de mediana y 152 s en el percentil 90, frente a 61 y 98 s. La ficha del viaje usa la misma regla de paso y la misma llegada. El estado `likely_departed` tiene así dos usos: en la capa 3 lleva la hora publicada y retraso 0; aquí, la hora actual y el retraso que da el GPS. Quien lea la API no debe tratar igual su llegada en los dos casos.

**Registro de Pasos**

Un viaje que ya terminó, o que ha dejado de aparecer en el feed, no tiene entrada, y sus filas irían a la capa 3 como si el autobús no hubiera salido. Para evitarlo, un proceso externo guarda cada minuto, para cada referencia de viaje del feed, el último orden de parada visto y su hora:

$$\text{pasos}[D][\rho] = \big(n,\ t\big), \qquad t = t_{\text{rec}} \text{ si existe; si no, } t_{\text{gps}},$$

que es la hora del registro en que el feed dio ese orden de parada (la capa 2 y la memoria del feed prefieren, en cambio, $t_{\text{gps}}$), donde $D$ es el día del viaje, no el del reloj: pasada la medianoche, un autobús de un viaje de ayer se apunta en pasos$[D-1]$, con $t$ en las horas de su horario (sección «Viajes que Pasan de las 24:00»). Hasta el 7 de noviembre de 2026 se apuntaba en el día del reloj: el autobús de las 23:30 de ayer, visto a las 00:30, quedaba en los pasos de hoy, y las paradas del de las 23:30 de hoy, que es el mismo viaje, salían esa noche como ya pasadas hasta su salida. Los pasos de cada día caducan 36 horas después de terminar ese día. Al responder, solo para los viajes de las filas que no tienen entrada en el feed, se leen los pasos de todas sus referencias (un viaje fusionado puede tener varias, sección «Fusión de Variantes y Alias de Viaje») y se queda el de mayor orden. Si la fila tiene $q < n$, sale como `departed` con ETA $= \min(t, t_{\text{now}})$. Esta comprobación va antes que la capa 3.

**Autobuses que Faltan en una Lectura del Feed**

El feed deja fuera a veces, durante una o dos lecturas, a un autobús que sigue en su viaje. En las capturas del 2 al 4 de noviembre de 2026 pasa unas 700 veces al día con una sola lectura y otras 250 a 400 con dos o tres. Sin más, en esas lecturas las filas del viaje no tienen entrada y

caen a la capa 3, o a «Solo horario» si la hora corregida ya ha pasado, aunque el autobús se viera un minuto antes.

Por eso, al leer el feed, se le añade la última entrada de cada autobús que falta en la lectura nueva si se cumplen las tres condiciones siguientes:

- el dato tiene como mucho $H = 180$ s (`HOLD_SECONDS`): $0 \leq t_{\text{now}} - t_{\text{gps}} \leq H$, con $t_{\text{rec}}$ si no hay posición;
- el autobús no aparece en la lectura nueva con otro viaje;
- su viaje no lo lleva ya otro autobús en la lectura nueva.

La entrada se usa tal cual, y la capa 1 parte de su $t_{\text{gps}}$ (22): como ya se explicó, la antigüedad del dato no sesga la llegada, solo la hace algo menos precisa. La memoria es un fichero temporal por red en el servidor; en un servidor recién arrancado no hay nada guardado y el feed se usa tal cual.

**Tabla 8**

*Llegadas de Autobuses que Faltaban en el Feed, con y sin Guardar su Último Dato*

| Llegada en | *n* | Sin guardar | Guardando 180 s | Mediana del error |
|---|---|---|---|---|
| 0-5 min | 9 572 | 54,5 % | 77,5 % | de 1,8 a 1,0 min |
| 5-10 min | 8 945 | 51,6 % | 65,2 % | |
| 10-20 min | 8 676 | 37,3 % | 42,8 % | |

*Nota.* Aciertos dentro de ±2 min contra el paso real, con las capturas del 2 al 4 de noviembre de 2026: cada lectura en la que falta un autobús visto en los 180 s anteriores y que vuelve después al mismo viaje.

La Tabla 8 recoge el resultado. Guardar el dato mejora en los tres días y en los tres horizontes; con 60, 120 y 300 s la mejora a 0-5 min es del 55,7 al 80,9, del 54,8 al 79,6 y del 54,3 al 76,0 % (cada umbral mide sus propias lecturas, las de autobuses vistos en los $H$ segundos anteriores, y por eso cambia también la cifra sin guardar). Cuanto más viejo es el dato, algo menos acierta, y 180 s cubre la mayoría de los cortes. El coste es el autobús que desaparece de verdad (acaba o se pierde el viaje): sus paradas siguen «en directo» hasta $H$, cuando sin guardar habrían salido con la capa 3, que tampoco se cumple.

**Viajes que Pasan de las 24:00**

El horario cuenta las horas desde la medianoche del día del viaje: un autobús del viernes que pasa a las 02:10 del sábado figura a las 26:10 (94 200 s). En Bizkaibus, 56 viajes pasan de las 24:00 y 43 de ellos salen ya pasadas las 24:00: 15 circulan todas las noches, hasta las 26:05, y con los horarios del viernes y del sábado llegan hasta las 32:10, las 08:10 del día siguiente. El feed, en cambio, da $t_{\text{gps}}$ y $t_{\text{rec}}$ en horas del día en curso, y la referencia de un viaje que circula todos los días es la misma cada día. Con un solo emparejador, el autobús de las 23:30 de ayer, visto a las 00:30, se emparejaría con el viaje de las 23:30 de hoy, y no con el suyo.

Por eso cada entrada del feed se asigna al día de su viaje. Es de un viaje de ayer si la salida de cabecera de su referencia queda más de 12 h por delante de la hora actual (`RealtimeMatcher::daysAgo`):

$$d_0^{\text{feed}} - t_{\text{now}} > 43\,200 \text{ s}. \tag{35}$$

A las 00:30 ($t_{\text{now}} = 1\,800$) son de ayer el viaje que salió a las 23:30 ($d_0^{\text{feed}} = 84\,600$) y el de las 24:10 (87 000). La regla supone que el feed solo trae viajes en marcha o a punto de salir, y que la referencia da la salida en horas del horario, como la de Bizkaibus (1).

Las entradas de ayer van a un emparejador propio en el que todo está en las horas del día del viaje: $t_{\text{gps}}$, $t_{\text{rec}}$ y $t_{\text{now}}$ llevan 86 400 s más (las 02:10 son las 26:10), la fecha de servicio $D$ es la de ayer y el registro de pasos, el de ayer (`RealtimeMatcher::vehiclesOfDay`). El cálculo es el de las secciones anteriores, sin cambios: la franja de las 26:10 de un viernes, por ejemplo, es la `laborable.madrugada` del viernes (definición 4). El emparejador de hoy solo recibe las entradas de hoy, y una entrada sin ninguna hora de hoy no va a ninguno (sección «El Feed en Tiempo Real»).

La aplicación pasa al emparejador de ayer las salidas de ayer que pasan de las 24:00, con sus horas del horario, y devuelve sus llegadas en horas de hoy, ETA − 86 400 (sección «Después de la Última Salida del Día»). De madrugada, la ficha de un viaje es la del de ayer mientras no haya

terminado, hasta 10 min después de su última parada (`LATE_GRACE_SECONDS`); en autobús, solo si el directo no trae ningún autobús en el de hoy. En el mapa de una línea, cada autobús se sitúa con el emparejador de su día.

En la práctica, de 23:58 a 05:00 el feed de Bizkaibus no se actualiza (sección «El Feed en Tiempo Real»), así que los nocturnos salen con la capa 3 o con el horario; el mapa no dice que siga a ningún autobús, y la ficha no dice que a un viaje le falte autobús. Antes, el mapa de la línea 3136 enseñaba toda la noche seis autobuses de las 23:58. El emparejador de ayer sirve en los feeds que siguen de noche y, en Bizkaibus, en los 11 viajes de los horarios del viernes y del sábado que acaban después de las 29:00 (las 05:00 del día siguiente), en las líneas 3923 y 3932. De día ninguna entrada es de ayer: con el código anterior y el nuevo sobre el mismo feed, 28 paradas, 256 fichas y 56 mapas de línea dan las mismas respuestas.

**La Decisión Completa**

El algoritmo de la Figura 6 reúne las reglas anteriores en el orden exacto en que se aplican. Cualquier error inesperado durante el cálculo de una fila (un dato corrupto, una base inaccesible) se captura y la fila sale por la capa 3 o como horario: una fila nunca impide responder a las demás.

***El Tope de 45 Minutos***

Un desfase de más de 45 minutos, de retraso o de adelanto ($|\delta| > 2\,700$ s, `MAX_PLAUSIBLE_DELAY_SECONDS`), casi siempre indica un emparejamiento erróneo, no un retraso real: en los datos, los retrasos reales llegan a $+37$ min y los viajes mal asignados dan valores como $+111$ min. Por encima del tope el retraso no se afirma ($\delta$ nulo), pero la llegada se conserva, porque se basa en la posición real del autobús y sigue siendo útil. Ocultar también la llegada se probó y empeoraba la precisión (del 85,2 % al 84,1 % dentro de $\pm 2$ min).

***El Margen de 120 Segundos***

Una llegada de la capa 2 hasta dos minutos en el pasado (`STALE_GRACE_SECONDS`) se sigue mostrando como estimada: el autobús puede estar en la parada o saliendo de ella. Más allá, la estimación ya no es creíble y la fila vuelve a la capa 3. Con GPS no hay margen: una llegada

ya pasada sale con ETA $= t_{\text{now}}$ hasta que se confirme el paso (sección «Llegando sin Confirmar el Paso»).

**Retraso y Minutos que Se Muestran**

El desfase en la posición $\delta_{\text{pos}}$ no es el retraso que entiende el usuario: compara al autobús con un horario interpolado en un punto donde no hay parada. Lo que el usuario compara es la hora publicada de *su* parada con la llegada que se le muestra. Por eso el retraso que se muestra es siempre una diferencia entre una llegada estimada y una hora publicada:

- **En la lista de una parada**, para cada fila,

$$\text{retraso} = \text{rnd}\big(\delta/60\big)\ \text{min}, \qquad \text{minutos} = \text{rnd}\big((\text{ETA} - t_{\text{now}})/60\big), \tag{36}$$

  con el retraso nulo si $\delta$ es nulo. Las filas con menos de $-2$ minutos se retiran de la lista. Se ordenan por $(\mathbf{1}[\text{estado} = \texttt{likely_departed}],\ \text{ETA})$: por llegada estimada, no por la hora teórica (un autobús adelantado puede llegar antes que uno anterior en el horario), y con los que probablemente ya pasaron al final.

- **En la ficha de un autobús**, el retraso es el de su próxima parada: con posición, $\text{ETA}(p_n) - a_n$ calculado con la capa 1. Se muestra $\text{rnd}(\cdot/60)$ minutos, o nada si pasa de 45 minutos. Sin posición se calcula $\delta_{\text{reg}}$ (o 0 si tampoco hay registro), pero no se afirma: la ficha se marca como estimada y sin retraso, igual que la lista.

La aplicación traduce el retraso $r$ en minutos a un texto: «Retraso $+r$m» si $r > 0$, «Adelantado $r$m» si $r \leq -2$, con su signo («Adelantado $-2$m»), y «En hora» en otro caso. La asimetría es deliberada: un autobús que llega un minuto antes no se anuncia como adelantado, porque la resolución del dato es de un minuto. Nótese el efecto del redondeo de PHP: $\text{rnd}(-90/60) = \text{rnd}(-1{,}5) = -2$, de modo que un adelanto de 90 s ya se anuncia.

Con esta definición, la lista y la ficha nunca se contradicen: se comprobó con el feed real que 0 de 232 salidas mostraban un retraso incoherente con su llegada.

**Después de la Última Salida del Día**

La lista se arma con el calendario del día y, de madrugada, con las salidas del día anterior que pasan de las 24:00, con el calendario de ayer y sus horas en las de hoy ($a - 86\,400$: las 26:10 son las 02:10), mezcladas con las de hoy por hora de salida. Las de autobús pasan por el emparejador de ayer (sección «Viajes que Pasan de las 24:00») y las demás, por el emparejamiento por viaje de la sección «Redes que Publican Retraso o Predicción». Cuando ya no queda ninguna salida por delante (las que acaban de salir siguen en la lista por si alguna viene con retraso), se añaden detrás las primeras del día siguiente, con su calendario y sus horas desplazadas 24 h ($a + 86\,400$), en estado `scheduled` y sin pasar por el motor: el directo de hoy no dice nada de esos viajes. La aplicación las muestra con la hora («mañana 07:00») en lugar de los minutos.

**Ejemplo Completo**

La Tabla 9 recoge las dieciséis situaciones de los vectores de prueba (Apéndice D) sobre el viaje sintético del ejemplo 1, con los tramos aprendidos, las proporciones y el factor del ejemplo 4. Las horas publicadas son 28 800, 28 890, 29 010, 29 310 y 29 430 s (8:00:00 a 8:10:30), salvo en el escenario 8. Todas las cifras se han obtenido con el código de producción.

***Escenario 1***

Es el del ejemplo 1: $T_{\text{pos}} = t_{\text{gps}} = 28\,950$. Las paradas $S0$ y $S1$ tienen $a_{\text{dest}} < T_{\text{pos}}$ y salen como pasadas a las $28\,950 - (28\,950 - 28\,800) = 28\,800$ y 28 890. Para $S2$, $S3$ y $S4$ la llegada es $t_{\text{gps}}$ más los tiempos esperados del ejemplo 4: $28\,950 + 70 = 29\,020$, $\text{rnd}(28\,950 + 270{,}4) = 29\,220$ y $\text{rnd}(28\,950 + 367{,}72) = 29\,318$. El horario publicado de $S3$ y $S4$ era 90 y 112 s más lento que lo esperado, y la aplicación anunciaría «Adelantado −2m» en ambas ($\text{rnd}(-1{,}5) = -2$ y $\text{rnd}(-1{,}87) = -2$).

***Escenario 2***

Sin trazado, la proyección sobre la recta $S1S2$ da el mismo $T_{\text{pos}}$, pero los tramos sin aprender ya no tienen longitud y caen al factor de ritmo: 29 290 y 29 398.

***Escenarios 3 y 4***

Con el GPS a 60 m (fuera del pasillo de 50 m) o sin GPS, se usa el registro: $n = 3$, $t_{\text{ref}} = 28\,950$, $\theta = \text{rnd}(0{,}5 \cdot 140) = 70$. La llegada a $S2$ es $28\,950 + 70 = 29\,020$, y a $S3$, $\text{rnd}(28\,950 + 70 + 200{,}4) = 29\,220$: las mismas cifras que con GPS, porque en este ejemplo el autobús estaba justo a mitad del tramo. Las paradas pasadas salen a $t_{\text{ref}} - \text{máx}(0, a_2 - a_{\text{dest}})$: $28\,950 - 90 = 28\,860$ para $S0$ y $28\,950$ para $S1$. Todas como estimadas y sin retraso.

***Escenario 5***

El contraste de la sección «Contraste Entre el GPS y la Parada Registrada» invalida GPS y registro. La capa 3 da para $S2$ un horario corregido de $28\,800 + 60 + 250 = 29\,110 < t_{\text{now}}$, que no cumple las condiciones de «probablemente pasado», de modo que todo sale como horario publicado.

***Escenario 6***

Sin datos y antes de la salida, el horario corregido es $28\,860 + E$: 28 970 para $S1$ (110 s aprendidos), 29 110 para $S2$ (110+140), 29 310 para $S3$ ($250 + 200{,}4 = 450{,}4$, redondeado a 450) y 29 408 para $S4$ ($450{,}4 + 97{,}32 = 547{,}72$, redondeado a 548). La cabecera $S0$ no tiene $a_{\text{dest}} > d_1$ y sale como horario.

***Escenario 8***

Con todos los tramos aprendidos (110, 140, 240 y 100 s) y un horario publicado que pone $S3$ a las 30 810, el corregido es $C = 28\,860 + 490 = 29\,350$. A las 31 200 se cumple $C < 31\,200 - 600$, $a_{\text{dest}} - C = 1\,460 \geq 600$ y $\lambda = 1$: «probablemente ya ha pasado». Para $S2$, $C = 29\,110$ es posterior a su hora publicada y la regla no se aplica.

***Escenario 9***

Un dato de hace 680 s supera la antigüedad máxima de 600 s: no hay posición ni registro, y las filas van a la capa 3 como en el escenario 6, salvo $S1$, cuyo corregido (28 970) ya es anterior a $t_{\text{now}} = 28\,980$.

***Escenarios 10 y 11***

Con la tabla de corrección de los vectores de prueba (Apéndice D), el viaje tiene en laborable 20, 40 y 60 s para sus tres tercios. Las paradas $S1$ a $S4$ ocupan las posiciones 1 a 4 de cinco y caen en los tercios 0, 1, 1 y 2 (33): el escenario 6 se desplaza 20, 40, 40 y 60 s. En el festivo, $D'$ es el domingo. La tabla de tramos no tiene franja de fin de semana y se usa la de todas horas (100 y 150 s), el factor vale 1, los tramos sin aprender usan las proporciones `all` y la corrección de fin de semana es $-30$, 0 y 30 s. Para $S4$,
$C = 28\,860 + \mathrm{rnd}(100 + 150 + 192 + 92{,}28) + 30 = 29\,424$.

***Escenarios 12 y 13***

Diez minutos antes de salir, el autobús está a 11 m de la primera parada con $n = 2$: el feed ya le apunta a la segunda. Queda situado en la cabecera (sección «En la Cabecera, Antes de Salir»), con $T_{\mathrm{pos}} = a_1$ y $t_{\mathrm{gps}}$ igual a la salida, y todas las paradas salen en directo, $S0$ incluida, que no cuenta como pasada aunque $n = 2$. Las llegadas son las de la capa 3 del escenario 6, porque el autobús aún no ha recorrido nada; el retraso que se afirma es el de esas llegadas frente al horario publicado. A 600 m de la parada (escenario 13) puede estar acabando otro viaje: no se sitúa, y las filas son las del escenario 6.

**Figura 6**

*Estado y Llegada de una Fila (Motor de Autobús)*

**Entrada:** fila $(J, q, a_{\text{dest}})$, entrada del feed $e$ (o ninguna), $t_{\text{now}}$
**Salida:** (estado, ETA, $\delta$)

$P \leftarrow$ posición de $e$ en $J$ con su contraste (el capítulo «Emparejamiento y Localización», la sección «Contraste Entre el GPS y la Parada Registrada»)
$G \leftarrow$ registro de $e$ en $J$ (la sección «Capa 2: Llegada Desde la Parada Registrada»)
**si** $e$ no existe **o** ($P$ no existe **y** $G$ no existe) **entonces**
    **devolver** SINDATOS$(J, q, a_{\text{dest}})$
**si** hay $t_{\text{paso}}$ (la sección «Paradas Ya Pasadas») **entonces**
    **devolver** $(\texttt{departed},\ \text{mín}(t_{\text{paso}}, t_{\text{now}}),\ 0)$
**si** $P$ no existe **y** ($q$ desconocido **o** la capa 2 no da llegada para $q$) **entonces**
    **devolver** SINDATOS$(J, q, a_{\text{dest}})$
**si** $P$ existe **entonces**
    $(\text{ETA}, \delta) \leftarrow$ capa 1, ecuaciones (22) o (23); $\pi \leftarrow$ verdadero
**si no**
    ETA $\leftarrow$ capa 2, ecuación (28); $\delta \leftarrow \text{ETA} - a_{\text{dest}}$; $\pi \leftarrow$ falso
**si** $|\delta| > 2700$ **entonces** $\delta \leftarrow$ nulo
    ▷ retraso no creíble: se conserva la llegada
$\alpha \leftarrow (\text{ETA} < t_{\text{now}} - 120)$ ▷ ya debería haber pasado
**si** $\alpha$ **y no** $\pi$ **entonces**
    **devolver** SINDATOS$(J, q, a_{\text{dest}})$
**si no** $\pi$ **entonces**
    **devolver** $(\texttt{estimated},\ \text{ETA},\ 0)$
**si** ETA $< t_{\text{now}}$ **entonces** ▷ llegando, sin confirmar el paso
    **si** $\delta$ no es nulo **entonces** $\delta \leftarrow \text{máx}(\delta,\ t_{\text{now}} - a_{\text{dest}})$
    **si** $t_{\text{now}} - \text{ETA} > 180$ **entonces**
        **devolver** $(\texttt{likely_departed},\ t_{\text{now}},\ \delta)$
    **devolver** $(\texttt{live},\ t_{\text{now}},\ \delta)$
**devolver** $(\texttt{live},\ \text{ETA},\ \delta)$

**función** SINDATOS$(J, q, a_{\text{dest}})$
    **si** $e$ no existe **y** el registro de pasos tiene $(n, t)$ para $J$ con $q < n$ **entonces**
        **devolver** $(\texttt{departed}, \text{mín}(t, t_{\text{now}}), 0)$
    **devolver** capa 3 (la sección «Capa 3: Horario Corregido»): `estimated`, `likely_departed` o `scheduled`

**Tabla 9**

*Estado y Llegada Estimada en Cada Parada del Viaje Sintético*

| Esc. | $t_{\text{now}}$ | $S0$ | $S1$ | $S2$ | $S3$ | $S4$ |
|---|---|---|---|---|---|---|
| 1 | 28 980 | S 28 800 | S 28 890 | V 29 020 (+10) | V 29 220 (−90) | V 29 318 (−112) |
| 2 | 28 980 | S 28 800 | S 28 890 | V 29 020 (+10) | V 29 290 (−20) | V 29 398 (−32) |
| 3 | 28 980 | S 28 860 | S 28 950 | E 29 020 | E 29 220 | E 29 318 |
| 4 | 28 980 | S 28 860 | S 28 950 | E 29 020 | E 29 220 | E 29 318 |
| 5 | 29 420 | P 28 800 | P 28 890 | P 29 010 | P 29 310 | P 29 430 |
| 6 | 28 700 | P 28 800 | E 28 970 | E 29 110 | E 29 310 | E 29 408 |
| 7 | 29 600 | P 28 800 | P 28 890 | P 29 010 | P 29 310 | P 29 430 |
| 8 | 31 200 | P 28 800 | P 28 890 | P 29 010 | Pp 30 810 | Pp 30 930 |
| 9 | 28 980 | P 28 800 | P 28 890 | E 29 110 | E 29 310 | E 29 408 |
| 10 | 28 700 | P 28 800 | E 28 990 | E 29 150 | E 29 350 | E 29 468 |
| 11 | 28 700 | P 28 800 | E 28 930 | E 29 110 | E 29 302 | E 29 424 |
| 12 | 28 210 | V 28 800 (0) | V 28 970 (+80) | V 29 110 (+100) | V 29 310 (0) | V 29 408 (−22) |
| 13 | 28 210 | P 28 800 | E 28 970 | E 29 110 | E 29 310 | E 29 408 |
| 14 | 29 100 | S 28 800 | S 28 890 | V 29 100 (+90) | V 29 220 (−90) | V 29 318 (−112) |
| 15 | 29 040 | S 28 800 | S 28 890 | V 29 040 (+30) | V 29 214 (−96) | V 29 311 (−119) |
| 16 | 29 320 | S 28 800 | S 28 890 | Pp 29 320 (+310) | V 29 320 (+10) | V 29 320 (−110) |

*Nota.* Esc.: escenario. Llegadas en segundos. V: `live`; E: `estimated`; S: `departed`; P: `scheduled`; Pp: `likely_departed`. Entre paréntesis, el retraso en segundos cuando se afirma. Escenarios: 1, GPS a 700 m del origen y 11 m del trazado; 2, el mismo sin trazado; 3, GPS a 60 m del trazado; 4, sin GPS, con $n = 3$ y $t_{\text{ref}} = 28\,950$; 5, GPS y registro discrepan; 6 y 7, sin datos del autobús antes de salir y con el horario corregido ya pasado; 8, horario publicado muy lento ($S3$ y $S4$ a 30 810 y 30 930); 9, GPS de hace 680 s; 10, como el 6, con la corrección por viaje de los vectores de prueba; 11, el 10 en un festivo; 12, el autobús espera en la primera parada 10 min antes de salir; 13, el mismo a 600 m de ella; 14, como el 1 con $t_{\text{now}} = 29\,100$: la llegada a $S2$ ya pasó sin confirmar el paso; 15, GPS 50 m pasada $S2$ con el feed aún en ella (no basta para darla por pasada); 16, como el 14 con $t_{\text{now}} = 29\,320$, más de 3 min llegando.

## Aprendizaje Continuo

Los parámetros del modelo (los tiempos por tramo, las proporciones por longitud y los factores de ritmo) no se escriben a mano: se aprenden del propio feed en tiempo real. Este capítulo describe el ciclo completo: cómo se captura el feed, cómo se reconstruye el instante real en que cada autobús pasa por cada parada, qué muestras se extraen, cuánto se conservan y con qué reglas se decide si una nueva calibración sustituye a la anterior.

### El Ciclo

1. **Captura**, continua y en bloques de una hora de reloj: se descarga el feed cada 20 s y se guarda cada versión distinta.

2. **Muestras y medida**, al terminar cada bloque: se reconstruyen los pasos por parada, se extraen las muestras de tramo y de ritmo y se añaden a dos ficheros acumulados; además se mide la precisión de las llegadas (capítulo «Resultados») y se archivan las capturas en bruto del bloque.

3. **Recalibración**, cada día a las 04:00 UTC: con todo lo acumulado se calculan nuevos parámetros, se validan y, solo si mejoran y cambian, se escriben.

4. **Despliegue**, cada día a las 03:00 UTC: la aplicación se reconstruye con la última versión de los parámetros. Producción usa, por tanto, parámetros de entre 23 y 47 horas de antigüedad, salvo despliegue manual.

Todo el ciclo usa el mismo código de localización y predicción que producción (la misma clase), de modo que lo que se aprende y lo que se mide es lo que ve el usuario. La captura se ejecuta en una plataforma de integración continua que limita cada trabajo a seis horas y que retrasa o se salta muchos trabajos programados: hasta el 31 de octubre de 2026 la captura era de 45 minutos cada hora y solo se ejecutaron 163 de unas 600 capturas previstas, algunas con horas de retraso. Desde entonces cada trabajo captura unas cinco horas y media y, si termina bien, lanza

el siguiente, que espera su turno y empieza en cuanto acaba el anterior; un disparo programado cada hora queda como respaldo.

**Captura del Feed**

Cuando se fijó el intervalo de descarga, en octubre de 2026, el servidor del feed publicaba una versión nueva cada 55 s aproximadamente (el 5 de noviembre, cada 64 s de mediana). Cada 20 s (`intervalo_s`) se descarga, y la versión se guarda solo si su marca `RecordedAtTime` es distinta de la anterior. Con el intervalo inicial de 45 s se perdía alguna versión cuando dos publicaciones caían entre dos descargas (46 versiones guardadas en 45 minutos, de unas 49 publicadas); con 20 s se guardan todas, y muestrear más deprisa no aporta nada, porque el feed no cambia entre publicaciones.

La captura se organiza en bloques que terminan a la hora en punto, de modo que ninguno cruza la medianoche y cada uno usa el horario de su día. Las capturas no usan el emparejador de ayer de la sección «Viajes que Pasan de las 24:00»: la parte de un viaje posterior a la medianoche no se sitúa con el GPS, y no aporta muestras ni entra en las medidas. En Bizkaibus afecta sobre todo a los 11 viajes de fin de semana que acaban después de las 29:00, porque el resto de la noche el feed no se actualiza. Mientras se captura un bloque se procesa el anterior, sin huecos entre ellos. Para medir la precisión de las predicciones de un bloque se usan también las capturas del bloque siguiente, solo para ver cuándo pasan los autobuses que estaban a punto de llegar al terminar el bloque. Las capturas en bruto de cada bloque, comprimidas (unos 250 KB por hora), se archivan junto con las muestras nuevas que salieron de él, sin límite de antigüedad: cualquier cálculo se puede repetir más adelante aunque los ficheros acumulados descarten lo antiguo.

Al procesar una captura, cada fichero se identifica por su `RecordedAtTime` (los duplicados se ignoran) y se convierte a fecha y segundos de Madrid. Las instantáneas se ordenan por tiempo y se agrupan en *ventanas de observación*: una ventana termina cuando el hueco hasta la siguiente instantánea supera 300 s (`MAX_GAP_BETWEEN_SNAPSHOTS_SECONDS`). Cada instantánea guarda el instante final de su ventana, $t_{\text{fin}}$.

**Pistas**

Para cada instantánea, en orden, se construye el emparejador con $t_{\text{now}}$ igual a la hora de la instantánea y la fecha de la instantánea como fecha de servicio, y para cada entrada del feed con viaje conocido se calcula su posición (capa 1, con el contraste de la sección «Contraste Entre el GPS y la Parada Registrada»). Si existe, se añade el punto $(t_{\text{gps}}, T_{\text{pos}})$ a la *pista* de la clave (vehículo, viaje).

**Definición 9** (Pista)**.** *La pista de un vehículo v en un viaje J durante una captura es la sucesión, en el orden de las instantáneas, de los puntos* $(t_k, \theta_k)$*, con* $t_k = t_{\text{gps}}$ *y* $\theta_k = T_{\text{pos}}$ *de cada instantánea en que el vehículo tenía posición en ese viaje.*

Solo se aprende de posiciones GPS válidas: las capas 2 y 3, que son estimaciones, nunca alimentan el aprendizaje, y tampoco las posiciones en la cabecera (sección «En la Cabecera, Antes de Salir»), que no describen ningún recorrido.

**El Instante Real de Paso por una Parada**

No existe en el feed ningún dato de «el autobús pasó por esta parada». Se reconstruye a partir de la pista, usando el horario como escala de distancia: cada $\theta_k$ indica, en horas del horario, dónde estaba el autobús.

**Definición 10** (Paso real)**.** *Dada una pista* $\big((t_k, \theta_k)\big)_{k=1}^{K}$ *y la hora publicada a de una parada, el paso real* $P(a)$ *se obtiene recorriendo los pares consecutivos* $k = 2, \ldots, K$ *y tomando el primero que cumpla*

$$\theta_k > \theta_{k-1}, \qquad t_k - t_{k-1} \leq 300\ s, \qquad \theta_{k-1} \leq a \leq \theta_k;$$

*entonces*

$$P(a) = t_{k-1} + \frac{a - \theta_{k-1}}{\theta_k - \theta_{k-1}} \big(t_k - t_{k-1}\big). \tag{37}$$

*Si ningún par cumple las condiciones, el paso real no se conoce.*

Es una interpolación lineal entre las dos observaciones que rodean a la parada. Los pares en que el autobús no avanza ($\theta_k \leq \theta_{k-1}$, por ejemplo una posición repetida) o separados más de

cinco minutos se descartan, porque no permiten interpolar con fiabilidad.

**Ejemplo 6.** Una pista $(28\,900, 28\,880)$, $(28\,960, 28\,940)$, $(29\,020, 29\,030)$ y una parada con $a = 28\,950$: el par que la rodea es el segundo, y $P = 28\,960 + \frac{28\,950-28\,940}{29\,030-28\,940} \cdot 60 = 28\,966{,}67$ s. Para una parada con $a = 28\,800$, anterior a la primera observación, el paso no se conoce.

Con una posición por minuto, el error de la interpolación está acotado por el tiempo entre dos observaciones: si el autobús se detiene o acelera entre ellas, el paso real puede estar en cualquier punto del intervalo. En la práctica se trata como una resolución de $\pm 30$ s, que es también la del método independiente que se usa para evaluar (sección «La Verdad de Referencia»).

**Muestras de Tramo**

Para cada pista y cada par de paradas consecutivas $(p_i, p_{i+1})$ de su viaje, si se conocen $P_i = P(a_i)$ y $P_{i+1} = P(a_{i+1})$ y $P_{i+1} > P_i$, se produce una muestra de tramo con los campos

| Campo | Valor |
|---|---|
| `track` | «$D\|v\|J$»: fecha, vehículo y viaje |
| `line`, `date` | línea y fecha de servicio |
| `at` | $\lfloor P_i \rfloor$ (truncado: parte entera) |
| `band` | franja$(D, \lfloor P_i \rfloor)$ |
| `from`, `to` | identificadores de $p_i$ y $p_{i+1}$ |
| `scheduled` | $a_{i+1} - a_i$ |
| `actual` | $\mathrm{rnd}_1(P_{i+1} - P_i)$, con un decimal |

La franja es la del momento en que el autobús entra en el tramo. La pista incluye la fecha para que el mismo vehículo en el mismo viaje en dos días distintos cuente como dos pistas.

**Muestras de Ritmo**

Las muestras de ritmo sirven para estimar $k$ (sección «Factor de Ritmo»). En cada instantánea, para cada entrada con posición y cada parada de su viaje con $a_{\text{dest}} > T_{\text{pos}}$, se registra la predicción: la hora de la posición, $T_{\text{pos}}$ y $a_{\text{dest}}$. Cuando se conocen las pistas completas, se calcula el paso real $P(a_{\text{dest}})$ y la predicción se convierte en muestra si cumple:

1. el paso real se conoce;
2. la ventana de observación se extiende al menos 10 minutos después de la hora publicada: $t_{\text{fin}} \geq a_{\text{dest}} + 600$ (`OBSERVED_AFTER_PREDICTION_SECONDS`);
3. el tiempo programado que faltaba es de al menos dos minutos: $H = a_{\text{dest}} - T_{\text{pos}} \geq 120$ s (`MIN_REMAINING_SECONDS_FOR_FACTOR`);
4. el tiempo real que faltaba es positivo: $Y = P(a_{\text{dest}}) - t_{\text{gps}} > 0$.

Se guardan la pista, el vehículo, la línea, la fecha, `at` $= t_{\text{gps}}$, la franja de $t_{\text{gps}}$, $H$, $Y$ y el paso real (estos dos con un decimal).

***Por Qué Exigir la Ventana***

Solo se conoce el paso real de los autobuses que pasan antes de que termine la captura. Si se aceptaran todas las predicciones con paso conocido, las de autobuses lentos (que pasan después del final) quedarían fuera y $k$ saldría sesgado hacia autobuses rápidos: es un problema de *censura*. Exigir que la ventana cubra al menos diez minutos después de la hora publicada reduce ese sesgo a los autobuses que van más de diez minutos tarde respecto a su horario en ese tramo. La misma regla, con la llegada estimada en lugar de la publicada, se aplica al medir el error (capítulo «Resultados»).

***Aclarado***

Un autobús visto durante 45 minutos con veinte paradas por delante produce cientos de predicciones muy correlacionadas. Para que no pese más que otros, se conservan como mucho 30 por pista (`MAX_SAMPLES_PER_TRACK`): si una pista tiene $c > 30$ muestras, en su orden original, se conservan las de índices

$$\left\lfloor \frac{i\,c}{30} \right\rfloor, \qquad i = 0, 1, \ldots, 29. \tag{38}$$

Con $c = 100$ son los índices 0, 3, 6, 10, 13, 16, 20, …, 96: un muestreo uniforme que conserva toda la duración de la pista.

**Acumulación y Retención**

Las muestras se añaden a dos ficheros JSON Lines acumulados (formatos en el Apéndice C). Al añadir:

- se descartan las muestras con fecha anterior a 120 días antes de hoy (`SAMPLES_RETENTION_DAYS`);
- en las muestras de tramo, si un tramo tiene más de 300 de un mismo tipo de día, laborable o fin de semana (`MAX_SAMPLES_PER_SEGMENT`), se conservan las 300 más recientes de ese tipo por (fecha, `at`), sin alterar el orden del resto del fichero;
- en las de ritmo, si el total pasa de 500 000 (`MAX_PACE_SAMPLES`), se ordenan por (fecha, `at`) y se conservan las 500 000 más recientes.

Estos topes mantienen los ficheros por debajo de unos 120 MB y la recalibración por debajo de 2 GB de memoria, con un margen amplio respecto a los límites de la plataforma. El tope hace además de memoria: al quedarse solo lo más reciente, un cambio en la calle (un desvío, unas obras) entra en lo aprendido en uno o dos días.

Hasta el 5 de noviembre de 2026 el tope era de 300 muestras por tramo, sin separar el tipo de día. Con las primeras capturas, 300 muestras eran unos diez días de un tramo muy transitado, y diez días siempre incluyen un fin de semana: se comprobó con 25 días que en los 109 tramos (de 2 487) que llegaban al tope las franjas estaban igual de repartidas que en el resto. Desde el 2 de noviembre las capturas recogen unas diez veces más pasos al día, y el 5 de noviembre 533 de los 2 757 tramos estaban en el tope: muchos guardaban solo los últimos 3 a 5 días, y 127 no tenían ninguna muestra de fin de semana. En esos tramos el sábado y el domingo se usaba el tiempo de laborable, y el lunes, al revés, la lista estaba llena de muestras del fin de semana.

Con un tope por tipo de día cada lista se renueva por su cuenta y siempre queda un fin de semana guardado. Para medirlo se aprendió con las muestras anteriores a cada día y se probó en ese día (Tabla 10). Con el tope de 300 no se puede comparar con los datos guardados, porque el

fichero ya venía recortado por el tope anterior; por eso se reprodujo la misma situación con topes menores en los tramos que conservan todo su historial. El tope por tipo de día nunca empeora y mejora sobre todo el lunes que sigue al fin de semana.

**Tabla 10**

*Error por Tramo con el Tope por Tramo y con el Tope por Tramo y Tipo de Día*

| Tope | Día de prueba | Pasos | Error medio (s) | Dentro de ±30 s (%) |
|---|---|---|---|---|
| 20 | sábado 31 oct. | 22 787 | 19,3 → 19,0 | 82,8 → 83,3 |
| 20 | domingo 1 nov. | 21 441 | 19,1 → 18,9 | 83,3 → 83,6 |
| 20 | lunes 2 nov. | 52 910 | 21,4 → 20,3 | 79,4 → 81,1 |
| 20 | martes 3 nov. | 58 509 | 20,9 → 20,9 | 80,4 → 80,4 |
| 40 | sábado 31 oct. | 22 787 | 19,0 → 19,0 | 83,3 → 83,5 |
| 40 | domingo 1 nov. | 21 543 | 18,9 → 18,7 | 83,7 → 84,0 |
| 40 | lunes 2 nov. | 52 910 | 20,7 → 19,9 | 80,5 → 81,7 |
| 40 | martes 3 nov. | 61 081 | 20,3 → 20,3 | 81,0 → 81,1 |
| 80 | sábado 31 oct. | 22 787 | 19,0 → 19,0 | 83,5 → 83,5 |
| 80 | domingo 1 nov. | 21 543 | 18,8 → 18,7 | 83,8 → 84,0 |
| 80 | lunes 2 nov. | 52 910 | 20,0 → 19,8 | 81,7 → 82,0 |
| 80 | martes 3 nov. | 61 081 | 20,1 → 20,0 | 81,5 → 81,6 |

*Nota.* Error de cada tramo entre dos paradas seguidas, no de la llegada completa: antes de la flecha, con un tope por tramo; después, con un tope por tramo y tipo de día. Se aprendió con las muestras anteriores al día de prueba, con las mismas medianas por franja de la sección «Mediana y Contracción Hacia el Tramo a Cualquier Hora». Solo cuentan los pasos con tiempo aprendido en los dos casos.

Para que una red recién puesta en marcha no empiece sin datos, el repositorio del motor incluye una semilla de muestras de tramo que se usa si todavía no existe el fichero acumulado.

**Recalibración Diaria**

Cada día se recalibran, en este orden, el factor de ritmo, los tiempos por tramo y la corrección por viaje. El orden importa: la validación de los tiempos por tramo compara contra el factor de ritmo que se acaba de escribir, y la corrección por viaje se mide contra el horario corregido que dan los tiempos recién escritos.

***Pliegues***

La validación es cruzada en dos pliegues por *pistas*: cada muestra pertenece al pliegue

$$\text{pliegue}(x) = \text{crc32}\big(\texttt{track}(x)\big) \text{ mód } 2, \tag{39}$$

donde crc32 es el CRC de 32 bits de IEEE 802.3 (el de `zlib`, PNG o Ethernet) aplicado a los bytes UTF-8 de la cadena, interpretado *sin signo*. Todas las muestras de una misma pista caen en el mismo pliegue, de modo que nunca se aprende y se mide con el mismo recorrido (un autobús en un viaje de un día). El mismo vehículo, en otro viaje u otro día, puede caer en el otro pliegue, y los dos pliegues comparten los días. Se aprende con un pliegue y se mide en el otro, y al revés.

***Factor de Ritmo***

1. Se ajustan los factores con todas las muestras (ecuación (18) y los mínimos de la sección «Factor de Ritmo»).

2. Para cada pliegue $f$, se ajustan los factores con las muestras del otro pliegue. Después, para cada muestra de $f$ con factor aplicable $k \neq 1$, se calculan los errores absolutos en minutos

$$e_{\text{antes}} = \frac{|\texttt{at} + H - P|}{60}, \qquad e_{\text{después}} = \frac{|\texttt{at} + kH - P|}{60}.$$

3. Se escribe el fichero de factores solo si hay alguna muestra de prueba, si la media de $e_{\text{después}}$ es estrictamente menor que la de $e_{\text{antes}}$, y si los factores nuevos no son iguales a los del fichero actual.

***Tiempos por Tramo y Proporciones***

1. Para cada pliegue $f$: se ajustan los tiempos por tramo con el otro pliegue y, para cada muestra de $f$ cuyo tramo tenga tiempo aprendido en su franja (ecuación (12)), se calculan

$$e_{\text{antes}} = \frac{|kh - y|}{60}, \qquad e_{\text{después}} = \frac{|\tau - y|}{60},$$

con $k$ el factor de ritmo vigente para la línea, la fecha y el instante de la muestra.

2. Del mismo modo se validan las proporciones por longitud (con la longitud mediana de cada par de paradas en el horario): para cada muestra de prueba con $h > 0$ y respaldo definido, $|k\,h - y|$ frente a $|\tau^{\mathrm{L}} - y|$. Las proporciones se incluyen en la tabla solo si la suma de errores baja.

3. Se escribe el fichero solo si la suma de $e_{\text{después}}$ de los tiempos por tramo es estrictamente menor que la de $e_{\text{antes}}$ y si los tiempos o las proporciones cambian respecto al fichero actual. Si los tiempos por tramo no mejoran, no se escribe nada, aunque mejoren las proporciones.

La línea de base de la validación es siempre lo que haría el sistema sin lo aprendido (el horario por el factor de ritmo). Con las 12 755 medidas iniciales, el error medio por tramo bajó de 0,85 min a 0,39 min en las medidas de prueba cuyo tramo estaba aprendido; con las proporciones por longitud, los tramos sin aprender pasaron de 0,88 a 0,69 min. La Tabla 11 cuenta todas las medidas de prueba, aprendidas o no, y por eso parte de 0,89 min.

***Corrección por Viaje***

1. Con las muestras de tramo se reconstruye el paso real $R$ de cada viaje por cada parada y día: una muestra da el paso por su parada de salida (`at`) y, si falta el de la de llegada, también ese (`at` más el tiempo real del tramo). La pista dice el viaje, que se busca en el horario o en sus alias. Por eso el recalibrado reconstruye antes la base del horario del día.

2. Para cada paso se calcula el desvío $e = R - C_0$, donde $C_0$ es el horario corregido (30) sin $\Delta_v$, con la $D'$ de ese día. Se saltan la cabecera, las paradas repetidas en un recorrido y los pasos a más de 45 minutos de su hora publicada. Cada desvío se recorta a $\pm 900$ s para que un día de atasco no mande.

3. Con $b$ la franja de la salida de cabecera según $D'$, se calculan

$$g(b) = Q_{0,4}(e_b), \qquad l(\ell, b) = \frac{n_{\ell,b}\, Q_{0,4}(e_{\ell,b}) + 20\, g(b)}{n_{\ell,b} + 20},$$
$$\Delta(v, \theta, p) = \frac{\sum_{\text{días}} Q_{0,4}(e_{\text{día}}) + l(\ell, b)}{n_{\text{días}} + 1}, \tag{40}$$

   donde $Q_{0,4}$ es el percentil 40 (con la misma regla que la mediana baja: el valor de la posición $\lfloor 0{,}4(n-1) \rfloor$ de los ordenados), $e_b$ los desvíos de la franja, $e_{\ell,b}$ los de la línea en ella, $n_{\ell,b}$ su número y $e_{\text{día}}$ los del viaje en ese tercio cada día. Un tercio sin pasos toma el valor de su línea. Todo se guarda en segundos enteros. Hasta el 3 de noviembre de 2026 eran medias; el 40 es el percentil de partida, y la recalibración elige cada noche el que más acierta (sección «La Media o un Percentil»).

4. Se aprende con todos los días menos los 7 últimos y se mide en esos 7 (una sola partición; la evaluación de la sección «Corrección por Viaje de la Capa 3» es más exigente y aprende cada día de prueba con todos los anteriores), con cada percentil entre el 30 y el 50; se queda el que más acierta a $\pm 2$ minutos. La tabla, aprendida con todos los días y ese percentil, se escribe solo si el error medio absoluto de la capa 3 baja con la corrección y si cambia respecto a la actual. La herramienta informa además de cada festivo de las muestras, aprendiendo con los demás días.

Aquí la validación es por días y no por pistas: la corrección es de cada viaje, y con pliegues por pistas el mismo viaje de otro día estaría en las dos mitades.

***Qué Garantiza y Qué No***

Las reglas de escritura son conservadoras: un día con datos anómalos (una avería del feed, un festivo atípico) solo cambia el modelo si el cambio mejora la predicción en recorridos que no se usaron para aprenderlo. Si no hay cambios, no se escribe nada y el fichero conserva su fecha. La validación por pistas es una *salvaguarda* diaria, no un método para elegir reglas: para decidir si una regla nueva entra en el método se usa la evaluación por días del capítulo «Resultados», que reproduce mejor lo que pasará con datos futuros.

### Curva de Aprendizaje

Para saber cuánto mejora el modelo con más datos, se separó la mitad de los autobuses de la primera sesión de captura (12 755 medidas de tramo, un laborable a mediodía) para probar, y con la otra mitad se aprendieron los tiempos por tramo usando cada vez más autobuses (Tabla 11).

**Tabla 11**

*Curva de Aprendizaje con la Primera Sesión de Captura*

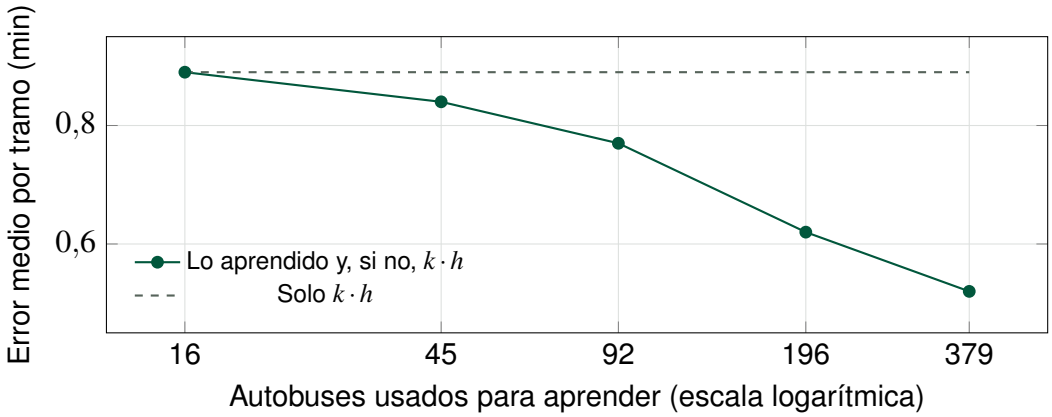
| Autobuses | Tramos vistos | Cobertura | Aprendido y $k$ | Solo $k \cdot h$ |
|---|---|---|---|---|
| 16 | 278 | 1,2 % | 0,89 | 0,89 |
| 45 | 610 | 13,0 % | 0,84 | 0,89 |
| 92 | 1 001 | 32,2 % | 0,77 | 0,89 |
| 196 | 1 458 | 62,8 % | 0,62 | 0,89 |
| 379 | 1 916 | 77,5 % | 0,52 | 0,89 |

*Nota.* Error medio por tramo, en minutos, en los autobuses de prueba. Tramos vistos: tramos distintos en las medidas usadas para aprender (aprendidos o no). Cobertura: parte de las medidas de prueba cuyo tramo tiene tiempo aprendido en su franja; las demás usan $k \cdot h$.

El error no se había estabilizado con 379 autobuses. Desde entonces la captura pasó a ser horaria, y a finales de octubre de 2026 las muestras acumuladas suman 195 786 tramos de 18 604 pistas en 26 días, con 2 454 tramos aprendidos.

**Figura 7**

*Error por Tramo Según el Número de Autobuses*




*Nota.* Las medidas de prueba sin tramo aprendido (100 % menos la cobertura de la Tabla 11) bajan del 98,8 al 22,5 %, un 40–45 % en cada una de las dos últimas duplicaciones, y el error sigue bajando.

## Resultados

Un método de predicción solo vale lo que se puede medir. Este capítulo define la medida del error y explica cómo se obtiene la verdad de referencia sin un dato de «el autobús pasó por aquí». Después presenta los resultados por horizonte, por capa y por cobertura de tramos aprendidos, los repite con 17 días de capturas, evalúa la contracción de la sección «Mediana y Contracción Hacia el Tramo a Cualquier Hora» con un protocolo por días y compara el método, con ese mismo protocolo, con filtros de Kalman y redes neuronales. Termina con las ideas que se probaron y no entraron. La interpretación de los resultados y sus limitaciones están en el capítulo «Discusión».

### Medida del Error

Para cada predicción con paso real conocido, el error es

$$e = \frac{t_{\text{mostrada}} - t_{\text{real}}}{60} \quad \text{minutos}, \tag{41}$$

donde $t_{\text{mostrada}}$ es la llegada que vería el usuario (la ETA del capítulo «Predicción en Tres Capas») o, para comparar, la hora publicada $a_{\text{dest}}$. Un error positivo significa que el autobús llega *antes* de lo que se muestra.

Las predicciones se agrupan por *horizonte*, el tiempo que de verdad faltaba para la llegada, porque la precisión depende sobre todo de él. Para cada grupo se dan:

- el **sesgo**, mediana de $e$: si es distinto de cero, el método se adelanta o se retrasa sistemáticamente;
- la **mediana** y el **percentil 90** de $|e|$;
- el **porcentaje de aciertos** dentro de $\pm 1$, $\pm 2$ y $\pm 3$ minutos.

Los porcentajes dentro de un umbral son la medida principal porque responden a la pregunta del usuario («¿llegaré a tiempo si salgo cuando dice?») y no se dejan dominar por unos pocos errores

grandes, como el error cuadrático medio. Como todas las predicciones están en la misma unidad, las medidas en segundos y minutos son comparables entre métodos (sección «Validación»). Los percentiles se calculan como en todo el método, sin interpolar ($P_p = x_{(\lfloor p(n-1) \rfloor)}$).

**La Verdad de Referencia**

Se usan dos reconstrucciones independientes del instante real de paso.

***Por Posición***

Es la del aprendizaje (definición 10): interpolar la pista del autobús en la hora publicada de la parada. Sirve para calibrar y para la validación cruzada, pero no es del todo independiente de la predicción, porque ambas usan la misma proyección del GPS.

***Por Cambio de Parada***

Para medir lo que ve el usuario se usa una reconstrucción que no depende del GPS: el feed declara en cada instantánea la próxima parada $n$ de cada autobús. Si en la instantánea $m$ un autobús, que sigue en el mismo viaje desde la instantánea de la predicción, declara por primera vez una próxima parada posterior a $q$, el paso por $p_q$ ocurrió entre las instantáneas $m-1$ y $m$, y se toma

$$t_{\text{real}} = \frac{t_{m-1} + t_m}{2}, \tag{42}$$

con $t_m$ la hora más reciente de las posiciones de la instantánea $m$. La resolución es de medio intervalo entre instantáneas, $\pm 30$ s, y el error medido incluye ese margen. Se evalúan las paradas con $n \leq q \leq n+8$ y horizontes de 0 a 20 minutos.

**Resultados**

Las cifras de esta sección se midieron el 14 de octubre de 2026 con 15 capturas del feed real (14 minutos de un laborable), que dan 7 627 predicciones con paso real conocido. Es una muestra pequeña, pero en ella cada llegada se puede calcular con las tres capas a la vez. La sección «Evaluación a lo Largo de Varios Días» repite la medida principal con 17 días de capturas.

**Tabla 12**

*Precisión de la Llegada Mostrada con GPS y del Horario Publicado, por Horizonte*

| La parada queda a | Mediana de $\lvert e \rvert$ | $\pm$1 min | $\pm$2 min | $\pm$3 min | Horario, $\pm$2 min |
|---|---|---|---|---|---|
| 0–3 min | 0,5 min | 80 % | 94 % | 96 % | 19 % |
| 3–6 min | 0,7 min | 63 % | 89 % | 95 % | 18 % |
| 6–10 min | 1,0 min | 49 % | 80 % | 90 % | 16 % |
| 10–20 min | 1,2 min | 41 % | 73 % | 89 % | 13 % |

*Nota.* Capa 1 frente al paso real por la parada; el horario publicado se mide en las mismas llegadas.

***Por Horizonte***

Con GPS, el método acierta dentro de $\pm$2 minutos unas cinco veces más que el horario publicado, y sin sesgo: la mediana de $e$ es cero en todos los horizontes. La precisión decrece de forma regular con el horizonte, como predice la tercera consecuencia de la proposición 2.

***Por Capa***

Para medir las capas 2 y 3 se repite la evaluación sobre las mismas capturas quitando a todos los autobuses el GPS (capa 2) o todo dato del vehículo (capa 3), contra los mismos pasos reales.

**Tabla 13**

*Aciertos Dentro de $\pm$2 Minutos por Capa, en las Mismas Llegadas*

| La parada queda a | Capa 1 GPS | Capa 2 parada registrada | Capa 3 sin datos | Horario publicado |
|---|---|---|---|---|
| 0–3 min | 94 % | 94 % | 42 % | 19 % |
| 3–6 min | 89 % | 84 % | 45 % | 18 % |
| 6–10 min | 80 % | 74 % | 43 % | 16 % |
| 10–20 min | 73 % | 67 % | 39 % | 13 % |

La parada registrada, sin ningún GPS, queda a pocos puntos de la capa 1: el dato que más información aporta es saber qué tramo recorre el autobús y desde cuándo. La capa 3, que no sabe nada del vehículo, duplica o triplica el acierto del horario publicado, porque corrige el error de forma de la interpolación (esta muestra es anterior a la corrección por viaje de la sección

«Corrección por Viaje de la Capa 3»). Por longitud del tramo en que está el autobús, la capa 2 acierta dentro de ±2 min el 89 % en tramos de hasta 3 min, el 83 % de 3 a 6, el 86 % de 6 a 10 y el 52 % de 10 a 20 (frente al 18, 20, 15 y 0 % del horario en las mismas llegadas); en tramos de más de 20 min baja al 7 %, sin mejorar al horario (1 %), y por eso el umbral de la sección «Capa 2: Llegada Desde la Parada Registrada».

***Por Cobertura de Tramos Aprendidos***

La variable que más explica el error, después del horizonte, es si todos los tramos del camino entre el autobús y la parada tienen tiempo aprendido.

**Tabla 14**

*Aciertos de la Capa 1 Según la Cobertura del Camino*

| El autobús llega en | Todo el camino aprendido | Algún tramo sin aprender |
|---|---|---|
| 0–5 min | 96 % ±2 · 99 % ±3 (3 923) | 80 % ±2 · 88 % ±3 (528) |
| 5–10 min | 87 % ±2 · 96 % ±3 (1 811) | 59 % ±2 · 75 % ±3 (369) |
| 10–20 min | 76 % ±2 · 93 % ±3 (439) | 61 % ±2 · 79 % ±3 (77) |

*Nota.* Entre paréntesis, el número de llegadas. Solo cuentan las llegadas en directo, que son las que tienen cobertura calculada: 7 147 de las 7 627.

En llegadas a menos de 10 minutos, con todo el camino aprendido se acierta el 93 % dentro de ±2 min; con un tramo sin aprender, el 76 %; con dos o más, el 59 %. Con el 86 % de las llegadas con todo el camino aprendido, la combinación da el 90 %. El 93 % es un techo para este feed: el 7 % restante se debe a la resolución de ±30 s del paso real, a una sola posición por minuto y a lo que nadie puede prever (semáforos, tráfico, esperas en parada). Superarlo exige otra fuente de datos, no más capturas de esta.

**Evaluación a lo Largo de Varios Días**

Desde el 15 de octubre de 2026, cada captura horaria medía la precisión de la llegada mostrada con el método de la sección «La Verdad de Referencia» y dejaba el resumen en el registro de la plataforma de ejecución. Se recuperaron esos resúmenes: 88 capturas de 17 días (del 15 al 31 de octubre), de las que 61 tienen predicciones (en las de madrugada el feed no trae posiciones de hoy: de 23:58 a 05:00 repite la lectura de las 23:58; sección «El Feed en Tiempo

Real»), con 2 369 529 predicciones contrastadas con el paso real, de las que 2 208 468 quedan a menos de 20 minutos (las de la Tabla 15). Hay dos diferencias con la sección anterior. Estas cifras incluyen todas las llegadas que muestra la aplicación, de cualquier capa, y no solo las de la capa 1. Y el método cambió durante esos días (por ejemplo, la contracción de la sección «Mediana y Contracción Hacia el Tramo a Cualquier Hora» se adoptó al final del periodo), así que miden la aplicación tal como estaba cada día.

Los intervalos de confianza del 95 % se obtienen por *bootstrap* de capturas (Hastie et al., 2009): se remuestrean capturas enteras y no predicciones sueltas, porque las predicciones de una misma captura comparten autobuses, tráfico y meteorología, y tratarlas como independientes daría intervalos demasiado estrechos.

**Tabla 15**

*Precisión de Todas las Llegadas Mostradas en 17 Días, por Horizonte*

| La parada queda a | Predicciones | ±1 min | ±2 min (IC 95 %) | ±3 min | Horario, ±2 min |
|---|---|---|---|---|---|
| 0–3 min | 534 046 | 84 % | 96 % (96–96) | 98 % | 24 % |
| 3–6 min | 485 601 | 67 % | 90 % (90–91) | 96 % | 24 % |
| 6–10 min | 548 058 | 53 % | 82 % (81–82) | 92 % | 23 % |
| 10–20 min | 640 763 | 39 % | 67 % (65–68) | 82 % | 25 % |

*Nota.* Llegadas de cualquier capa en 61 capturas de unos 45 minutos, del 15 al 31 de octubre de 2026. IC: intervalo de confianza por *bootstrap* de capturas. El horario publicado se mide en las mismas llegadas. Los porcentajes de cada captura se redondearon a enteros al imprimirse, y se suman ponderados por su número de predicciones.

La Tabla 15 confirma la Tabla 12 a corto plazo (96 % frente a 94 % dentro de ±2 min a menos de 3 minutos) y la corrige a largo plazo: entre 10 y 20 minutos se acierta el 67 % y no el 73 %, porque la muestra de 14 minutos tenía pocas llegadas lejanas. El horario publicado acierta en torno al 24 % en todos los horizontes. La precisión apenas cambia con el tipo de día: dentro de ±2 min, los laborables (40 capturas) dan el 96, 90, 82 y 67 % por horizonte, y los fines de semana (21 capturas), el 96, 90, 80 y 64 %, con intervalos que se solapan salvo a 6–10 minutos (82–83 frente a 79–81 %).

**Tabla 16**

*Aciertos Dentro de ±2 Minutos en 17 Días, por Origen del Dato y por Cobertura*

| Llegadas a menos de 10 min | Predicciones | ±2 min (IC 95 %) |
|---|---|---|
| *Origen del dato y tramo en que está el autobús* | | |
| GPS, tramo de hasta 10 min | 1 299 494 | 92 % (92–93) |
| GPS, tramo de más de 10 min | 58 410 | 82 % (81–83) |
| Parada registrada, tramo corto | 91 533 | 77 % (76–78) |
| Parada registrada, tramo largo | 1 813 | 59 % (56–62) |
| Sin dato en vivo, tramo corto | 111 557 | 71 % (69–73) |
| Sin dato en vivo, tramo largo | 4 898 | 17 % (14–20) |
| *Tramos sin tiempo aprendido hasta la parada (llegadas en vivo)* | | |
| Ninguno | 1 251 203 | 93 % (93–94) |
| Uno | 120 897 | 76 % (76–77) |
| Dos o más | 49 176 | 63 % (62–65) |

*Nota.* «Parada registrada»: llegada en vivo sin posición GPS utilizable (capa 2). «Sin dato en vivo»: llegadas que no se muestran en vivo (estimadas u horario corregido). Tramo corto: el autobús recorre un tramo de hasta 10 minutos programados.

La Tabla 16 repite con 17 días los desgloses de las Tablas 13 y 14, con el mismo orden: en tramos cortos se acierta el 92 % dentro de ±2 min con la posición GPS, el 77 % con la parada registrada y el 71 % sin dato en vivo, y la cobertura del camino es, de nuevo, la variable que más mueve la precisión (del 93 % con todo el camino aprendido al 63 % con dos o más tramos sin aprender). Desde el 31 de octubre la captura es continua y guarda, por bloque de una hora, los errores en histogramas de 15 s por capa, horizonte, franja y tipo de día, con la llegada calculada también sin GPS y sin feed: la recalibración diaria publica con ellos las mismas tablas con intervalos de confianza, ya separadas por capa.

**Evaluación por Días de la Contracción**

La validación cruzada por pistas del capítulo «Aprendizaje Continuo» decide si se escribe una calibración, pero mezcla en entrenamiento y prueba autobuses del mismo día, que comparten tráfico y meteorología. Para decidir si una *regla* nueva entra en el método se usa un protocolo más exigente, que reproduce lo que ocurre en producción (Hyndman & Athanasopoulos, 2021):

1. Para cada uno de los últimos siete días $D$ de las muestras, se aprende con todas las muestras de días anteriores a $D$ y se prueba con las de $D$.

2. Para cada pista de prueba, ordenada por `at`, y para cada tramo inicial, se encadenan los tramos siguientes mientras sean consecutivos (el destino de uno es el origen del siguiente) y tengan tiempo aprendido. Se van sumando el tiempo real y el predicho con la franja del tramo inicial.

3. Cada vez que el tiempo real acumulado cae en uno de los horizontes $[4,8)$, $[8,12)$ o $[12,20)$ min, se anota el error absoluto de la suma.

4. La regla se adopta si mejora en todos los horizontes y en casi todos los días, no solo en la media.

Es una medida de punta a punta: un cambio puede mejorar cada tramo por separado y no la suma de varios, y eso es lo que ve el usuario. La Tabla 17 compara la mediana de la franja sola ($n_0 = 0$) con la contraída, para dos valores de $n_0$, con las muestras acumuladas a 31 de octubre de 2026 (195 786 tramos de 26 días; días de prueba del 25 al 31 de octubre).[1]

Con $n_0 = 5$ mejoran todas las medidas en todos los horizontes. Con $n_0 = 10$ la mejora tramo a tramo se mantiene a corto plazo, pero a 12–20 min empeora frente a no contraer: la contracción excesiva borra diferencias reales entre franjas, que se acumulan al sumar muchos tramos (el término $\beta^2$ de la proposición 3). Cuando se adoptó la regla, con 193 864 tramos de 25 días, la mejora se observó en los siete días de prueba, y la validación por pistas bajó la suma de errores de 65 854 a 63 204 minutos.

**Corrección por Viaje de la Capa 3**

La corrección por viaje (sección «Corrección por Viaje») se evaluó con el protocolo por días: de 238 740 pasos reales con viaje reconocido en 27 días (del 6 de octubre al 1 de noviembre de 2026), se probaron los 7 últimos, y cada día de prueba aprendió con todos los anteriores (87 333 pasos de prueba).

[1] La herramienta de evaluación usa la mediana del tramo cuando la franja tiene menos de dos pistas, aunque producción solo la guarda si hay muestras de dos franjas; esto afecta por igual a las reglas comparadas.

**Tabla 17**

*Evaluación por Días de la Contracción Hacia la Mediana del Tramo*

| Horizonte | Recorridos | $n_0 = 0$ | $n_0 = 5$ | $n_0 = 10$ |
|---|---|---|---|---|
| *Error medio (s)* | | | | |
| 4–8 min | 107 052 | 42,8 | **42,0** | 42,3 |
| 8–12 min | 83 336 | 62,7 | **61,7** | 62,4 |
| 12–20 min | 113 581 | 89,3 | **88,2** | 89,4 |
| *Dentro de* $\pm$60 *s* | | | | |
| 4–8 min | | 75,6 % | **76,3 %** | 75,9 % |
| 8–12 min | | 58,2 % | **58,9 %** | 58,4 % |
| 12–20 min | | 42,7 % | **43,0 %** | 42,8 % |
| *Dentro de* $\pm$120 *s* | | | | |
| 4–8 min | | 96,0 % | **96,2 %** | 96,0 % |
| 8–12 min | | 87,4 % | **87,9 %** | 87,5 % |
| 12–20 min | | 73,6 % | **74,1 %** | 73,4 % |

*Nota.* Error absoluto medio de la suma de tramos y aciertos dentro de $\pm$60 y $\pm$120 s.

### *Corrección y Partes del Recorrido*

La Tabla 18 da el error de la hora que se mostraría sin ningún dato del autobús, sin corrección y con cada nivel de la corrección.

Quitar el sesgo con una sola cifra por franja baja el error medio, pero empeora los aciertos ajustados ($\pm$60 y $\pm$120 s): el sesgo no es igual en todas las líneas ni en todos los viajes. Con la corrección por viaje y tercio mejoran todas las columnas. El error medio baja un 25 %, y la hora que acierta con $\pm$2 minutos pasa del 47 al 54 %. Partir el recorrido en más trozos no mejora: con menos pasos por trozo, el ruido de cada día pesa más. Los tiempos por tramo de la capa 3 se aprendieron con los 27 días, también los de prueba, y eso favorece a la fila sin corrección: la mejora medida, si acaso, se queda corta.

### *La Media o un Percentil*

Hasta el 3 de noviembre de 2026, cada valor de la corrección era una media de desvíos. Los desvíos tienen una cola larga de retrasos (un día de atasco, un autobús que sale tarde de la cabecera), que sube la media por encima de lo típico: con la media, la hora mostrada quedaba casi siempre algo después del paso del autobús, y el que la creía podía perderlo. Se probó a tomar

**Tabla 18**

*Error de la Hora sin Datos del Autobús, con y sin Corrección*

| Hora mostrada | Error medio | ±60 s | ±120 s | ±300 s | Sesgo |
|---|---|---|---|---|---|
| Horario publicado | 438 s | 13,1 % | 24,4 % | 49,8 % | −263 s |
| Capa 3 sin corrección | 210 s | 27,0 % | 46,6 % | 76,3 % | +131 s |
| Corrección de la franja, *g* | 199 s | 20,9 % | 41,4 % | 80,2 % | +14 s |
| Corrección de la línea, *l* | 177 s | 25,8 % | 47,7 % | 83,4 % | +18 s |
| Un valor por viaje | 164 s | 28,2 % | 51,1 % | 85,6 % | +17 s |
| **Por viaje y tercio**, Δ | **158 s** | **30,3 %** | **54,0 %** | **86,2 %** | +17 s |
| Por viaje y quinto | 159 s | 30,3 % | 53,9 % | 85,7 % | +18 s |
| Por viaje y parada | 162 s | 30,1 % | 53,3 % | 85,3 % | +18 s |

*Nota.* Sesgo: media de la hora real menos la mostrada; positivo si el autobús llega después de lo mostrado. 87 333 pasos del 26 de octubre al 1 de noviembre de 2026. Cada fila usa las anteriores como respaldo: la del viaje usa la de la línea cuando el viaje no tiene datos.

en cada grupo (franja, línea y franja, viaje y tercio de cada día) la mediana o un percentil más bajo, con el resto del cálculo igual.

La prueba es la más exigente posible. Se reconstruyó lo que habría enseñado la aplicación el lunes 2 y el martes 3 de noviembre de 2026, captura a captura (una de cada tres, de las 05:00 a las 23:00). Los tiempos por tramo y la corrección se aprendieron solo con los días anteriores a cada uno, y cada fila «Estimado» sin autobús se comparó con el paso real del autobús por esa parada (Tabla 19).

El percentil 40 acierta más que la media con ±2 minutos los dos días (4,1 y 2,5 puntos), con el mismo acierto a ±5 minutos (0,4 y 0,1 puntos menos) y algo menos de error medio, y baja en un tercio las veces que el autobús pasa más de 2 minutos antes de la hora mostrada. A cambio, hay que esperarlo más a menudo, que es el error barato: llegar un poco antes a la parada no hace perder el autobús. El percentil 35 ya no acierta más que el 40 y pierde a ±5 minutos. Con la prueba de la recalibración (los 7 últimos días, aprendiendo a la vez con todos los anteriores: 216 534 pasos de todos los viajes, con autobús o sin él, hasta el 3 de noviembre), la media da 162 s de error medio y el 51,8 % a ±2 min, y el percentil 40, 160 s y el 54,2 %.

La corrección también da la llegada del autobús que espera en la cabecera (sección «En la

**Tabla 19**

*Media o Percentil en la Corrección por Viaje*

| Valor | ±2 min | ±5 min | Error | Antes | Después |
|---|---|---|---|---|---|
| Media (hasta ahora) | 51,5 / 58,2 % | 83,5 / 87,8 % | 190 / 162 s | 26,6 / 22,4 % | 22,0 / 19,4 % |
| Mediana | 53,6 / 59,8 % | 83,5 / 87,9 % | 187 / 160 s | 22,4 / 19,1 % | 24,0 / 21,2 % |
| **Percentil 40** | **55,6 / 60,7 %** | 83,1 / 87,7 % | **187 / 159 s** | 17,1 / 15,2 % | 27,3 / 24,2 % |
| Percentil 35 | 56,0 / 60,5 % | 82,7 / 87,5 % | 188 / 160 s | 15,1 / 13,7 % | 28,9 / 25,8 % |

*Nota.* Lunes 2 / martes 3 de noviembre de 2026, sin verlos al aprender: 642 427 y 488 640 filas «Estimado» sin autobús con paso real. Error: error medio absoluto. Antes: el autobús pasa más de 2 min antes de la hora mostrada (se pierde si uno se fía); después: más de 2 min después (hay que esperarlo). El paso real es el punto medio entre las dos capturas en las que el feed deja atrás esa parada.

Cabecera, Antes de Salir»). En esas filas, los mismos dos días, el acierto a ±2 min pasa del 53,1 al 58,3 % y del 59,7 al 64,3 %, y las veces que el autobús pasa más de 2 minutos antes de lo mostrado, del 31,8 al 21,5 % y del 26,1 al 17,3 %. «Sin datos del bus» (`likely_departed`) sale en un 13–14 % más de filas, con la misma fiabilidad: el 96,5 y el 94,8 % de esos viajes no se ven en todo el día, como con la media (con una de cada tres capturas de 05:00 a 23:00 y sin ver el día al aprender; la Tabla 23, con una de cada seis capturas de 07:00 a 16:00 y lo aprendido hasta ese mediodía, da el 97,8 % el 3 de noviembre). Los estados con el autobús en ruta y «Ya pasó» no dependen de la corrección y no cambian. Desde ese día, la corrección usa un percentil y no la media.

El percentil no queda fijo: cada noche la recalibración prueba el 30, el 35, el 40, el 45 y el 50 (`TRIP_OFFSET_RANKS`) con la misma prueba de los 7 últimos días, se queda el que más acierta a ±2 min (con empate, el de menos error medio) y lo anota en `trip-offsets.json` (`rank`). Con las muestras del 3 de noviembre, el 35 y el 40 empatan a 54,2 % y gana el 40 por un segundo de error medio (161 y 160 s); el 30 y el 50 dan 53,8 y 53,3 %. Si el tráfico cambia la forma de los desvíos, el percentil cambia con él.

### *Tipo de Día, Festivos y Temporada*

El horario ya distingue los días: cada fecha circula con los viajes de su calendario (laborable, sábado, domingo, festivo, temporada de verano). Lo que se aprende es el tráfico, y la Tabla 20 compara varias formas de agruparlo. El único festivo entre semana de las muestras, el lunes 12 de octubre, se probó aprendiendo con los otros 26 días.

**Tabla 20**

*Error Medio de la Capa 3 Corregida Según el Tipo de Día*

| Tipo de día de la corrección | 7 días de prueba | Festivo del 12 de octubre |
|---|---|---|
| Sin corrección (el festivo, con tramos de laborable) | 210 s | 172 s |
| Laborable o fin de semana por el día de la semana | 158 s | 196 s |
| **Laborable o fin de semana; el festivo, fin de semana** | **158 s** | **162 s** |
| Laborable, sábado y domingo o festivo | 159 s | 156 s |
| Lunes a jueves, viernes, sábado y domingo | 160 s | — |
| Solo con los 14 últimos días | 163 s | — |

Tratar el festivo por su día de la semana le aplicaba el retraso de un laborable con el tráfico de un domingo, y lo dejaba peor que sin corrección. Como fin de semana, con los tramos y la corrección que se aprendieron juntos, baja a 162 s. Separar el sábado del domingo acierta algo más el festivo, pero algo menos el resto, con la mitad de datos por grupo; separar el viernes o usar solo los días recientes empeora. La temporada del año (verano, vacaciones escolares) no se puede medir con muestras de un solo mes. Mientras tanto, lo aprendido se recalcula cada día con las muestras de los últimos 120 días y se va adaptando a la estación.

### Comparación con Otros Métodos

El capítulo «Marco Teórico y Estado del Arte» describe tres familias de métodos: medias históricas, filtros de Kalman y redes neuronales. El método de este trabajo es de la primera. Para saber cuánto se pierde por no usar las otras, se compararon con el protocolo de la sección anterior: los mismos siete días de prueba, los mismos recorridos encadenados y los mismos horizontes. A diferencia del método, los rivales usan también lo que está pasando ese día. Cada uno predice, desde el instante $t_0$ en que el autobús sale del tramo inicial, la suma de los tramos

siguientes, y solo puede usar los días anteriores y los tramos del propio día que terminaron al menos 60 s antes de $t_0$, lo que tarda el feed en publicar una posición nueva. Los métodos son estos:

- **Horario**: la suma de los tiempos programados.
- **Método actual**: las medianas por tramo y franja contraídas con $n_0 = 5$.
- **Escalado**: el método actual multiplicado por un factor único, para separar la corrección del sesgo de todo lo demás.
- **Buses previos**: el método actual multiplicado, en cada tramo, por la mediana del cociente real/aprendido de los últimos tres autobuses que lo terminaron, contraída hacia 1 como si valiera $n_0$ autobuses.
- **Kalman por tramo**: un filtro de Kalman (Kalman, 1960) por tramo cuyo estado es la desviación logarítmica de ese día frente a lo aprendido. Se actualiza con cada autobús que termina el tramo, vuelve a cero con una constante de tiempo $\tau$ (un proceso de Ornstein-Uhlenbeck) y se proyecta a la hora en que el autobús llegará al tramo. Es la idea de Shalaby y Farhan (2004), que usan los tiempos de los autobuses anteriores.
- **Kalman del propio autobús**: el mismo filtro, con la desviación del propio autobús en los tramos que ya ha hecho.
- **Red neuronal**: un perceptrón con dos capas ocultas de 32 y 16 neuronas, como los de Chien et al. (2002) y R. Jeong y Rilett (2005), que predice la desviación logarítmica de cada tramo frente al método actual. Usa 28 variables: el tiempo aprendido y el programado, con cuántos autobuses se aprendió, la distancia y la posición en el recorrido, la hora, la franja y el tipo de día, y lo que se sabe del día (el ritmo del propio autobús, los autobuses previos en el tramo y la desviación media reciente en la red y en la línea). Se entrena para cada día de prueba con los anteriores (entre 424 000 y 632 000 filas), con pérdida de Huber.

- **Red sin datos del día**: la misma red solo con las variables que no dependen de lo que pasa ese día.

Los parámetros de los rivales se eligieron por búsqueda en rejilla con los siete días anteriores a los de prueba (del 18 al 24 de octubre), minimizando la suma de los errores medios de los tres horizontes: factor 1,02; ventana de 30 minutos y $n_0 = 4$ para los buses previos; $\tau = 120$ min para el filtro por tramo y $\tau = 30$ min para el del propio autobús, con una varianza estacionaria del estado de 0,5 y 0,1 veces la del ruido de observación. Ningún parámetro quedó en el borde de la rejilla. El método actual reproduce exactamente la columna $n_0 = 5$ de la Tabla 17, lo que comprueba que todos los métodos se evalúan sobre los mismos recorridos.

**Tabla 21**

*Comparación con Otros Métodos con el Protocolo por Días*

| | Error medio (s) | | | ±120 s | Días mejor |
|---|---|---|---|---|---|
| Método | 4–8 min | 8–12 min | 12–20 min | 12–20 min | que el actual |
| Horario | 153,9 | 233,5 | 329,7 | 24,0 % | 0 |
| Método actual | 42,0 | 61,7 | 88,2 | 74,1 % | — |
| Escalado | 42,6 | 61,9 | 86,9 | 74,9 % | $2 \cdot 4 \cdot 5$ |
| Buses previos | 41,6 | 60,7 | 86,5 | 75,0 % | $5 \cdot 7 \cdot 7$ |
| Kalman por tramo | 41,7 | 60,7 | 86,4 | 75,0 % | $5 \cdot 7 \cdot 7$ |
| Kalman del propio autobús | 40,4 | 59,3 | 84,6 | 76,0 % | $6 \cdot 7 \cdot 7$ |
| Red sin datos del día | 40,7 | 58,8 | 82,5 | 77,2 % | $6 \cdot 6 \cdot 6$ |
| Red neuronal | **39,5** | **57,5** | **81,7** | **77,4 %** | $7 \cdot 7 \cdot 7$ |

*Nota.* Error absoluto medio de la suma de tramos en 107 052, 83 336 y 113 581 recorridos (días de prueba del 25 al 31 de octubre de 2026). «Días mejor que el actual»: de los siete días de prueba, en cuántos el método tiene menos error medio que el actual, en los tres horizontes. Las redes, con la semilla 20261031; con otra semilla, la red neuronal da 39,5, 57,3 y 81,4 s, y la red sin datos del día, 40,8, 59,7 y 85,3 s.

La Tabla 21 muestra que el método actual se queda con casi toda la mejora sobre el horario: el error medio a 4–8 minutos baja de 154 a 42 s. Las alternativas que usan el propio día mejoran algo más y en casi todos los días, pero poco. Los autobuses que acaban de pasar por los

mismos tramos, solos o con un filtro de Kalman, restan entre 0,3 y 1,8 s, un 1 o 2 %. El ritmo del propio autobús resta entre 1,6 y 3,6 s, un 4 %. La red neuronal, que combina todo, resta entre 2,5 y 6,5 s, entre un 6 y un 7 %, y es la única que mejora los siete días en los tres horizontes. Las diferencias con el método actual tienen intervalos de confianza del 95 % que no incluyen el cero (*bootstrap* de pistas; a 12–20 minutos, de $-7{,}9$ a $-5{,}2$ s para la red y de $-4{,}5$ a $-2{,}8$ s para el Kalman del propio autobús).

Corregir solo el sesgo no explica la mejora. Sumar medianas subestima el recorrido largo, porque los tiempos de tramo tienen cola a la derecha: a 12–20 minutos la mediana del error del método actual es de $-29$ s. El factor único la lleva a $-11$ s pero apenas cambia el error medio (de 88,2 a 86,9 s). La red sin datos del día mejora con las dos semillas, pero mucho menos con una que con otra (82,5 y 85,3 s a 12–20 minutos), así que esa parte de la mejora no es fiable. La de la red completa sí lo es: las dos semillas dan lo mismo (81,7 y 81,4 s). La sección «Relación con Otros Enfoques» discute qué significan estas cifras para el método.

**Trazados del Derecho y Espera en la Cabecera**

Los dos cambios del 3 de noviembre de 2026 (secciones «Trazados que No Casan con sus Paradas» y «En la Cabecera, Antes de Salir») se midieron con las capturas de ese día: diez bloques de unos 55 minutos, de 07:00 a 16:00, con el bloque siguiente para ver el paso real de las llegadas del final de cada uno. Son 689 292 llegadas con paso real conocido, de las que 586 425 quedan a menos de 20 minutos. Todas las variantes usan la misma base del horario de ese día y las mismas capturas; solo cambian el motor y la corrección de los trazados:

- A: la aplicación antes de los cambios;
- B: con los trazados corregidos;
- C: B y, además, el autobús en la cabecera saliendo a su hora en punto (22);
- D: B y el autobús en la cabecera con la llegada de la capa 3 (24), que es la adoptada.

La corrección de los trazados (B) mejora en todos los horizontes y no empeora en ninguno. Sus efectos mayores no están en la tabla, que mide la misma lista de llegadas en las

**Tabla 22**

*Aciertos Dentro de ±2 Minutos con los Trazados Corregidos y la Cabecera, por Horizonte*

| La parada queda a | Llegadas | A | B | C | D |
|---|---|---|---|---|---|
| 0–3 min | 124 872 | 94,1 % | 95,5 % | 95,6 % | 95,5 % |
| 3–6 min | 114 153 | 87,6 % | 88,7 % | 89,3 % | 88,7 % |
| 6–10 min | 143 882 | 80,8 % | 80,9 % | 81,0 % | 80,9 % |
| 10–20 min | 203 518 | 67,0 % | 67,0 % | 64,0 % | 67,0 % |

*Nota.* Las mismas 586 425 llegadas a menos de 20 minutos en las cuatro columnas, de cualquier capa. A: antes de los cambios; B: trazados corregidos; C: B con el autobús en la cabecera saliendo a su hora; D: B con la llegada de la capa 3 en la cabecera. Sesgo medio a 10–20 min (media de *e*, negativa si el autobús llega después de lo mostrado): −0,51 (A), −0,54 (B), −1,02 (C) y −0,54 min (D).

cuatro columnas, sino en cuántas de ellas salen en directo: las llegadas calculadas en directo pasan de 470 596 a 507 023, y las que solo tienen el horario, de 115 829 a 79 402. Contando cada autobús con GPS en una de cada tres capturas, el 73,8 % quedaba situado antes de los cambios y el 97,4 % después: el 81,2 % en ruta y el 16,2 % en la cabecera. Los demás aún no han salido y están lejos de la primera parada (1,7 %), van junto al recorrido sin situarse (0,7 %) o han terminado o van a más de 1 km de su recorrido (0,3 %). Con las capturas de ese día, las líneas que aprenden tramos pasan de 95 a 98 de 99, con 2 624 tramos distintos frente a 2 404; las líneas 2610, 3642 y 3932 no aprendían ninguno.

Si el autobús de la cabecera sale a su hora en punto (C), se gana algo a menos de 6 minutos y se pierden 3 puntos a 10–20 minutos, con el doble de sesgo: los autobuses salen después de su hora. Con la llegada de la capa 3 (D), la precisión es la de B en todos los horizontes y las llegadas en directo suben a 569 769, con solo 16 656 de horario. Una captura en directo de 18 minutos de esa misma tarde, con 12 825 llegadas, da lo mismo: D igual o mejor que A en cada horizonte, con 12 421 llegadas en directo frente a 10 941.

### Fiabilidad de lo que Dice la Aplicación

Cada estado de la Tabla 7 afirma algo: que el autobús ya pasó, que espera en la cabecera, que viene. Con las capturas del 3 de noviembre de 2026 se comprobó cada afirmación contra lo que pasó después: para una de cada seis capturas, el estado de cada parada de cada viaje cercano (de hora y media antes a una hora después), con el registro de pasos reconstruido desde las capturas anteriores, frente al paso real por cambio de parada. No cuentan las afirmaciones de la última hora de capturas, sin tiempo para ver qué pasó.

**Tabla 23**

*Lo que Afirma Cada Estado y Cuántas Veces Fue Cierto*

| Estado y lo que afirma | Veces | Resultado |
|---|---|---|
| En directo en la cabecera: el autobús espera allí su viaje | 20 924 | 97,0 % hizo después ese viaje desde allí |
| Ya pasó (`departed`) | 158 833 | 99,9 % había pasado; 0,1 % pasó después |
| Estimado sin autobús (capa 3) | 170 649 | 90,6 % vino un autobús (±2 min el 65 %, ±5 min el 90 %); 5,2 % el viaje no se vio en todo el día |
| Aún sin autobús asignado | 2 372 viajes | 96,7 % recibió autobús después, con una mediana de 7 min antes de su salida (el 17 % después de su hora) |
| Sin datos del bus (`likely_departed`) | 2 601 | 97,8 % de esos viajes no se vio en todo el día; 2,0 % sí había pasado |

*Nota.* Una de cada seis capturas de diez bloques del 3 de noviembre de 2026, de 07:00 a 16:00. El paso real es el primer instante en que el feed da como próxima parada una posterior.

La fila «Estimado sin autobús» se midió con lo aprendido hasta ese mediodía, que ya incluía la mañana medida. Sin ver el día al aprender, el mismo martes da el 58,2 % a ±2 min, y el 60,7 % con la corrección por el percentil 40 (sección «La Media o un Percentil»).

Los estados que afirman algo del autobús aciertan casi siempre. El de `likely_departed` no se puede comprobar: solo se aplica a viajes de los que no hay ningún dato, y casi ninguno aparece después. Por eso su texto pasó de «Probablemente ya ha pasado» a «Sin datos del bus: puede que ya haya pasado», y el del autobús en la cabecera, de «En ruta» a «En la parada».

### Ideas Probadas y Descartadas

La regla del método es que una idea entra solo si la medida mejora. La Tabla 24 recoge las que se probaron y no entraron, con lo medido, para no repetirlas.

**Tabla 24**

*Ideas Probadas y No Adoptadas*

| Idea | Resultado medido | Decisión |
|---|---|---|
| Corregir con la velocidad del último minuto del autobús (la esencia de un filtro) | 92,5 % $\rightarrow$ 91,9 % dentro de $\pm$2 min, y hasta 78,3 % con más peso | Descartada |
| Mezclar lo aprendido con el horario cuando hay pocas muestras | 83,5 % $\rightarrow$ 83,2 % (peso 1) y hasta 80,0 % (peso 8) | Descartada |
| Rellenar los tramos sin aprender con la proporción real/horario del propio viaje | 70 % $\rightarrow$ 67 % | Descartada |
| Franja de cada tramo a la hora prevista de paso por él (no la de la observación) | Sin diferencia en conjunto (71,5 % $\rightarrow$ 71,6 % a 12–20 min); mejor en los recorridos que cruzan de franja a 12–20 min, peor a 4–12 min. Solo el 3–9 % de los recorridos cruza de franja | Descartada |
| Festivos como fin de semana en los tiempos por tramo (capas 1 y 2) | El lunes festivo 12 de octubre se parece más a un laborable: 20,6 s de error con tiempos de laborable frente a 24,2 s con los de fin de semana (387 tramos) | Descartada |

| Idea | Resultado medido | Decisión |
|---|---|---|
| Corrección por viaje con el sábado y el domingo por separado | Festivo del 12 de octubre: 162 → 156 s, pero en los 7 días de prueba 158 → 159 s, con la mitad de datos por grupo | No adoptada |
| Corrección por viaje con el viernes aparte | 158 → 160 s; los viernes, 145 → 151 s | Descartada |
| Corrección por viaje solo con los 14 últimos días | 158 → 163 s | Descartada |
| Corrección por viaje por quintos o por parada en lugar de tercios | 158 → 159 y 162 s | Descartada |
| Mínimo de 1 pista por tramo | Una sola medida supera al respaldo por longitud tramo a tramo (24,1 frente a 40,1 s), pero de punta a punta solo 72 → 74 % a 10–20 min y nada en el resto | No adoptada |
| Sin datos de la franja, la mediana del mismo tipo de día antes que la de todas las horas | 19,5 → 20,0 s de error medio en las 7 493 llegadas afectadas | Descartada |
| Curva no lineal dentro del tramo (exponente sobre la fracción recorrida) | Mejora del 1,3 % con exponente 0,9; el error grande está al principio del tramo y es variabilidad, no sesgo | Descartada |
| Reparto del tramo en la parada registrada distinto de la mitad | 0,5 es el óptimo: 83,0 % frente a 82,1 % (0,35) y 81,4 % (0,65) | Sin cambio |
| Instante exacto del cambio de próxima parada en la capa 2 | 91 % → 91 % en 5 086 llegadas; exigiría guardar estado entre peticiones | Descartada |

| Idea | Resultado medido | Decisión |
|---|---|---|
| Tope de $k$ más alto o mínimos de pistas menores para el $k$ por línea | 128,3 s frente a 128,0 s con la mejor variante; diferencias de $\pm$0,7 s con intervalos del 95 % que incluyen el cero | Descartada |
| Contracción de la franja con $n_0 = 10$ | Mejor tramo a tramo, peor a 12–20 min (Tabla 17) | Descartada |
| Autobús en la cabecera saliendo a su hora en punto | 67,0 % $\rightarrow$ 64,0 % dentro de $\pm$2 min a 10–20 min y sesgo de $-1{,}02$ min (Tabla 22); se usa la capa 3 | Descartada |
| Usar la parada del feed (`StopPointRef`) cuando no coincide con el orden $n$ | 2 329 llegadas más en directo, pero 88,7 % $\rightarrow$ 88,5 % a 3–6 min y 15 ms más por petición | Descartada |
| Arrastrar al viaje siguiente de la misma clave «$\ell \mid v$» el retraso del autobús que aún va en el anterior | Donde se aplica, 60,9 % $\rightarrow$ 5,8 % dentro de $\pm$2 min a 20–60 min y $+22$ min de sesgo: solo el 11 % de esos viajes los hace el mismo autobús | Descartada |
| Seguir el autobús que desaparece del feed desde donde se le vio por última vez (hasta 30 min) | 39,8 % $\rightarrow$ 23,2 % dentro de $\pm$2 min y $-25$ min de sesgo en 22 690 llegadas medidas tras volver al feed: el autobús que falta no suele estar recorriendo el viaje | Descartada |
| Sin autobús pasada su hora de salida, no dar una llegada anterior a salir ahora mismo de la cabecera | En esas filas, del 26,5 al 27,8 % (2 de nov.) y del 32,3 al 32,7 % (3 de nov.) dentro de $\pm$2 min; en el conjunto, $+0{,}1$ y $+0{,}02$ puntos. Muchos de esos autobuses salen a su hora sin aparecer en el feed | Descartada |

| Idea | Resultado medido | Decisión |
|---|---|---|
| Corregir la capa 3 con lo que se apartan ahora los demás viajes de la misma línea, sentido o red | Del 51,5 % al 51,9 % en el mejor caso (2 de nov.) y peor en casi todos: el desvío de un viaje un día es suyo (lo explica el autobús que lo hace), no de la línea | Descartada |

Por el mismo criterio, entraron estas otras:

- el mínimo de 2 pistas por tramo en lugar de 3: un 17 % más de tramos, con el mismo error;
- conservar la llegada cuando el retraso no es creíble;
- la parada registrada hasta tramos de 20 min: sin GPS, del 79,8 % al 83,0 % dentro de $\pm 2$ min;
- el horario corregido de la capa 3: sin datos del autobús, del 18 % al 36 % dentro de $\pm 2$ min;
- la corrección por viaje de la capa 3: de 210 a 158 s de error medio y del 47 al 54 % dentro de $\pm 2$ min;
- en la corrección por viaje, el percentil 40 de los desvíos en lugar de la media: en las filas «Estimado» sin autobús de dos días no vistos al aprender, del 51,5 al 55,6 % y del 58,2 al 60,7 % dentro de $\pm 2$ min, y un tercio menos de autobuses que pasan antes de la hora mostrada;
- en la capa 3, el festivo entre semana como fin de semana: de 196 a 162 s el 12 de octubre;
- el respaldo por longitud: con un tramo sin aprender, del 69 % al 76 %; con dos o más, del 49 % al 59 %;
- la captura horaria, y después continua, y la contracción con $n_0 = 5$;

- descargar el feed cada 20 s en lugar de 45 s: con 45 s se perdía alguna versión (46 guardadas en 45 minutos, cuando a una versión cada 55 s se publican unas 49).

## Discusión

Este capítulo interpreta los resultados del capítulo «Resultados», los sitúa respecto a otros enfoques de la literatura, recoge las limitaciones del trabajo y discute qué hace falta para aplicar el método a otras redes.

### Interpretación de los Resultados

Los resultados permiten separar de dónde sale la precisión del método.

#### *El Tramo en que Está el Autobús*

Saber en qué tramo está el autobús aporta la mayor parte de la información. La capa 2, que solo conoce la próxima parada y la hora del dato, queda a pocos puntos de la capa 1 con GPS (Tabla 13). La posición exacta dentro del tramo mejora la predicción, pero lo decisivo es partir de dónde está el autobús y no de la hora publicada, como anticipa la proposición 2.

#### *La Corrección del Horario*

Corregir el error de forma del horario ya mejora mucho al horario. La capa 3 no sabe nada del autobús y aun así duplica o triplica los aciertos del horario publicado. Esa mejora solo puede venir de sustituir las duraciones interpoladas por los tiempos aprendidos de cada tramo. Aun así le quedaba un sesgo: llegaba de media 131 s antes que el autobús. La corrección por viaje lo deja en 17 s con lo que cada viaje suele hacer, sin saber nada del autobús ese día (sección «Corrección por Viaje de la Capa 3»). Quitar el sesgo de la media no es lo mismo que acertar lo típico: con el percentil 40 de los desvíos en lugar de la media, la hora sin autobús acierta más y el autobús pasa antes de lo mostrado un tercio menos de veces (sección «La Media o un Percentil»). Lo que queda es el desvío de cada viaje ese día, que se explica por el autobús que lo hace: con ese desvío conocido, el acierto a $\pm 2$ min de las filas «Estimado» sin autobús del 2 y el 3 de noviembre pasaría del 52–58 % (con la media) al 81–85 %, pero ni los demás viajes de la línea ni la hora que ya ha pasado sin autobús lo predicen.

### *La Cobertura de Tramos Aprendidos*

La cobertura de tramos aprendidos es lo que más mueve la precisión. Con todo el camino aprendido se acierta el 93 % dentro de ±2 minutos a menos de diez minutos, frente al 76 % con un tramo sin aprender y el 59 % con dos o más (Tabla 14 y el texto que la sigue; con 17 días, el 93, el 76 y el 63 %, Tabla 16). La curva de aprendizaje (Tabla 11) indica que esa cobertura sigue creciendo con los datos.

### *El Error y la Distancia*

El error crece con la distancia y tiene un techo. Cada tramo suma su propia variación, de modo que el acierto baja de forma regular con el horizonte. En las llegadas a menos de diez minutos con todo el camino aprendido, el techo medido está en torno al 93 % dentro de ±2 minutos, limitado por la resolución del dato (una posición por minuto y ±30 s en el paso real) y por lo que ningún modelo puede prever con estos datos: semáforos, tráfico y esperas en parada.

## Relación con Otros Enfoques

El método pertenece a la familia de los modelos basados en tiempos históricos (capítulo «Marco Teórico y Estado del Arte»), con dos diferencias: los tiempos se aprenden por tramo y franja en lugar de por línea, y la posición en tiempo real reinicia la predicción en cada actualización. Otros trabajos han usado filtros de Kalman con datos de localización y de conteo de pasajeros (Shalaby & Farhan, 2004) o redes neuronales (Chien et al., 2002; R. Jeong & Rilett, 2005); en el caso de R. Jeong y Rilett (2005), las redes neuronales superaron a un modelo de datos históricos y a modelos de regresión en una línea de Houston.

Las cifras de este trabajo no se pueden comparar directamente con las de esos estudios: las redes, los datos, los horizontes y las medidas de error son distintos. Por eso se implementaron esos métodos sobre las mismas muestras y con el mismo protocolo por días (sección «Comparación con Otros Métodos»). El resultado tiene dos lecturas. La primera es que el modelo sencillo se queda con casi toda la mejora sobre el horario: de 154 a 42 s de error medio a 4–8 minutos, mientras que la mejor alternativa, una red neuronal que además usa lo que pasa ese día, baja a 39,5 s. La segunda es que esa diferencia, aunque pequeña, es real: la red mejora entre

un 6 y un 7 % todos los días de prueba en todos los horizontes, y un filtro de Kalman sobre el ritmo del propio autobús consigue algo más de la mitad de esa mejora con dos parámetros.

Esto no contradice que corregir con la velocidad reciente del autobús empeorara la medida (sección «Ideas Probadas y Descartadas»). Esa corrección usaba la velocidad del último minuto, muy ruidosa, sobre la llegada completa; el filtro usa la desviación de los tramos ya terminados, la suaviza y la olvida con una constante de 30 minutos. Que los autobuses previos en el mismo tramo aporten tan poco (un 1 o 2 %) indica que, en esta red, la variación de un día a otro en un tramo pesa menos que la de cada conductor y cada viaje, que es la que capta el ritmo del propio autobús. La comparación se hizo sobre sumas de tramos, no sobre la llegada completa que ve el usuario; antes de adoptar el filtro habría que medirlo así, con la regla de la sección «Evaluación por Días de la Contracción».

**Limitaciones**

- **Una muestra pequeña con las tres capas.** Las Tablas 12 a 14, las únicas que calculan cada llegada con las tres capas a la vez, salen de 14 minutos de feed de un laborable; sus cifras a horizonte largo tienen intervalos de confianza anchos. La medida principal se repitió con 17 días (Tabla 15) y corrigió la de 10–20 minutos del 73 al 67 %.

- **Un método que cambia.** Los 17 días miden la aplicación tal como estaba cada día, con calibraciones y reglas que cambiaron en ese periodo; no son la precisión de una versión fija del método.

- **Resolución de la verdad.** El paso real se conoce con $\pm$30 s. Las medidas dentro de $\pm$1 minuto están especialmente afectadas.

- **Definición de paso.** La verdad por cambio de parada mide cuándo el feed cambia de parada, que puede ir algo por detrás del paso físico. Afecta por igual al método y al horario publicado.

- **Censura.** Solo se evalúan los autobuses cuyo paso se observa dentro de la ventana de captura; los muy retrasados pueden quedar fuera.

- **Dependencia temporal.** Las predicciones de un mismo autobús en instantáneas sucesivas están correlacionadas; el número efectivo de observaciones independientes es menor que el número de predicciones.

- **Temporada y festivos.** Las muestras van del 6 de octubre a los primeros días de noviembre de 2026 y solo tienen un festivo entre semana. La regla de los festivos se apoya en ese día, y el efecto de la temporada (verano, vacaciones) está sin medir.

- **Lo que el feed no dice.** Un viaje sin autobús en el feed sale con la capa 3 aunque no se haga, y un refuerzo que no está en el horario no sale en las paradas. Según las capturas del 3 de noviembre de 2026, en torno al 4 % de los viajes que deberían estar en marcha no se pueden interpretar: los que salen tarde y los que no aparecen en todo el día se ven igual en el momento (sección «El Feed en Tiempo Real»).

- **De madrugada.** De 23:58 a 05:00 el feed de Bizkaibus repite la lectura de las 23:58 y los autobuses nocturnos salen con la capa 3 o con el horario (sección «Viajes que Pasan de las 24:00»). Esas llegadas no están medidas, y la parte de un viaje posterior a la medianoche no aporta muestras al aprendizaje.

- **Calidad del horario.** El método depende de que trazado y paradas casen. En el horario de Bizkaibus del 3 de noviembre de 2026, 1 417 de los 6 130 viajes (el 23 %) tenían alguna parada a más de 200 m de su trazado, 670 de ellos (el 11 %) con el trazado al revés, y la corrección de la sección «Trazados que No Casan con sus Paradas» lo arregla en la construcción; otra red puede traer errores distintos que haya que buscar antes de medir el método.

- **Una sola red.** Todos los resultados son de Bizkaibus. La generalización a otras redes es una hipótesis razonable (sección «Aplicabilidad a Otras Redes»), no un resultado.

- **Evaluación hecha por el autor.** El método lo ha evaluado quien lo ha diseñado, con sus propias herramientas. Los vectores de prueba y la implementación de referencia permiten

comprobar que el método descrito es el que se ha medido, pero no sustituyen a una replicación independiente con otros datos.

- **Sin revisión por pares.** Este documento no ha pasado por una revisión externa.

**Límites de la Infraestructura**

Todo el sistema funciona sin un servidor propio y casi sin coste, y eso condiciona el método tanto como los datos.

- **La aplicación, sin estado.** La aplicación y la API corren en una plataforma sin servidor (Vercel): cada petición es una función independiente que no recuerda nada de la anterior, salvo una caché corta en disco y un almacén clave-valor externo (Upstash Redis) que se paga por comando. Cada función admite 250 MB, de los que la aplicación usa 208, y cada red nueva suma su base de datos del horario.
- **El aprendizaje, en una plataforma de pruebas.** La captura y la recalibración corren en una plataforma de integración continua (GitHub Actions) pensada para compilar y probar, no para trabajar sin parar: cada trabajo dura como mucho seis horas y los disparos programados se retrasan o se saltan (sección «El Ciclo»). Hasta el 31 de octubre de 2026 solo se ejecutaron 163 de unas 600 capturas previstas.
- **Los datos, en ficheros.** Las muestras acumuladas no están en una base de datos, sino en ficheros JSON Lines guardados como adjuntos del repositorio. Cada recalibración los descarga enteros, los recalcula en memoria y los vuelve a subir. Por eso existen los topes de la sección «Acumulación y Retención»: 120 días, 300 muestras por tramo y tipo de día y 500 000 de ritmo, con ficheros de unos 100–120 MB y menos de 2 GB de memoria.
- **Una vez al día.** Los parámetros se recalculan una vez al día y producción usa parámetros de entre 23 y 47 horas: un desvío o unas obras de hoy no entran en lo aprendido hasta uno o dos días después.

Estos límites explican varias de las limitaciones anteriores:

- **Memoria corta.** Con 120 días y un tope por tramo no hay un año de historia. No se puede aprender una fiesta con la misma fiesta del año anterior (Navidad, Semana Santa, Aste Nagusia), ni el verano o las vacaciones escolares, y los días especiales se mezclan con los normales al aprender.
- **Grupos gruesos.** Con pocas muestras por tramo, lo aprendido se agrupa en seis franjas y dos tipos de día. Con más historia se podría separar por día de la semana, por hora o por fecha especial sin quedarse sin datos en cada grupo.
- **Sin otras fuentes.** No se cruzan la meteorología, el calendario escolar, los eventos (un partido o un concierto) ni las incidencias publicadas, porque no hay dónde guardar su historia junto a la de los pasos.
- **Sin modelos más capaces.** La red neuronal de la sección «Comparación con Otros Métodos», que usa además lo que está pasando ese día, reduce el error medio de la suma de tramos entre un 6 y un 7 % frente al método. Se descartó para producción porque lo sencillo se puede auditar, pero también porque entrenarla y servirla en vivo exige un servidor que no existe.

Con inversión en infraestructura se podría quitar cada uno de estos límites:

- un servidor que capture el feed sin pausa, sin huecos ni capturas saltadas;
- una base de datos de series temporales que guarde años de pasos sin tope y permita consultar solo lo necesario, en lugar de descargar y recalcular ficheros enteros;
- aprender por fecha especial (festivos, Navidad, fiestas de cada pueblo, vacaciones escolares) comparando con el mismo día del año anterior, y cruzar los pasos con la meteorología, los eventos y las incidencias;
- recalibrar cada hora o de forma continua, de modo que un desvío entre en lo aprendido en horas y no en días;

- servir en vivo modelos más capaces, como la red neuronal ya medida, e intervalos de predicción;
- añadir redes sin el límite de tamaño de cada función.

Ninguna de estas mejoras está medida. Son la dirección que señalan las medidas de este trabajo: la cobertura de tramos aprendidos es lo que más mueve la precisión (sección «Curva de Aprendizaje»), y más historia y más fuentes dan más cobertura y grupos más finos. Cuánto mejoraría la precisión en festivos o en verano solo se sabrá con esos datos y con el mismo protocolo de medida.

**Aplicabilidad a Otras Redes**

El método se diseñó para Bizkaibus, pero casi nada de él es específico de esa red. Esta sección separa lo que es general de lo que hay que adaptar, describe las otras dos situaciones que ya resuelve la aplicación (redes que publican su propio retraso, en el motor, o su propia predicción, fuera de él) y da una lista de pasos para ponerlo en marcha en otra ciudad.

***Requisitos***

El método necesita, por orden de importancia:

1. **Un horario GTFS** con viajes, paradas y horas de paso. Sin él no hay recorrido sobre el que situar al vehículo.
2. **Un feed en tiempo real que identifique el viaje** de cada vehículo y la **próxima parada**, con la hora del dato. Con eso funcionan las capas 2 y 3 y el aprendizaje queda limitado a lo que se pueda reconstruir.
3. **La posición GPS** de cada vehículo y la hora en que se tomó. Con ella funciona la capa 1 y el aprendizaje.
4. **Trazados con distancias** (`shapes.txt` y `shape_dist_traveled`). Sin ellos la localización usa rectas entre paradas, con un pasillo más ancho, y no hay respaldo por longitud.

5. **Una forma de capturar** el feed periódicamente y de guardar las muestras: en este trabajo, un trabajo programado de una plataforma de integración continua y un fichero adjunto a una publicación del repositorio.

Un operador que no publique retrasos ni predicciones, como Bizkaibus, es precisamente el caso para el que el método aporta más. Uno que sí las publique puede usarlas directamente (sección «Redes que Publican Retraso o Predicción»).

***Qué Hay que Adaptar***

**La Lectura del Feed y el Emparejamiento.** Es la única parte que depende del formato concreto del operador: cómo se extraen la línea, el número de viaje y la salida de cabecera de la referencia del viaje (expresión (1)) y cómo se encuentra el viaje del GTFS (sección «Emparejamiento con el Horario»). Si la referencia del feed coincide con el `trip_id` del GTFS, basta la tabla de alias. El motor deduce además de qué día es cada viaje por la salida de cabecera del feed (35), que en Bizkaibus va en horas del horario. Los lectores que la toman de `OriginAimedDepartureTime`, en hora del reloj (Bilbobus, Lurraldebus), tomarían por de hoy un viaje que empezara pasadas las 24:00; en esas dos redes no hay ninguno.

**Las Franjas.** Las seis franjas por tipo de día responden a la demanda de Bizkaia. Una ciudad con otra distribución de la hora punta debería revisarlas, con el mismo protocolo de evaluación por días.

**Las Constantes Medidas.** Las constantes del Apéndice A son de dos clases. Unas son geométricas o de diseño y no deberían cambiar (la ventana de tramos, la regla de desempate, la mitad del tramo en la capa 2). Otras se midieron en Bizkaibus y conviene volver a medirlas: el pasillo de 50 m depende de la calidad de los receptores GPS, los 20 minutos de la capa 2 dependen de la longitud típica de los tramos, el tope de 45 minutos de los retrasos reales de la red y los 60 s de salida de cabecera de la práctica de los conductores.

**La Semilla.** Sin muestras, la tabla de tiempos por tramo está vacía y el modelo se reduce al factor de ritmo, que sin muestras vale 1: las diferencias de horario de la proposición 2, sin corrección de forma. Ya eso mejora mucho al horario solo, porque elimina el retraso acumulado.

Con unos días de capturas aparecen los primeros tramos aprendidos y el modelo mejora sin intervención.

### *El Mismo Motor en Otra Red de Autobús*

El motor se reutiliza por herencia en dos redes más. Lurraldebus (Gipuzkoa) recibe su información en tiempo real en varios feeds SIRI-VM, uno por operador; Bilbobus (los autobuses urbanos de Bilbao) la recibe en un solo feed del Consorcio de Transportes de Bizkaia. En las dos, una clase derivada del emparejador solo cambia la red a la que se aplica y los ficheros con los datos aprendidos (factor de ritmo, tiempos por tramo y correcciones por viaje); el cálculo es el mismo código. La lectura del feed sí es distinta: un lector genérico toma la referencia del viaje tal cual la publica el operador y la empareja con los alias del horario, con el nombre del operador delante cuando hay varios. Las dos redes están activadas, pero sus feeds llegan vacíos durante largos periodos: entonces la aplicación muestra el horario oficial sin afirmar que sigue a ningún vehículo, y el aprendizaje de tramos no tiene con qué empezar hasta que los feeds traigan vehículos de forma regular. Su GTFS no trae los metros de cada parada; la construcción los mide sobre el trazado (tabla 3), así que, cuando lleguen vehículos, se situarán con el mismo pasillo de 50 m que Bizkaibus y no con la recta entre paradas.

### *Redes que Publican Retraso o Predicción*

**Retraso Declarado.** En el código, «red de autobús» es la red para la que se configuró el emparejador; las demás, sean o no de autobuses, usan este camino. En ellas las capas 2 y 3 y el contraste no se aplican, y la entrada se considera en vivo si no tiene posición o si la tiene y es válida. La llegada se calcula a partir del retraso declarado por el operador, $\delta_{\text{decl}}$:

$$\text{ETA}_{\text{plano}} = a_{\text{dest}} + \delta_{\text{decl}}. \tag{43}$$

Si la entrada no tiene posición pero sí parada actual y orden $n$, y el horario da la llegada $a_n$ a esa parada, se calcula también una llegada por diferencias desde ahora,

$$\text{ETA}_{\text{pos}} = t_{\text{now}} + (a_{\text{dest}} - a_n),$$

y se usa en lugar de la plana si $a_{\text{dest}} - a_n \geq 0$ y $|\text{ETA}_{\text{pos}} - \text{ETA}_{\text{plano}}| \leq 900$ s (`POSITION_SANITY_SECONDS`). Con posición válida se usa la capa 1 tal cual. En estas redes no se aplica el tope de 45 minutos ni el estado `estimated`.

**Predicciones de Llegada (GTFS-Realtime Trip Updates).** Metro Bilbao, Euskotren, los tranvías de Bilbao y Vitoria-Gasteiz y Álavabus publican, para cada vehículo, la hora prevista en cada parada que le queda; Renfe, Tuvisa y Dbus, un retraso por viaje. Ahí no hace falta estimar: basta con emparejar cada vehículo con su viaje del horario y comprobar que lo que dice el operador es un dato. Ese trabajo no aprende nada, así que vive en la aplicación y no en el motor.

***Emparejar por el Viaje.*** Todos esos feeds traen el identificador del viaje del horario (con el horario del 4 de noviembre de 2026: los 71 viajes de Metro en servicio, 188 de Euskotren, 46 y 73 de los tranvías), y se empareja por él. La hora en una parada es la del feed; si el feed se salta esa parada, la de la siguiente que trae menos lo que se tarda hasta ella según el horario; con un retraso por viaje, la del horario más ese retraso. El retraso que se enseña es la diferencia con la hora publicada, redondeado a cero por debajo de 60 s (`ON_TIME_SECONDS`). Las horas del feed son del reloj y las del horario pueden pasar de las 24:00: cada viaje guarda los días enteros que separan unas de otras (24 h para un tren anotado a las 26:00 que pasa a las 02:00; 0 el resto del día), y todas las comparaciones los descuentan. Sin eso, ese tren salía con $\pm 1\,440$ min de retraso.

La versión anterior emparejaba por paradas y hora: cada tren del feed con la fila del horario más cercana dentro de 10 minutos, asignando de menor a mayor diferencia. Se suponía que con trenes separados varios minutos la ambigüedad era rara. El 4 de noviembre, con retrasos generalizados de 15 a 20 minutos en Metro, no lo era: cada tren se pegaba a la salida siguiente, la aplicación enseñaba «+2 min» o «4 min adelantado» a trenes que iban 18 minutos tarde y dejaba salidas sin tren como «solo horario». Ese emparejamiento queda solo de reserva, para cuando ningún viaje del feed esté en el horario (por ejemplo, un horario recién cambiado).

***Comprobaciones del Directo.*** Antes de mostrar nada se comprueba, con los datos de cada consulta:

- *Un feed que copia el horario.* Si todas las horas del feed son las del horario al segundo, no es directo y sale el horario. Es el caso de los trenes de Euskotren (2 504 horas, ninguna distinta); los tranvías, en el mismo feed, sí traen horas distintas.

- *Unas horas que contradicen al GPS.* Si, para la mayoría de al menos 3 vehículos con posición, la hora del feed en la parada por la que van se aparta más de 20 minutos de la hora actual (`VEHICLE_DISAGREEMENT_SECONDS`), las horas del feed no valen. Es el caso de Álavabus, que las da con una hora de más.

- *Un viaje que aún no ha salido.* Si no ha salido de su origen y el operador solo repite su hora publicada (Renfe, hasta una hora antes, con retraso cero), sale con el horario.

***Vehículos con Posición y sin Predicción.*** Su retraso es lo que van tarde en la parada por la que van, nunca adelanto, y vale para las que les quedan. No cuentan los que están en la última parada de su viaje ni los que darían más de 45 minutos (`MAX_GPS_DELAY_SECONDS`), que suelen ser vehículos parados o con el viaje anterior.

***Predicción Vencida.*** Si un vehículo aún no ha llegado a la parada a la que va y la hora que le daba el operador ya pasó hace más de 60 s, va al menos ese tiempo más tarde: se suma a esa parada y a las siguientes, y crece mientras no avance. Con más de una hora de desfase no se extrapola (`MAX_STALE_PREDICTION_SECONDS`).

***Salidas sin Datos y Salidas Pasadas.*** En los trenes, una salida que ya debería estar en la estación y no está en el feed va al final como «sin datos del tren». Se miran también las salidas de la última hora, para que un tren muy retrasado siga en la lista, y se retiran las que ya salieron: las en vivo 45 s después de su llegada y las programadas, 60 s después.

***Vigilancia.*** El servidor publica para cada red un diagnóstico (viajes del feed, cuántos encuentra en el horario, cuántos deberían ir en marcha según el horario, si copia el horario o contradice al GPS y la mediana y el percentil 95 de los retrasos que enseña), y un proceso lo revisa cada hora: si un feed deja de responder, si una red deja de encontrar sus viajes o si los retrasos no son creíbles (mediana de más de 30 minutos o percentil 95 de más de 60), avisa. Un

feed vacío, o uno cuyos viajes no están en el horario, solo cuenta como fallo si según el horario van en marcha al menos tres viajes de la red; el vacío, además, si la red no es de las que lo publican vacío a ratos aunque haya servicio (Bilbobus, Lurraldebus y Álavabus). Un feed con vehículos y ninguno con hora de hoy, como el de Bizkaibus de madrugada, queda como aviso. Así se detectaron las horas desfasadas de Álavabus, vehículos ya llegados que seguían contando y trenes de madrugada con $\pm 1\,440$ min de retraso.

***Pasos Para Otra Ciudad***

1. Comprobar qué publica el operador. Con Trip Updates, basta el emparejamiento de la sección anterior y sus comprobaciones. Con SIRI-VM (o GTFS-Realtime Vehicle Positions), hace falta el motor de este trabajo.

2. Construir la base del horario con la fusión de variantes y los alias (sección «Fusión de Variantes y Alias de Viaje») y los trazados con distancias.

3. Adaptar la lectura del feed y el emparejamiento, y comprobar con capturas reales que todos los vehículos del feed encuentran su viaje.

4. Poner en marcha la captura periódica y la recalibración diaria. Sin calibrar, el modelo tiene poco con qué corregir el horario.

5. Medir con el protocolo del capítulo «Resultados» sobre capturas de la propia red antes de mostrar nada como «en directo», y volver a medir las constantes de la sección «Qué Hay que Adaptar».

6. Comprobar que una implementación propia reproduce los vectores del Apéndice D.

# Conclusiones y Líneas Futuras

## Resumen

Esta memoria ha descrito un método para predecir la llegada de autobuses a sus paradas en una red cuyo operador publica la posición de sus vehículos pero ningún retraso, y cuyo horario intermedio es una interpolación a velocidad constante. El método se resume en cuatro pasos:

1. **Situar** el GPS sobre el trazado oficial, en una ventana alrededor de la próxima parada que declara el feed y con una regla de desempate para los recorridos que vuelven sobre sí mismos, y traducir esa posición a la hora a la que el horario pone allí al autobús.

2. **Sumar** el tiempo real que cuesta recorrer lo que falta, tramo a tramo, con tiempos aprendidos por franja, contraídos hacia el tiempo del tramo a cualquier hora, y con una cascada de respaldo por longitud y por factor de ritmo.

3. **Elegir** en cada predicción la mejor fuente fiable entre el GPS, la última parada registrada y el horario corregido, con reglas explícitas para abstenerse y sin afirmar nunca un retraso que no se puede sostener.

4. **Aprender y medir** cada día con el propio feed, y cambiar los parámetros solo si el error baja en recorridos que no se usaron para aprenderlos; las reglas nuevas, solo si mejoran en días no vistos al aprender.

## Resultados

En 2,21 millones de llegadas a menos de veinte minutos, de 17 días, la llegada mostrada cae dentro de $\pm 2$ minutos en el 96 % de los casos cuando el autobús está a menos de tres minutos y en el 67 % entre diez y veinte minutos, frente al 24 % y el 25 % del horario publicado, con intervalos de confianza de uno a tres puntos. En una muestra de 14 minutos con GPS, la llegada no tiene sesgo y acierta el 94 % a menos de tres minutos. Sin GPS, con solo la parada registrada, el acierto es el mismo a menos de tres minutos (94 %) y baja seis puntos entre diez y veinte (del

73 al 67 %). Sin ningún dato del vehículo, el horario corregido duplica o triplica el acierto del horario publicado, y la corrección por viaje baja su error medio de 210 a 158 s; con el percentil 40 de los desvíos en lugar de la media, la hora sin autobús acierta entre 2 y 4 puntos más a $\pm 2$ minutos en días no vistos al aprender. Cuando todo el camino tiene tiempos aprendidos, el 93 % de las llegadas a menos de diez minutos cae dentro de $\pm 2$ minutos, el techo que permite para ellas la resolución de este feed. Con el mismo protocolo por días, un filtro de Kalman sobre el ritmo del propio autobús y una red neuronal reducen el error de las sumas de tramos un 4 % y entre un 6 y un 7 %.

## Lecciones

### *Lo Único Vivo Es la Posición*

Antes de modelar nada hubo que comprobar qué datos eran fiables. El retraso declarado no existía y el horario intermedio era una interpolación. Todo el diseño parte de usar la posición y diferencias de horario, que hacen irrelevante el retraso acumulado (proposición 2).

### *Revisar los Datos de Entrada*

Durante semanas, 670 de los 6 130 viajes (el 11 %, en 25 líneas) no se pudieron situar con el GPS porque el horario oficial dibujaba sus trazados al revés, y esos viajes tampoco aprendían. No era un fallo del método, sino de los datos, y solo se vio al revisar uno a uno los autobuses que no salían en directo. Corregir los trazados en la construcción y situar al autobús que espera en la cabecera subió los autobuses situados del 73,8 al 97,4 %, más que cualquier mejora del modelo.

### *Medir Antes de Adoptar*

Muchas ideas razonables empeoraron la medida: corregir con la velocidad del último minuto, mezclar lo aprendido con el horario, calcular la franja de cada tramo a la hora prevista, tratar los festivos como fin de semana en los tiempos por tramo, suponer que el autobús de la cabecera sale a su hora, arrastrar el retraso del viaje anterior o seguir al autobús que desaparece del feed. Tratar los festivos como fin de semana sí mejora la capa 3 corregida, donde la primera versión los empeoraba: cada caso se midió por separado. La regla de adoptar solo lo que mejora la medida, por días y de punta a punta, ha sido más útil que cualquier intuición sobre el tráfico.

### *Abstenerse Es Parte de la Precisión*

Mostrar «estimado» o «solo horario» cuando no hay datos, ocultar un retraso no creíble y desconfiar de dos fuentes que se contradicen no aumentan los aciertos medidos, pero evitan los errores que más daño hacen a la confianza del usuario.

### *Lo Sencillo y Auditable Compite*

Medianas por tramo, una contracción con una sola constante, una cascada de cuatro niveles: cada parámetro tiene un significado físico y se puede comprobar a mano. Frente a una red neuronal entrenada con lo mismo y con lo que pasa cada día, se pierde entre un 6 y un 7 % de error medio; a cambio, cualquier cifra que muestre la aplicación se puede explicar.

## Líneas Futuras

- **El ritmo del propio autobús.** El filtro de Kalman sobre la desviación del propio autobús mejora las sumas de tramos un 4 % con dos parámetros (sección «Comparación con Otros Métodos»). El paso siguiente es medirlo sobre la llegada completa, con la regla de adopción de la sección «Evaluación por Días de la Contracción».

- **Capas a lo largo de los días.** La captura continua guarda desde el 31 de octubre de 2026 los errores de cada bloque por capa, horizonte, franja y tipo de día. Con semanas de datos, las Tablas 13 y 14 se podrán repetir con intervalos de confianza estrechos.

- **Temporada y tipo de día.** Con meses de muestras se podrá medir si conviene separar el verano y las vacaciones escolares, o el sábado del domingo, en la corrección por viaje y en los tiempos por tramo.

- **Fechas especiales.** Los tiempos aprendidos distinguen laborable y fin de semana, pero no los días de fiestas, eventos o Navidad. Marcar esas fechas para que no cuenten al aprender evitaría que contaminen los días normales; aprenderlas aparte exige capturas del año anterior.

- **Viajes sin autobús.** Distinguir en la aplicación el viaje que ya debería haber salido y no tiene autobús en el feed (en torno al 4 % de los que deberían estar en marcha), en lugar de

darlo como estimado. Antes de su hora, la ficha ya dice que aún no tiene autobús asignado.

- **De madrugada.** De 23:58 a 05:00 el feed de Bizkaibus no se actualiza y los nocturnos salen sin directo (sección «Viajes que Pasan de las 24:00»). Si el operador lo mantiene de noche, el emparejador de ayer los situará sin cambios; para medir esas llegadas y aprender de ellas, la captura tendría que separar también los viajes de ayer.
- **La secuencia de cada vehículo.** En una cabecera, el autobús que empieza un viaje suele ser uno que acaba de terminar otro, y su retraso pasa al siguiente. La clave del feed no lo dice (solo el 11 % de los viajes seguidos de una clave los hace el mismo autobús), pero las capturas guardan qué vehículo hace cada viaje cada día. Si esa secuencia se repite de un día a otro, se podría seguir al autobús en su viaje anterior y estimar la salida real; hay que medir antes si se repite.
- **Revisión externa.** Que otras personas repliquen el método con la implementación de referencia y lo apliquen a otra red.
- **Intervalos de predicción.** Las muestras permiten estimar, además de la mediana de cada tramo, su dispersión. Sumando varianzas por tramo se podría mostrar un intervalo («llega en 6–8 minutos») con una cobertura medida.
- **Más capacidad técnica.** Un servidor propio y una base de datos con años de historia quitarían los topes de la sección «Límites de la Infraestructura»: aprender festivos, fiestas y temporadas con el año anterior, cruzar meteorología y eventos, recalibrar cada hora y servir modelos más capaces. Hay que medir cuánto mejora cada paso.
- **Más fuentes de datos.** Pasar del 93 % en las llegadas a menos de diez minutos con todo el camino aprendido exige información que este feed no tiene: posiciones más frecuentes, apertura de puertas o conteo de viajeros.
- **Incidencias.** Los cortes y desvíos publicados por el operador no entran hoy en el cálculo hasta que se notan en la posición. Incorporarlos exige medir antes cómo afectan.

- **Planificación de viajes.** Las llegadas estimadas pueden alimentar un buscador de rutas con transbordos (por ejemplo, con el algoritmo RAPTOR), de modo que el viaje propuesto use las horas reales y no las publicadas.
- **Más redes.** Aplicar el método a otras ciudades y comprobar cuánto de lo medido aquí se mantiene.

### Reproducibilidad

Para que cualquiera pueda reproducir el método, esta memoria incluye todas las constantes (Apéndice A), el pseudocódigo (Apéndice B), los formatos de datos (Apéndice C), vectores de prueba generados con el código de producción (Apéndice D) y una implementación de referencia independiente que los reproduce exactamente (Apéndice E). Una implementación nueva, en cualquier lenguaje, puede darse por correcta cuando reproduce todos los vectores al segundo.

## Referencias

Brakewood, C. y Watkins, K. (2019). A literature review of the passenger benefits of real-time transit information. *Transport Reviews*, *39*(3), 327-356. https://doi.org/10.1080/01441647.2018.1472147

Cathey, F. W. y Dailey, D. J. (2003). A prescription for transit arrival/departure prediction using automatic vehicle location data. *Transportation Research Part C: Emerging Technologies*, *11*(3–4), 241-264. https://doi.org/10.1016/S0968-090X(03)00023-8

Cats, O. y Loutos, G. (2016). Real-time bus arrival information system: An empirical evaluation. *Journal of Intelligent Transportation Systems*, *20*(2), 138-151. https://doi.org/10.1080/15472450.2015.1011638

Chien, S. I.-J., Ding, Y. y Wei, C. (2002). Dynamic bus arrival time prediction with artificial neural networks. *Journal of Transportation Engineering*, *128*(5), 429-438. https://doi.org/10.1061/(ASCE)0733-947X(2002)128:5(429)

Comité Europeo de Normalización. (2022). *Public transport. Service interface for real-time information relating to public transport operations. Part 3: Functional service interfaces* (Norma europea N.º EN 15531-3:2022). https://iteh.eu/catalog/standards/cen/25167cb0-42d9-4dd8-abf6-15c7c49f0402/en-15531-3-2022

Efron, B. y Morris, C. (1975). Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, *70*(350), 311-319. https://doi.org/10.1080/01621459.1975.10479864

Gal, A., Mandelbaum, A., Schnitzler, F., Senderovich, A. y Weidlich, M. (2017). Traveling time prediction in scheduled transportation with journey segments. *Information Systems*, *64*, 266-280. https://doi.org/10.1016/j.is.2015.12.001

García-Mauriño, C., Zufiria, P. J. y Jarabo-Peñas, A. (2024). Improving bus arrival time predictors using only public transport API data. *Transportation Letters*, *16*(8), 804-813. https://doi.org/10.1080/19427867.2023.2245994

Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A. y Rubin, D. B. (2013). *Bayesian data analysis* (3.ª ed.). CRC Press. https://sites.stat.columbia.edu/gelman/book/

Hastie, T., Tibshirani, R. y Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2.ª ed.). Springer. https://doi.org/10.1007/978-0-387-84858-7

Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, *35*(1), 73-101. https://doi.org/10.1214/aoms/1177703732

Hyndman, R. J. y Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3.ª ed.). OTexts. https://otexts.com/fpp3/

Hyndman, R. J. y Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, *22*(4), 679-688. https://doi.org/10.1016/j.ijforecast.2006.03.001

James, W. y Stein, C. (1961). Estimation with quadratic loss. En J. Neyman (Ed.), *Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability* (Vol. 1, pp. 361-379). University of California Press. https://projecteuclid.org/euclid.bsmsp/1200512173

Jeong, R. y Rilett, L. R. (2005). Prediction model of bus arrival time for real-time applications. *Transportation Research Record: Journal of the Transportation Research Board*, *1927*, 195-204. https://doi.org/10.1177/0361198105192700123

Jeong, S., Oh, C. y Jeong, J. (2024). BAT-Transformer: Prediction of bus arrival time with transformer encoder for smart public transportation system. *Applied Sciences*, *14*(20), Artículo 9488. https://doi.org/10.3390/app14209488

Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, *82*(1), 35-45. https://doi.org/10.1115/1.3662552

Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. En *Proceedings of the 14th International Joint Conference on Artificial Intelligence* (Vol. 2, pp. 1137-1145). Morgan Kaufmann. https://mlanthology.org/ijcai/1995/kohavi1995ijcai-study

Kumar, B. A., Singh, R., Shaji, H. E. y Vanajakshi, L. (2025). Bus arrival time prediction: A comprehensive review. *IEEE Transactions on Intelligent Transportation Systems*, *26*(6), 7362-7379. https://doi.org/10.1109/TITS.2025.3545695

Lin, W.-H. y Zeng, J. (1999). Experimental study of real-time bus arrival time prediction with GPS data. *Transportation Research Record: Journal of the Transportation Research Board*, *1666*, 101-109. https://doi.org/10.3141/1666-12

Ma, J., Chan, J., Rajasegarar, S. y Leckie, C. (2022). Multi-attention graph neural networks for city-wide bus travel time estimation using limited data. *Expert Systems with Applications*, *202*, Artículo 117057. https://doi.org/10.1016/j.eswa.2022.117057

Manibardo, E. L., Lana, I. y Del Ser, J. (2022). Deep learning for road traffic forecasting: Does it make a difference? *IEEE Transactions on Intelligent Transportation Systems*, *23*(7), 6164-6188. https://doi.org/10.1109/TITS.2021.3083957

Mazloumi, E., Currie, G. y Rose, G. (2010). Using GPS data to gain insight into public transport travel time variability. *Journal of Transportation Engineering*, *136*(7), 623-631. https://doi.org/10.1061/(ASCE)TE.1943-5436.0000126

Ministerio de Transportes y Movilidad Sostenible. (s.f.). *Punto de Acceso Nacional de transporte multimodal*. Recuperado el 31 de octubre de 2026, de https://nap.transportes.gob.es

Mishalani, R. G., McCord, M. M. y Wirtz, J. (2006). Passenger wait time perceptions at bus stops: Empirical results and impact on evaluating real-time bus arrival information. *Journal of Public Transportation*, *9*(2), 89-106. https://doi.org/10.5038/2375-0901.9.2.5

MobilityData. (s.f.-a). *GTFS realtime reference*. Recuperado el 31 de octubre de 2026, de https://gtfs.org/documentation/realtime/reference/

MobilityData. (s.f.-b). *GTFS schedule reference*. Recuperado el 31 de octubre de 2026, de https://gtfs.org/documentation/schedule/reference/

Pang, J., Huang, J., Du, Y., Yu, H., Huang, Q. y Yin, B. (2019). Learning to predict bus arrival time from heterogeneous measurements via recurrent neural network. *IEEE Transactions*

*on Intelligent Transportation Systems*, *20*(9), 3283-3293. https://doi.org/10.1109/TITS.2018.2873747

Petersen, N. C., Rodrigues, F. y Pereira, F. C. (2019). Multi-output bus travel time prediction with convolutional LSTM neural network. *Expert Systems with Applications*, *120*, 426-435. https://doi.org/10.1016/j.eswa.2018.11.028

Quddus, M. A., Ochieng, W. Y. y Noland, R. B. (2007). Current map-matching algorithms for transport applications: State-of-the art and future research directions. *Transportation Research Part C: Emerging Technologies*, *15*(5), 312-328. https://doi.org/10.1016/j.trc.2007.05.002

Shalaby, A. y Farhan, A. (2004). Prediction model of bus arrival and departure times using AVL and APC data. *Journal of Public Transportation*, *7*(1), 41-61. https://doi.org/10.5038/2375-0901.7.1.3

Singh, N. y Kumar, K. (2022). A review of bus arrival time prediction using artificial intelligence. *WIREs Data Mining and Knowledge Discovery*, *12*(4), Artículo e1457. https://doi.org/10.1002/widm.1457

Snyder, J. P. (1987). *Map projections: A working manual* (Professional Paper N.º 1395). U.S. Geological Survey. https://pubs.er.usgs.gov/publication/pp1395

Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley. https://search.worldcat.org/oclc/3058187

Vanajakshi, L., Subramanian, S. C. y Sivanandan, R. (2009). Travel time prediction under heterogeneous traffic conditions using global positioning system data from buses. *IET Intelligent Transport Systems*, *3*(1), 1-9. https://doi.org/10.1049/iet-its:20080013

Watkins, K. E., Ferris, B., Borning, A., Rutherford, G. S. y Layton, D. (2011). Where is my bus? Impact of mobile real-time information on the perceived and actual wait time of transit riders. *Transportation Research Part A: Policy and Practice*, *45*(8), 839-848. https://doi.org/10.1016/j.tra.2011.06.010

Wessel, N., Allen, J. y Farber, S. (2017). Constructing a routable retrospective transit timetable from a real-time vehicle location feed and GTFS. *Journal of Transport Geography*, *62*, 92-97. https://doi.org/10.1016/j.jtrangeo.2017.04.012

Woodard, D., Nogin, G., Koch, P., Racz, D., Goldszmidt, M. y Horvitz, E. (2017). Predicting travel time reliability using mobile phone GPS data. *Transportation Research Part C: Emerging Technologies*, *75*, 30-44. https://doi.org/10.1016/j.trc.2016.10.011

# Apéndice A

## Constantes

Todas las constantes del método, con su nombre en el código, su valor y la sección donde se explica. Las marcadas en la columna «Med.» se fijaron midiendo su efecto en Bizkaibus y conviene volver a medirlas en otra red (sección «Qué Hay que Adaptar»).

**Tabla A1**

*Constantes del Método*

| Nombre en el código | Valor | Significado | Sección | Med. |
|---|---|---|---|---|
| *Construcción de la base (trazados)* | | | | |
| `MAX_STOP_OFFSET_METERS` | 100 m | Parada mal medida si sus metros caen más lejos | «Trazados que No Casan con sus Paradas» | |
| `ROAD_VS_STRAIGHT_RATIO` | 3 | Búsqueda de la parada siguiente: veces la línea recta | «Trazados que No Casan con sus Paradas» | |
| `SEARCH_SLACK_METERS` | 500 m | Margen de esa búsqueda | «Trazados que No Casan con sus Paradas» | |
| `EARLIER_PASS_MARGIN_METERS` | 30 m | Ventaja exigida a un paso posterior por la misma zona | «Trazados que No Casan con sus Paradas» | |
| *Emparejamiento* | | | | |
| `iDepartureCluster GapSeconds` | 90 s | Separación máxima para fusionar variantes | «Fusión de Variantes y Alias de Viaje» | |
| `MATCH_TOLERANCE_SECONDS` | 180 s | Tolerancia del emparejamiento de respaldo | «Emparejamiento con el Horario» | |
| *Localización* | | | | |
| `METERS_PER_DEGREE` | 111 320 m | Metros por grado de latitud | «Proyección de un Punto Sobre un Segmento» | |

| Nombre en el código | Valor | Significado | Sección | Med. |
|---|---|---|---|---|
| `MAX_LOCATION_AGE_SECONDS` | 600 s | Antigüedad máxima del dato | «Condiciones Para Usar el GPS» | |
| `HOLD_SECONDS` | 180 s | Tiempo que se guarda un autobús que falta en el feed | «Autobuses que Faltan en una Lectura del Feed» | ✓ |
| `NOT_STARTED_GRACE_SECONDS` | 60 s | El viaje cuenta como empezado antes de su salida | «Condiciones Para Usar el GPS» | |
| `MAX_WAIT_AT_ORIGIN_SECONDS` | 3 600 s | Antelación máxima para situar al autobús en la cabecera | «En la Cabecera, Antes de Salir» | |
| `SEGMENTS_BEHIND_NEXT_STOP` | 3 | Primer tramo de la ventana: $n-3$ | «Ventana de Búsqueda» | |
| `SEGMENTS_AHEAD_OF_NEXT_STOP` | 2 | Último tramo de la ventana: $n+2$ (paradas hasta $p_{n+3}$) | «Ventana de Búsqueda» | |
| `MAX_DISTANCE_FROM_SHAPE_METERS` | 50 m | Pasillo con trazado | «Con Trazado» | ✓ |
| `BEYOND_NEXT_STOP_SLACK_METERS` | 100 m | Límite de los candidatos esperados | «Con Trazado» | |
| `AMBIGUITY_MARGIN_METERS` | 20 m | Ventaja exigida a un candidato «más allá» | «Con Trazado» | |
| `MAX_DISTANCE_FROM_ROUTE_METERS` | 150 m | Pasillo mínimo sin trazado; distancia máxima a la primera parada en la cabecera | «Sin Trazado» | ✓ |
| `MAX_DISTANCE_FROM_ROUTE_SEGMENT_RATIO` | 0,15 | Pasillo sin trazado, fracción del tramo | «Sin Trazado» | ✓ |
| *Modelo* | | | | |
| Franjas | 0, 6, 9, 13, 16 y 20 h | Inicio de cada periodo del día | 4 | |
| `LENGTH_CUTS_METERS` | 300, 600, 1 000, 1 500, 3 000 m | Cortes de los cubos de longitud | «Respaldo por Longitud del Tramo» | |

| Nombre en el código | Valor | Significado | Sección | Med. |
|---|---|---|---|---|
| Cotas de la proporción | 0,3 – 1,5 | Acotación del respaldo por longitud | «Respaldo por Longitud del Tramo» | |
| `MIN_FACTOR`, `MAX_FACTOR` | 0,6 – 1,2 | Acotación de $k$ | «Factor de Ritmo» | ✓ |
| *Predicción* | | | | |
| `REGISTRATION_SEGMENT_SHARE` | 0,5 | Fracción del tramo que falta en la capa 2 | «Capa 2: Llegada Desde la Parada Registrada» | ✓ |
| `REGISTRATION_MAX_SEGMENT_SECONDS` | 1 200 s | Tramo máximo para la capa 2 | «Capa 2: Llegada Desde la Parada Registrada» | ✓ |
| `REGISTRATION_CROSSCHECK_MAX_SEGMENT_SECONDS` | 360 s | Tramo máximo para el contraste | «Contraste Entre el GPS y la Parada Registrada» | ✓ |
| `MAX_GPS_VS_STOP_DISAGREEMENT_SECONDS` | 360 s | Discrepancia máxima GPS / parada | «Contraste Entre el GPS y la Parada Registrada» | ✓ |
| `SCHEDULE_ORIGIN_OFFSET_SECONDS` | 60 s | Salida de cabecera tras la hora publicada | «Capa 3: Horario Corregido» | ✓ |
| `WRONG_SCHEDULE_PASSED_SECONDS` | 600 s | «Probablemente pasado»: hace cuánto | «Capa 3: Horario Corregido» | |
| `WRONG_SCHEDULE_GAP_SECONDS` | 600 s | «Probablemente pasado»: horario lento | «Capa 3: Horario Corregido» | |
| `WRONG_SCHEDULE_LEARNED_SHARE` | 0,8 | «Probablemente pasado»: parte aprendida | «Capa 3: Horario Corregido» | |
| `TripOffsets::PARTS` | 3 | Partes del recorrido en la corrección por viaje | «Corrección por Viaje» | ✓ |
| Calendario de solo domingo | máscara 64 | Marca un festivo entre semana | «Corrección por Viaje» | |
| `MAX_PLAUSIBLE_DELAY_SECONDS` | 2 700 s | Retraso máximo que se afirma | «La Decisión Completa» | ✓ |
| `STALE_GRACE_SECONDS` | 120 s | Llegada de la capa 2 en el pasado que aún se muestra | «La Decisión Completa» | |

| Nombre en el código | Valor | Significado | Sección | Med. |
|---|---|---|---|---|
| `PASSED_DISTANCE_METERS` | 150 m | Distancia del GPS a la parada para darla por pasada sin que lo diga el feed | «Paradas Ya Pasadas» | |
| `ARRIVING_MAX_SECONDS` | 180 s | Tiempo en «0 min» sin confirmar el paso antes de «puede que ya haya pasado» | «Llegando sin Confirmar el Paso» | |
| `TTL_SECONDS` (registro de pasos) | 36 h | Caducidad de los pasos, desde el final de su día | «Registro de Pasos» | |
| `DAY_SECONDS/2` | 43 200 s | Salida del feed por delante de ahora a partir de la que la entrada es de un viaje de ayer | «Viajes que Pasan de las 24:00» | |
| `LATE_GRACE_SECONDS` | 600 s | De madrugada, la ficha es la del viaje de ayer hasta este tiempo después de su última parada | «Viajes que Pasan de las 24:00» | |
| Retirada de la lista | −2 min | Minutos a partir de los que se oculta una fila | «Retraso y Minutos que Se Muestran» | |
| Umbral de «adelantado» | −2 min | Adelanto mínimo que se anuncia | «Retraso y Minutos que Se Muestran» | |
| `POSITION_SANITY_SECONDS` | 900 s | Redes con retraso declarado | «Redes que Publican Retraso o Predicción» | |
| *Aprendizaje* | | | | |
| Captura | continua, bloques de 1 h, cada 20 s | Bloque e intervalo de captura | «Captura del Feed» | |
| `MAX_GAP_BETWEEN_SNAPSHOTS_SECONDS` | 300 s | Hueco que corta una ventana; par máximo del paso real | «El Instante Real de Paso por una Parada» | |
| `OBSERVED_AFTER_PREDICTION_SECONDS` | 600 s | Margen de observación tras la llegada | «Muestras de Ritmo» | |

| Nombre en el código | Valor | Significado | Sección | Med. |
|---|---|---|---|---|
| `MIN_REMAINING_SECONDS_FOR_FACTOR` | 120 s | Horizonte mínimo de una muestra de ritmo | «Muestras de Ritmo» | |
| `MAX_SAMPLES_PER_TRACK` | 30 | Muestras de ritmo por pista | «Muestras de Ritmo» | |
| `MIN_TRACKS_PER_SEGMENT` | 2 | Pistas mínimas por tramo y franja | «Mediana y Contracción Hacia el Tramo a Cualquier Hora» | ✓ |
| `SEGMENT_BAND_PRIOR_TRACKS` | 5 | Peso de la mediana del tramo, $n_0$ | «Mediana y Contracción Hacia el Tramo a Cualquier Hora» | ✓ |
| `MIN_SAMPLES_PER_LENGTH_RATIO` | 20 | Muestras mínimas por cubo y franja | «Respaldo por Longitud del Tramo» | |
| `MIN_TRACKS_PER_BAND` | 8 | Pistas mínimas para $k_b$ | «Factor de Ritmo» | |
| `MIN_TRACKS_PER_LINE` | 25 | Pistas mínimas para $k_{\ell,b}$ | «Factor de Ritmo» | ✓ |
| `MIN_VEHICLES_PER_LINE` | 5 | Vehículos mínimos para $k_{\ell,b}$ | «Factor de Ritmo» | |
| `SHRINKAGE_TRACKS` | 10 | Peso de $k = 1$ en la contracción | «Factor de Ritmo» | |
| `SAMPLES_RETENTION_DAYS` | 120 d | Retención de muestras | «Acumulación y Retención» | |
| `MAX_SAMPLES_PER_SEGMENT` | 300 | Muestras por tramo y tipo de día | «Acumulación y Retención» | |
| `MAX_PACE_SAMPLES` | 500 000 | Muestras de ritmo en total | «Acumulación y Retención» | |
| Pliegues | 2 | crc32(`track`) mód 2 | «Recalibración Diaria» | |

| Nombre en el código | Valor | Significado | Sección | Med. |
|---|---|---|---|---|
| `TRIP_OFFSET_RANKS` | 0,3 a 0,5 | Percentiles de los desvíos entre los que la recalibración elige cada noche: el que más acierta a $\pm 2$ min y, con empate, el de menos error medio (0,4 en los vectores) | «La Media o un Percentil» | ✓ |
| Pasos fuera de la corrección por viaje | 2 700 s | Pasos más lejos de su hora publicada que no se usan | «Recalibración Diaria» | |
| `TRIP_OFFSET_CLIP_SECONDS` | 900 s | Recorte de cada desvío de la corrección por viaje | «Recalibración Diaria» | ✓ |
| `TRIP_OFFSET_LINE_PRIOR_PASSES` | 20 | Peso de la franja en la corrección de la línea | «Recalibración Diaria» | |
| `TRIP_OFFSET_TRIP_PRIOR_DAYS` | 1 | Peso de la línea en la corrección del viaje, en días | «Recalibración Diaria» | ✓ |
| `TRIP_OFFSET_TEST_DAYS` | 7 d | Días de prueba de la corrección por viaje | «Recalibración Diaria» | |
| *Evaluación* | | | | |
| Paradas evaluadas | $n$ a $n+8$ | Paradas de cada autobús con las que se mide | «La Verdad de Referencia» | |
| Horizontes | 0–20 min | Tiempo hasta el paso real que se evalúa | «La Verdad de Referencia» | |
| Histogramas | 15 s | Anchura de las clases del error en la captura continua | «Evaluación a lo Largo de Varios Días» | |
| Datos del día para los rivales | 60 s | Antigüedad mínima de los tramos del día que pueden usar | «Comparación con Otros Métodos» | |
| *Trip Updates (Metro, Euskotren, tranvías, Renfe, Tuvisa, Dbus, Álavabus)* | | | | |
| `match_tolerance_seconds` | 600 s | Tolerancia del emparejamiento de reserva, por paradas y hora | «Redes que Publican Retraso o Predicción» | |

| Nombre en el código | Valor | Significado | Sección | Med. |
|---|---|---|---|---|
| VEHICLE_DISAGREEMENT_SECONDS | 1 200 s | Hora del feed que contradice al GPS | «Redes que Publican Retraso o Predicción» | |
| MIN_VEHICLES_TO_DISTRUST | 3 | Vehículos con posición para desconfiar del feed | «Redes que Publican Retraso o Predicción» | |
| MAX_GPS_DELAY_SECONDS | 2 700 s | Retraso máximo creíble por GPS | «Redes que Publican Retraso o Predicción» | |
| STALE_PREDICTION_SECONDS | 60 s | Predicción vencida | «Redes que Publican Retraso o Predicción» | |
| MAX_STALE_PREDICTION_SECONDS | 3 600 s | Desfase máximo que se extrapola | «Redes que Publican Retraso o Predicción» | |
| DUE_SECONDS | 60 s | Salida que ya debería estar en la estación | «Redes que Publican Retraso o Predicción» | |
| LATE_TRAINS_LOOKBACK_SECONDS | 3 600 s | Salidas pasadas que se miran por si van tarde | «Redes que Publican Retraso o Predicción» | |
| ON_TIME_SECONDS | 60 s | Retraso que se redondea a cero | «Redes que Publican Retraso o Predicción» | |
| LIVE_DEPARTED_AFTER_SECONDS | 45 s | Retirada de una fila en vivo | «Redes que Publican Retraso o Predicción» | |
| SCHEDULED_DEPARTED_AFTER_SECONDS | 60 s | Retirada de una fila programada | «Redes que Publican Retraso o Predicción» | |

## Apéndice B

## Pseudocódigo

Este apéndice reúne el pseudocódigo de todas las funciones del método, en el orden en que se usan. Los índices de paradas empiezan en 1, como en el texto: el viaje tiene paradas $p_1, \ldots, p_N$ y el tramo $i$ va de $p_i$ a $p_{i+1}$. rnd es el redondeo con las mitades alejándose del cero, med la mediana baja y clamp la acotación (sección «Convenciones»). «Nulo» significa que el valor no existe.

**Figura B1**

*Proyección de un Punto Sobre un Segmento*

**función** PROYECTAR($\varphi, \lambda, A, B$)
    $m_\lambda \leftarrow \cos(\varphi \cdot \pi/180) \cdot 111\,320$;  $m_\varphi \leftarrow 111\,320$
    $x_A \leftarrow (\lambda_A - \lambda)\, m_\lambda$; $y_A \leftarrow (\varphi_A - \varphi)\, m_\varphi$; $x_B \leftarrow (\lambda_B - \lambda)\, m_\lambda$; $y_B \leftarrow (\varphi_B - \varphi)\, m_\varphi$
    $\Delta x \leftarrow x_B - x_A$; $\Delta y \leftarrow y_B - y_A$; $\ell^2 \leftarrow \Delta x^2 + \Delta y^2$
    $f \leftarrow 0$
    **si** $\ell^2 > 0$ **entonces** $f \leftarrow \text{clamp}\big(-(x_A \Delta x + y_A \Delta y)/\ell^2,\ 0,\ 1\big)$
    **devolver** $\big(f,\ \sqrt{(x_A + f\Delta x)^2 + (y_A + f\Delta y)^2},\ \sqrt{\ell^2}\big)$  ▷ fracción, distancia, longitud

*Nota.* En un plano local centrado en el punto.

**Figura B2**

*Posición del Autobús en el Horario (Capa 1, sin el Contraste)*

**función** POSICIÓN($J$, entrada $e$, $t_{\text{now}}$)
  **si** $e$ no tiene $\varphi$, $\lambda$, $t_{\text{gps}}$ o $n$ **entonces devolver** nulo
  **si** $t_{\text{now}} - t_{\text{gps}} > 600$ **o** $N < 2$ **entonces devolver** nulo
  **si** $t_{\text{gps}} < d_1 - 60$ **entonces devolver** la posición en la cabecera (9) si $d_1 - t_{\text{gps}} \le 3\,600$ y el GPS está a 150 m o menos de $p_1$; si no, nulo
  $v \leftarrow$ el índice con orden de parada igual a $n$; **si** no existe, **devolver** nulo
  $i_0 \leftarrow \text{máx}(1, v-3)$;  $i_1 \leftarrow \text{mín}(N-1, v+2)$
  **si** $J$ tiene trazado $\Sigma$ **y** $s_i$ existe para $i = i_0, \ldots, i_1+1$ **entonces**
    $\sigma_0 \leftarrow s_{i_0}$; $\sigma_1 \leftarrow s_{i_1+1}$; $\sigma_{\text{lím}} \leftarrow s_v + 100$; $E \leftarrow$ nulo; $M \leftarrow$ nulo
    **para** $j = 1, \ldots, M_\Sigma - 1$ **hacer** ▷ segmentos del trazado, en orden
      **si** $\sigma_{j+1} < \sigma_0$ **o** $\sigma_j > \sigma_1$ **entonces continuar**
      $(f, d, \cdot) \leftarrow$ PROYECTAR($\varphi, \lambda, \Sigma_j, \Sigma_{j+1}$);  $s^* \leftarrow \sigma_j + f(\sigma_{j+1} - \sigma_j)$
      **si** $s^* \le \sigma_{\text{lím}}$ **entonces**
        **si** $E$ es nulo **o** $d < E.d$ **entonces** $E \leftarrow (d, s^*)$
      **si no si** $M$ es nulo **o** $d < M.d$ **entonces** $M \leftarrow (d, s^*)$
    $B \leftarrow E$
    **si** $M$ no es nulo **y** ($B$ es nulo **o** $M.d + 20 < B.d$) **entonces** $B \leftarrow M$
    **si** $B$ es nulo **o** $B.d > 50$ **entonces devolver** nulo
    $s^* \leftarrow \text{clamp}(B.s^*, \sigma_0, \sigma_1)$
    **para** $i = i_0, \ldots, i_1$ **hacer**
      **si** $s^* > s_{i+1}$ **y** $i < i_1$ **entonces continuar**
      $f \leftarrow 0$; **si** $s_{i+1} > s_i$ **entonces** $f \leftarrow \text{clamp}\big((s^* - s_i)/(s_{i+1} - s_i), 0, 1\big)$
      **devolver** $\big(T_{\text{pos}} = \text{rnd}(d_i + f(a_{i+1} - d_i)),\ t_{\text{gps}},\ B.d\big)$
  **si no** ▷ sin trazado: rectas entre paradas
    $B \leftarrow$ nulo
    **para** $i = i_0, \ldots, i_1$ **hacer**
      $(f, d, L) \leftarrow$ PROYECTAR($\varphi, \lambda, p_i, p_{i+1}$)
      **si** $B$ es nulo **o** $d < B.d$ **entonces** $B \leftarrow \big(d,\ \text{máx}(150, 0{,}15L),\ d_i + f(a_{i+1} - d_i)\big)$
    **si** $B.d > B.c$ **entonces devolver** nulo
    **devolver** $\big(T_{\text{pos}} = \text{rnd}(B.t),\ t_{\text{gps}},\ B.d\big)$

**Figura B3**

*Tiempo de un Tramo y Tiempo Esperado Entre Dos Horas del Horario*

**función** TIEMPOTRAMO$(i, b, k)$
    $h \leftarrow a_{i+1} - a_i$
    $\tau \leftarrow$ valor de $\kappa_i = u_i > u_{i+1}$ en la franja $b$; si no existe, el valor `all`; si no, nulo
    **si** $\tau$ no es nulo **entonces devolver** $\tau$
    **si** $s_i$ y $s_{i+1}$ existen **y** $L = s_{i+1} - s_i > 0$ **y** hay proporciones **entonces**
        $r \leftarrow$ proporción del cubo $c(L)$ en la franja $b$; si no existe, la `all`; si no, nulo
        **si** $r$ no es nulo **entonces devolver** $\text{clamp}(r, 0{,}3, 1{,}5) \cdot h$
    **devolver** $k \cdot h$

**función** ESPERADO$(J, t_0, t_1, b, k)$
    **si** la tabla de tramos está vacía **entonces devolver** $k\,(t_1 - t_0)$
    $S \leftarrow 0$
    **para** $i = 1, \ldots, N-1$ **hacer**
        **si** $a_{i+1} \le t_0$ **entonces continuar**
        **si** $a_i \ge t_1$ **entonces terminar**
        $h \leftarrow a_{i+1} - a_i; \quad w \leftarrow 1$
        **si** $h > 0$ **entonces** $w \leftarrow \big(\text{mín}(a_{i+1}, t_1) - \text{máx}(a_i, t_0)\big)/h$
        $S \leftarrow S + \text{máx}(0, w) \cdot$ TIEMPOTRAMO$(i, b, k)$
    **devolver** $S$

**Figura B4**

*Registro (Capa 2), Llegada Desde el Registro y Hora de Paso*

**función** REGISTRO($J, e, t_{\text{now}}$)
  **si** falta $n$ **o** el par $(J, v)$ no es fiable **o** $n < 2$ **entonces devolver** nulo
  $t_{\text{ref}} \leftarrow t_{\text{gps}}$ si existe; si no, $t_{\text{rec}}$
  **si** $t_{\text{ref}}$ no existe **o** $t_{\text{now}} - t_{\text{ref}} > 600$ **o** $t_{\text{ref}} < d_1 - 60$ **entonces devolver** nulo
  **si** no hay paradas de orden $n-1$ y $n$ **o** $a_n - a_{n-1} > 1200$ **entonces devolver** nulo
  $b \leftarrow \text{franja}(D, t_{\text{ref}})$; $k \leftarrow k(\ell, D, t_{\text{ref}})$
  $\theta \leftarrow \text{rnd}\left(0{,}5 \cdot \text{ESPERADO}(J, a_{n-1}, a_n, b, k)\right)$
  **devolver** $\left(n,\ t_{\text{ref}},\ a_{n-1},\ a_n,\ \theta,\ \delta_{\text{reg}} = t_{\text{ref}} + \theta - a_n,\ g = a_n - a_{n-1}\right)$

**función** LLEGADAREGISTRO($J, e, q, a_{\text{dest}}$)
  $G \leftarrow$REGISTRO($J, e, t_{\text{now}}$); **si** $G$ es nulo **o** $q < G.n$, **devolver** nulo
  $X \leftarrow 0$
  **si** $a_{\text{dest}} > G.a_n$ **entonces**
    $b \leftarrow \text{franja}(D, G.t_{\text{ref}})$; $k \leftarrow k(\ell, D, G.t_{\text{ref}})$
    $X \leftarrow$ESPERADO($J, G.a_n, a_{\text{dest}}, b, k$)
  **devolver** $\text{rnd}(G.t_{\text{ref}} + G.\theta + X)$

**función** HORADEPASO($J, e, q, a_{\text{dest}}$)
  $P \leftarrow$ posición con contraste
  **si** $P$ no es nulo **entonces**
    **si** $P$ es de cabecera **entonces devolver** nulo
    **si** $a_{\text{dest}} < P.T_{\text{pos}}$ **y** $(q < n$ **o** $\text{dist}(e, p_q) > 150)$ **entonces**
      **devolver** $P.t_{\text{gps}} - (P.T_{\text{pos}} - a_{\text{dest}})$
    **si** $q < n$ **entonces devolver** $P.t_{\text{gps}}$
    **devolver** nulo
  **si** $q < n$ **y** $G \leftarrow$REGISTRO($J, e, t_{\text{now}}$) no es nulo **entonces devolver** $G.t_{\text{ref}} - \text{máx}(0, G.a_{n-1} - a_{\text{dest}})$
  **devolver** nulo

**Figura B5**

*Posición con Contraste (Se Evalúa Antes que Cualquier Uso del Registro)*

1: **función** POSICIÓNCONCONTRASTE($J, e, t_{\text{now}}$)
2: $P \leftarrow$POSICIÓN($J, e, t_{\text{now}}$)
3: **si** $P$ no es nulo **entonces**
4: $G \leftarrow$REGISTRO($J, e, t_{\text{now}}$)
5: **si** $G$ no es nulo **y** $G.g \leq 360$ **y** $|(P.t_{\text{gps}} - P.T_{\text{pos}}) - G.\delta_{\text{reg}}| > 360$ **entonces**
6: marcar el par $(J, v)$ como no fiable durante esta petición; $P \leftarrow$ nulo
7: **devolver** $P$

**Figura B6**

*Capa 3: Horario Corregido, Corrección por Viaje y Parte Aprendida*

**función** SINDATOS($J, q, a_{\text{dest}}, t_{\text{now}}$)
  **si** $J$ no tiene entrada en el feed **y** el registro de pasos de $J$ tiene $(n,t)$ con $q < n$ **entonces devolver** $(\texttt{departed}, \min(t, t_{\text{now}}), 0)$
  $R \leftarrow (\texttt{scheduled}, a_{\text{dest}}, 0)$
  **si** la tabla de tramos está vacía **o** $J$ no tiene paradas **o** $a_{\text{dest}} \le d_1$ **entonces devolver** $R$
  $D' \leftarrow D$, o el domingo de esa semana si $D$ es festivo entre semana
  $t_m \leftarrow \lfloor (d_1 + a_{\text{dest}})/2 \rfloor$; $b \leftarrow$ franja$(D', t_m)$; $k \leftarrow k(\ell, D', t_m)$
  $C \leftarrow d_1 + 60 + \text{rnd}\left(\text{ESPERADO}(J, d_1 - 1, a_{\text{dest}}, b, k)\right) + \text{rnd}\left(\text{CORRECCIÓN}(J, q, D')\right)$
  **si** $C \ge t_{\text{now}}$ **entonces devolver** $(\texttt{estimated}, C, 0)$
  **si** $C < t_{\text{now}} - 600$ **y** $a_{\text{dest}} - C \ge 600$ **y** PARTEAPRENDIDA$(J, d_1 - 1, a_{\text{dest}}, b) \ge 0{,}8$ **entonces**
    **devolver** $(\texttt{likely_departed}, a_{\text{dest}}, 0)$
  **devolver** $R$

**función** CORRECCIÓN($J, q, D'$)
  **si** la tabla de correcciones está vacía **o** ninguna parada de $J$ tiene orden $q$ **entonces devolver** $0$
  $i \leftarrow$ posición de la parada de orden $q$ (0 la cabecera); $p \leftarrow \min(2, \lfloor 3i/N \rfloor)$
  $v \leftarrow (\ell, p_1, p_N, d_1, N)$; $\theta \leftarrow$ tipo de día de $D'$; $b_1 \leftarrow$ franja$(D', d_1)$
  **si** la tabla tiene $\Delta(v, \theta, \cdot)$ **entonces devolver** $\Delta(v, \theta, p)$
  **si** la tabla tiene $l(\ell, b_1)$ **entonces devolver** $l(\ell, b_1)$
  **si** la tabla tiene $g(b_1)$ **entonces devolver** $g(b_1)$
  **devolver** $0$

**función** PARTEAPRENDIDA($J, t_0, t_1, b$)
  $T \leftarrow 0$; $A \leftarrow 0$
  **para** $i = 1, \ldots, N-1$ **hacer**
    **si** $a_{i+1} \le t_0$ **entonces continuar**
    **si** $a_i \ge t_1$ **entonces terminar**
    $T \leftarrow T + (a_{i+1} - a_i)$
    **si** $\kappa_i$ tiene valor en $b$ o `all` **entonces** $A \leftarrow A + (a_{i+1} - a_i)$
  **devolver** $0$ si $T \le 0$; si no, $A/T$

**Figura B7**

*Paso Real por una Parada y Aclarado de Muestras*

**función** PASOREAL(pista $((t_k,\theta_k))_{k=1}^{K}$, $a$)
    **para** $k = 2,\dots,K$ **hacer**
        **si** $\theta_k \le \theta_{k-1}$ **o** $t_k - t_{k-1} > 300$ **entonces continuar**
        **si** $\theta_{k-1} \le a \le \theta_k$ **entonces**
            **devolver** $t_{k-1} + (a-\theta_{k-1})/(\theta_k-\theta_{k-1})\cdot(t_k-t_{k-1})$
    **devolver** nulo

**función** ACLARAR(lista $x_0,\dots,x_{c-1}$)
    **si** $c \le 30$ **entonces devolver** la lista
    **devolver** $\left(x_{\lfloor ic/30 \rfloor}\right)_{i=0}^{29}$ ▷ división entera

**Figura B8**

*Ajuste de los Tiempos por Tramo*

**función** AJUSTARTRAMOS(muestras de tramo, en orden)
    **para** cada muestra $(\kappa, j, b, y)$ **hacer**
        $V[\kappa][\texttt{all}][j] \leftarrow y$;  $V[\kappa][b][j] \leftarrow y$ ▷ la última de cada pista gana
    $T \leftarrow$ diccionario vacío
    **para** cada tramo $\kappa$ de $V$, con sus grupos en orden de aparición (`all` primero) **hacer**
        **si** $|V[\kappa][\texttt{all}]| < 2$ **entonces continuar**
        $t \leftarrow \{\texttt{n} : |V[\kappa][\texttt{all}]|\}$;  $m \leftarrow \text{med}(V[\kappa][\texttt{all}])$
        *varias* $\leftarrow$ (hay al menos dos franjas en $V[\kappa]$)
        **para** cada grupo $g$ de $V[\kappa]$ **hacer**
            **si** $g = \texttt{all}$ **y no** *varias* **entonces continuar**
            $n_g \leftarrow |V[\kappa][g]|$; **si** $n_g < 2$, **continuar**
            $x \leftarrow \text{med}(V[\kappa][g])$
            **si** $g \neq \texttt{all}$ **entonces** $x \leftarrow (n_g x + 5m)/(n_g+5)$
            $t[g] \leftarrow \text{rnd}(x)$
        **si** $t$ tiene algún grupo además de `n` **entonces** $T[\kappa] \leftarrow t$
    **devolver** $T$ ordenado por clave

**función** AJUSTARPROPORCIONES(muestras de tramo, longitudes $L[\kappa]$)
    **para** cada muestra con $L[\kappa] > 0$ y $h > 0$ **hacer**
        $c \leftarrow c(L[\kappa])$; añadir $y/h$ a $Q[c][\texttt{all}]$ y a $Q[c][b]$
    **devolver** $\{c : \{g : \text{rnd}_3(\text{med}\, Q[c][g])$ para cada $g$ con $|Q[c][g]| \ge 20\}\}$

**Figura B9**

*Ajuste del Factor de Ritmo*

**función** FACTORCONTRAÍDO(pistas $\mathscr{J}$, cada una con sus muestras $(H, Y)$)
    **para** cada pista $j$ **hacer** $\rho_j \leftarrow \sum Y / \sum H$
    $m \leftarrow |\mathscr{J}|$;  $\tilde{\rho} \leftarrow \operatorname{med}_j \rho_j$
    **devolver** $\operatorname{rnd}_3\big(\operatorname{clamp}(1 + (\tilde{\rho} - 1)\, m/(m+10),\ 0{,}6,\ 1{,}2)\big)$

**función** AJUSTARFACTORES(muestras de ritmo)
    **para** cada franja $b$ con $\geq 8$ pistas **hacer** $k_b \leftarrow$FACTORCONTRAÍDO(pistas de $b$)
    **para** cada línea $\ell \neq$ vacía y franja $b$ con $\geq 25$ pistas y $\geq 5$ vehículos **hacer**
        $k_{\ell,b} \leftarrow$FACTORCONTRAÍDO(pistas de $\ell$ en $b$)
    **devolver** $\{\texttt{global} : k_b,\ \texttt{lines} : k_{\ell,b}\}$

**Figura B10**

*Validación Cruzada y Escritura de los Tiempos por Tramo*

**función** RECALIBRARTRAMOS(muestras $X$, factores vigentes)
    $S_a \leftarrow 0$; $S_d \leftarrow 0$; $c \leftarrow 0$
    **para** $f \in \{0, 1\}$ **hacer**
        $T_f \leftarrow$AJUSTARTRAMOS($\{x \in X : \operatorname{crc32}(x.\texttt{track}) \bmod 2 \neq f\}$)
        **para** cada $x \in X$ con $\operatorname{crc32}(x.\texttt{track}) \bmod 2 = f$ y $\tau \leftarrow T_f(x.\kappa, x.b)$ definido **hacer**
            $k \leftarrow k(x.\ell, x.D, x.\texttt{at})$; $S_a \leftarrow S_a + |k x.h - x.y|/60$; $S_d \leftarrow S_d + |\tau - x.y|/60$; $c \leftarrow c + 1$
    validar igual las proporciones; incluirlas solo si su suma de errores baja
    **si** $c = 0$ **o** $S_d \geq S_a$ **entonces devolver** sin escribir
    $T \leftarrow$AJUSTARTRAMOS($X$)
    **si** $T$ y las proporciones son iguales a las del fichero actual **entonces devolver** sin escribir
    escribir $\{\texttt{generated},\ \texttt{samples} : |X|,\ \texttt{segments} : T,\ \texttt{lengthRatios}\}$

# Apéndice C

# Formatos de Datos

## Base de Datos del Horario

Una base SQLite por red, de solo lectura, reconstruida cada día desde el GTFS. Las tablas y columnas que usa el motor son estas (los tipos son los de SQLite):

```
CREATE TABLE stops (id TEXT PRIMARY KEY, name TEXT NOT NULL, lat REAL NOT
   NULL, lon REAL NOT NULL, ...);
CREATE TABLE lines (id TEXT PRIMARY KEY, code TEXT NOT NULL, name TEXT NOT
   NULL, ...);
CREATE TABLE journey_patterns (id TEXT PRIMARY KEY, line_id TEXT NOT NULL,
   headsign TEXT);
CREATE TABLE journey_pattern_stops (journey_pattern_id TEXT NOT NULL,
   seq_order INTEGER NOT NULL,
                                    stop_id TEXT NOT NULL);
CREATE TABLE service_journeys (id TEXT PRIMARY KEY, line_id TEXT NOT NULL,
                               journey_pattern_id TEXT NOT NULL,
   trip_number TEXT,
                               calendar_id TEXT NOT NULL,
   first_departure_seconds INTEGER,
                               shape_id TEXT);
CREATE TABLE passing_times (service_journey_id TEXT NOT NULL, seq_order
   INTEGER NOT NULL,
                            stop_id TEXT NOT NULL, arrival_seconds INTEGER,
                            departure_seconds INTEGER, dist_m REAL);
CREATE TABLE shapes (id TEXT PRIMARY KEY, points TEXT NOT NULL);   -- JSON
   [[lat, lon, dist], ...]
CREATE TABLE trip_aliases (trip_id TEXT PRIMARY KEY, service_journey_id
   TEXT NOT NULL);
```

Para Bizkaibus, en octubre de 2026: 100 líneas, 2 346 paradas, 483 recorridos, 6 097 viajes fusionados (23 332 `trip_id` originales en `trip_aliases`), 138 513 horas de paso y 440 trazados.

Una fila de `passing_times`:

```
('trp_A0651_737_OP43FIN_30900_O43FI0651_065151_3', 2, '1529', 32430, 32432,
   13223.0)
```

La consulta que usa el motor para un viaje devuelve sus paradas en orden de `seq_order`, con llegada, salida, `dist_m` y las coordenadas de la parada; la de un trazado devuelve la lista de

puntos.

**Tiempos por Tramo:** `segment-times.json`

```
{
  "generated": "2026-10-31 11:31",
  "samples": 195786,
  "segments": {
    "1003>706": { "n": 43, "all": 118, "laborable.mediodia": 118, "laborable
   .punta_tarde": 126, ... },
    ...
  },
  "lengthRatios": [
    { "all": 0.987, "laborable.punta_manana": 1.016, ... },
    { "all": 0.907, ... },
    ...
  ]
}
```

- `segments`: un objeto por tramo, con clave «`desde>hasta`» (identificadores de parada) y valores enteros en segundos por franja y `all`, más `n`, el número de pistas (informativo).
- `lengthRatios`: proporción por cubo de longitud (0 a 5) y franja. **Atención**: el serializador JSON de PHP escribe un objeto con claves enteras consecutivas desde 0 como una *lista*. Si están todos los cubos, `lengthRatios` es una lista de seis objetos (el índice es el cubo); si falta alguno, es un objeto con claves `"0"`, `"1"`, … Un lector debe aceptar las dos formas.
- Una proporción con valor exacto, como 1, puede aparecer como entero.

**Factores de Ritmo:** `pace-factors.json`

```
{
  "generated": "2026-10-31 11:10",
  "captures": 0,
  "samples": 356001,
  "global": { "laborable.manana": 0.885, "laborable.punta_tarde": 0.928, ...
    },
  "lines":  { "3411": { "laborable.manana": 1.03, ... }, ... },
  "meta":   { "global": { "laborable.manana": 2260, ... }, "lines": { ... }
   }
}
```

`global` y `lines` contienen los factores ya acotados y redondeados a tres decimales; `meta` guarda el número de pistas con que se calculó cada uno. Las claves de línea son el código numérico de la línea como texto. `captures` vale 0 cuando se calibra desde el fichero acumulado de muestras, que es lo habitual, y el número de capturas cuando se calibra directamente desde una carpeta de capturas.

**Corrección por Viaje:** `trip-offsets.json`

```
{
  "generated": "2026-11-07 11:53",
  "samples": 531799,
  "rank": 0.35,
  "trips": {
    "2151|1|3382|26102|18": { "laborable": [-39, -87, -32] },
    ...
  },
  "lines":  { "651": { "laborable.manana": -6, "fin_semana.punta_manana": -
   36, ... }, ... },
  "global": { "laborable.manana": 8, "fin_semana.noche": -44, ... }
}
```

- `trips`: un objeto por viaje con la clave «línea|primera parada|última parada|salida|número de paradas», con la salida en segundos desde medianoche. Por tipo de día (`laborable`, `fin_semana`) guarda tres enteros en segundos: la corrección del primer, del segundo y del tercer tercio del recorrido.
- `lines` y `global`: la corrección de cada línea en cada franja y la de cada franja, en segundos enteros. La franja es la de la salida de cabecera, y las claves de línea son su código como texto.
- `samples` es el número de pasos con que se calculó y `rank`, el percentil de los desvíos que eligió la recalibración esa noche (sección «La Media o un Percentil»). Un festivo entre semana se consulta como fin de semana.

**Muestras Acumuladas (JSON Lines)**

Un objeto JSON por línea. Una muestra de tramo:

```
{"track":"2026-10-06|1112|trp_A3411_805_OP9FIN_45000_O9FI3411_341101_7","
   line":"3411",
 "date":"2026-10-06","at":47066,"band":"laborable.mediodia","from":"1479","
   to":"1480",
 "scheduled":101,"actual":115.2}
```

Una muestra de ritmo:

```
{"track":"2026-10-02|1101|trp_A3411_804_OP9FIN_43200_O9FI3411_341104_6","
   vehicle":"1101",
 "line":"3411","date":"2026-10-02","at":45895,"band":"laborable.manana",
 "scheduledRemaining":217,"actualRemaining":409.8,"actual":46304.8}
```

Al leer, se ignoran las líneas que no son JSON válido o a las que les falta algún campo obligatorio: `track`, `at`, `band`, `from`, `to`, `scheduled` y `actual` en las de tramo; `track`, `band`, `scheduledRemaining` y `actualRemaining` en las de ritmo. La recalibración usa además `date` (retención y tope por tipo de día), `line` (factores por línea), `vehicle` (mínimo de vehículos por línea) y, en las de ritmo, `at` y `actual` (validación); las muestras que escribe la captura los traen siempre.

**Registro de Pasos**

Un almacén clave-valor con dos tablas hash por red y día:

| Clave | Campo | Valor |
|---|---|---|
| `bide:<red>:pass:<fecha>` | referencia $\rho$ | `"<orden>:<segundos>"`; caduca 36 h después de ese día |
| `bide:<red>:stops:<fecha>` | `"<ρ>#<orden>"` | `"<segundos>"`; caduca 3 días después de ese día |

La primera guarda el último orden visto de cada viaje y es la que usa la predicción (sección «Registro de Pasos»); la segunda guarda la hora en que se vio cada orden y sirve para el historial de una línea. Se escriben cada minuto. La `<fecha>` es la del día del viaje: pasada la medianoche, un autobús de un viaje de ayer se apunta en las tablas de ayer, con los segundos en las horas de su horario (más de 86 400; sección «Viajes que Pasan de las 24:00»).

**El Feed SIRI-VM**

Fragmento mínimo de una actividad con los elementos que se leen (Tabla 4):

```
<VehicleActivity>
  <RecordedAtTime>2026-10-28T15:12:30+01:00</RecordedAtTime>
  <MonitoredVehicleJourney>
    <VehicleJourneyRef>trp_A3414_808_OP9LSEPT_54000_...</VehicleJourneyRef>
    <VehicleRef>1112</VehicleRef>
    <VehicleLocation><Longitude>-2.9214</Longitude><Latitude>43.2601</
   Latitude></VehicleLocation>
    <LocationRecordedAtTime>2026-10-28T15:12:30+01:00</
   LocationRecordedAtTime>
    <MonitoredCall><StopPointRef>1480</StopPointRef><Order>3</Order></
   MonitoredCall>
  </MonitoredVehicleJourney>
</VehicleActivity>
```

Los retrasos (`Delay`) siguen el patrón `^(-?)PT(?:(\d+)H)?(?:(\d+)M)?(?:(\d+)S)?$`; cualquier otro valor se lee como cero.

# Apéndice D

## Vectores de Prueba

Los vectores de prueba son entradas concretas con la salida *exacta* del código de producción. Los genera `vectores.php`, un programa que va con el código privado: carga el motor tal como está (las clases del emparejador, los factores y los tiempos por tramo, y las funciones de calibración del script de aprendizaje, sin ejecutar su programa principal) y escribe `referencia/vectores.json`. Una implementación en cualquier lenguaje puede darse por correcta cuando reproduce todos los vectores; `referencia/comprobar.py` lo hace para la implementación de referencia del Apéndice E.

Los valores reales se dan con todos sus decimales para comprobar también los cálculos en coma flotante; los resultados que el método redondea deben coincidir exactamente.

### Funciones Básicas

**Tabla D1**

*Mediana Baja y Percentiles*

| Valores | $p$ | Resultado |
|---|---|---|
| 5, 1, 4, 2, 3 | 0,5 | 3 |
| 1, 2, 3, 4 | 0,5 | 2 (no 2,5: mediana baja) |
| 10, 20, …, 100 | 0,9 | 90 |

**Tabla D2**

*Franja de una Fecha y un Instante*

| Fecha | Día | Segundos | Franja |
|---|---|---|---|
| 2026-10-28 | miércoles | 28 950 (8:02:30) | `laborable.punta_manana` |
| 2026-10-28 | miércoles | 21 599 (5:59:59) | `laborable.madrugada` |
| 2026-10-28 | miércoles | 21 600 (6:00:00) | `laborable.punta_manana` |
| 2026-10-31 | sábado | 75 600 (21:00:00) | `fin_semana.noche` |
| 2026-10-28 | miércoles | 90 000 (25:00:00) | `laborable.madrugada` |
| 2026-11-01 | domingo | 46 800 (13:00:00) | `fin_semana.mediodia` |

**Tabla D3**

*Factor de Ritmo*

| Línea | Segundos | $k$ | Por qué |
|---|---|---|---|
| 3414 | 28 950 | 1,2 | Factor de línea 1,35, acotado |
| 9999 | 28 950 | 0,9 | Sin factor de línea: el global de la franja |
| 9999 | 79 200 | 0,6 | Global 0,4 de la noche, acotado |
| 9999 | 50 400 | 1,0 | Sin factor para el mediodía |
| 03414 | 28 950 | 1,2 | Se lee como la 3414 |

*Nota.* Con $k_{\texttt{laborable.punta_manana}} = 0{,}9$, $k_{\texttt{laborable.noche}} = 0{,}4$ y

$k_{3414,\texttt{laborable.punta_manana}} = 1{,}35$, el miércoles 2026-10-28.

**Tabla D4**

*Proyección Sobre un Segmento de Prueba*

| Punto $(\varphi, \lambda)$ | $f$ | $d_\perp$ (m) | $\|B - A\|$ (m) |
|---|---|---|---|
| (43,2601; −2,925) | 0,5000000000000222 | 11,132000000369544 | 810,6880955946056 |
| (43,2605; −2,935) | 0 | 409,14505231598264 | 810,6827696075283 |
| Segmento degenerado $A = B$, punto en $A$ | 0 | 0 | 0 |

*Nota.* Segmento de $A = (43{,}26; -2{,}93)$ a $B = (43{,}26; -2{,}92)$.

La longitud del segmento cambia ligeramente entre las dos primeras filas porque el factor $\cos\varphi$ se toma en la latitud del autobús, no en la del segmento.

**Tabla D5**

*Cubos de Longitud*

| Metros | Cubo | Metros | Cubo | Metros | Cubo |
|---|---|---|---|---|---|
| 0 | 0 | 999,9 | 2 | 2 999,9 | 4 |
| 299,9 | 0 | 1 000 | 3 | 3 000 | 5 |
| 300 | 1 | 1 499,9 | 3 | 12 000 | 5 |
| | | 1 500 | 4 | | |

## Calibración

### *Tiempos por Tramo*

Con las quince muestras siguientes (pista, franja, tramo, tiempo real), la calibración da exactamente `A>B`: $\{\texttt{n} : 8,\ \texttt{all} : 61,\ \texttt{laborable.punta_manana} : 69,\ \texttt{laborable.mediodia} : 60\}$,

**Tabla D6**

*Respaldo por Longitud*

| $h$ (s) | $L$ (m) | Franja | Resultado (s) | Por qué |
|---|---|---|---|---|
| 120 | 800 | `laborable.punta_manana` | 97,32000000000001 | $0{,}811 \cdot 120$ |
| 120 | 800 | `fin_semana.noche` | 92,28 | $0{,}769 \cdot 120$ (`all`) |
| 300 | 2 000 | `laborable.manana` | 450 | 2,0 acotado a 1,5 |
| 300 | 5 000 | `laborable.manana` | 90 | 0,1 acotado a 0,3 |
| 300 | 100 | `laborable.manana` | nulo | Cubo 0 sin proporción |
| 300 | desconocida | `laborable.manana` | nulo | Sin longitud |

*Nota.* Con las proporciones $r_{2,\texttt{all}} = 0{,}769$, $r_{2,\texttt{laborable.punta_manana}} = 0{,}811$, $r_{4,\texttt{all}} = 2{,}0$ y $r_{5,\texttt{all}} = 0{,}1$.

`C>D`: $\{\texttt{n} : 3,\ \texttt{laborable.noche} : 200\}$ y `D>E`: $\{\texttt{n} : 2,\ \texttt{laborable.manana} : 60\}$; el tramo `B>C`, con una sola pista, no se guarda.

| Pista | Franja | Tramo | $y$ | Pista | Franja | Tramo | $y$ |
|---|---|---|---|---|---|---|---|
| b1 | punta_manana | A>B | 90 | m5 | mediodia | A>B | 61 |
| b2 | punta_manana | A>B | 100 | b1 | punta_manana | B>C | 75 |
| m0 | mediodia | A>B | 60 | c0 | noche | C>D | 200 |
| m1 | mediodia | A>B | 58 | c1 | noche | C>D | 210 |
| m2 | mediodia | A>B | 62 | c2 | noche | C>D | 190 |
| m3 | mediodia | A>B | 64 | d1 | manana | D>E | 50 |
| m4 | mediodia | A>B | 59 | d1 | manana | D>E | 70 |
| | | | | d2 | manana | D>E | 60 |

El cálculo de `A>B` está en el ejemplo 2. En `D>E`, la pista d1 tiene dos muestras y cuenta la última (70); la mediana baja de $\{70, 60\}$ es 60.

***Factor de Ritmo***

Con una muestra por pista, de $H = 600$ s y $Y = 600\,\rho_j$:

| Proporciones por pista $\rho_j$ | $k$ |
|---|---|
| 0,80, 0,82, 0,84, 0,86, 0,88, 0,90, 0,92, 0,94 | 0,938 |
| cuarenta veces 0,85 | 0,88 |
| 1,6, 1,7, …, 2,5 | 1,2 |

***Paso Real***

Con la pista $(28\,900,\ 28\,880)$, $(28\,960,\ 28\,940)$, $(29\,020,\ 29\,030)$: para $a = 28\,950$, $P = 28\,966{,}666666666668$; para $a = 28\,800$, nulo.

***Aclarado***

De 100 muestras de una pista se conservan las de índices 0, 3, 6, 10, 13, 16, 20, 23, 26, 30, 33, 36, 40, 43, 46, 50, 53, 56, 60, 63, 66, 70, 73, 76, 80, 83, 86, 90, 93 y 96.

**Corrección por Viaje**

***Consulta***

La tabla de los vectores tiene el viaje `9999|S0|S4|28800|5` con $[20, 40, 60]$ en `laborable` y $[-30, 0, 30]$ en `fin_semana`; la línea `L2`, con 15 s en `laborable.punta_manana`; y por franja, 25 s en `laborable.punta_manana` y $-10$ s en `fin_semana.noche`. Los tercios (33) de las posiciones 0 a 4 de un recorrido de cinco paradas son 0, 0, 1, 1 y 2; los de las posiciones 1 y 2 de tres, 1 y 2; la posición 9 de cinco da 2, y un recorrido sin paradas, 0. En un festivo, la capa 3 calcula con el domingo de esa semana: el lunes 12 de octubre y el sábado 17 dan el 18; el domingo 1 de noviembre, el mismo día; y el martes 8 de diciembre, el 13.

| Viaje | Fecha | Salida, posición | Resultado (s) | Por qué |
|---|---|---|---|---|
| `9999\|S0\|S4\|28800\|5` | 2026-10-28 | 28 800, 4 de 5 | 60 | Tercer tercio, laborable |
| `9999\|S0\|S4\|28800\|5` | 2026-10-28 | 28 800, 2 de 5 | 40 | Segundo tercio |
| `9999\|S0\|S4\|28800\|5` | 2026-10-31 | 28 800, 1 de 5 | −30 | Sábado: fin de semana |
| `L2\|A\|B\|28800\|3` | 2026-10-28 | 28 800, 1 de 3 | 15 | Línea en su franja |
| `L2\|A\|B\|72000\|3` | 2026-10-31 | 72 000, 1 de 3 | −10 | La línea no tiene esa franja |
| `L3\|A\|B\|28800\|3` | 2026-10-28 | 28 800, 1 de 3 | 25 | Sin línea: la franja |
| `L3\|A\|B\|50400\|3` | 2026-10-28 | 50 400, 1 de 3 | 0 | Nada para esa franja |
| `09999\|S0\|S4\|28800\|5` | 2026-10-28 | 28 800, 4 de 5 | 60 | Se lee como la 9999 |

***Calibración***

Con los nueve desvíos siguientes (fecha, $D'$, viaje, posición y desvío en segundos), la calibración (40) da para el viaje `L1|A|D|28800|4` en laborable $[127, 150, 420]$; para `L1|A|D|32400|4`, $[-60, -60, -60]$ en laborable y $[30, 30, 40]$ en fin de semana; y para `L2|X|Y|61200|6`, $[300, 300, 300]$. Las líneas quedan en `L1`: {`laborable.punta_manana`: 140, `laborable.manana`: −60, `fin_semana.manana`: 30} y `L2`: {`laborable.punta_tarde`: 300}, con los mismos valores por franja.

| Fecha | $D'$ | Viaje | Posición | Desvío |
|---|---|---|---|---|
| 2026-10-26 | 2026-10-26 | `L1\|A\|D\|28800\|4` | 1, 2 y 3 de 4 | 100, 160 y 220 |
| 2026-10-27 | 2026-10-27 | `L1\|A\|D\|28800\|4` | 1 y 3 de 4 | 140 y 2 000 |
| 2026-10-26 | 2026-10-26 | `L1\|A\|D\|32400\|4` | 2 de 4 | −60 |
| 2026-10-31 | 2026-10-31 | `L1\|A\|D\|32400\|4` | 1 de 4 | 30 |
| 2026-10-12 | 2026-10-18 | `L1\|A\|D\|32400\|4` | 3 de 4 | 50 |
| 2026-10-26 | 2026-10-26 | `L2\|X\|Y\|61200\|6` | 5 de 6 | 300 |

Para `L1` en `laborable.punta_manana`, el desvío de 2 000 s se recorta a 900. Ordenados, $100, 140, 160, 220, 900$, el percentil 40 es el de la posición $\lfloor 0{,}4 \cdot 4 \rfloor = 1$: $g = 140$, y la línea, con el peso 20 de la franja, también vale 140. El primer tercio del viaje (posición 1 de 4) tiene un desvío

cada día, 100 y 140: rnd $\big((100+140+140)/3\big) = 127$. El tercer tercio, con 220 y 900, da rnd$(1\,260/3) = 420$. El festivo del 12 de octubre cuenta como fin de semana: con el 31 de octubre, la franja `fin_semana.manana` tiene 30 y 50, y su percentil 40 es 30; el primer tercio da $(30+30)/2 = 30$, el segundo toma el valor de la línea y el tercero da $(50+30)/2 = 40$.

**El Emparejador Completo**

El viaje sintético tiene cinco paradas, `S0` a `S4`, en línea recta hacia el este sobre el paralelo 43,26° desde la longitud −2,93°, a 0, 400, 1 000, 2 500 y 3 200 m (longitudes −2,93, −2,925065928003549, −2,917664820008872, −2,8991620500221806 y −2,890527424028391), con llegada igual a salida en cada parada: 28 800, 28 890, 29 010, 29 310 y 29 430 s. El trazado, cuando se usa, son las cinco paradas con sus distancias. La fecha es el miércoles 2026-10-28, la línea `9999` y el vehículo `V1`. Los parámetros son:

- tramos aprendidos: `S0>S1` {`n` : 10, `all` : 100, `laborable.punta_manana` : 110} y `S1>S2` {`n` : 6, `all` : 150, `laborable.punta_manana` : 140};
- proporciones: cubo 2, {`all` : 0,769, `laborable.punta_manana` : 0,811}; cubo 4, {`all` : 0,64, `laborable.punta_manana` : 0,668};
- factor global $k_{\texttt{laborable.punta_manana}} = 0{,}9$, sin factores de línea.

En el escenario de «horario muy lento», las horas de `S3` y `S4` pasan a 30 810 y 30 930, y la tabla de tramos se sustituye por `S0>S1`, `S1>S2`, `S2>S3` y `S3>S4` con 110, 140, 240 y 100 s en `laborable.punta_manana` (y `n`= 9).

Las entradas del feed de cada escenario son:

| Escenario | Entrada |
|---|---|
| 1 y 2 | $\varphi = 43{,}2601$, $\lambda = -2{,}9213653740062107$ (700 m), $t_{\text{gps}} = t_{\text{rec}} = 28\,950$, $n = 3$ |
| 3 | $\varphi = 43{,}260538986704994$ (60 m al norte), mismo $\lambda$ y horas, $n = 3$ |
| 4 | sin posición, $t_{\text{rec}} = 28\,950$, $n = 3$ |
| 5 | $\varphi = 43{,}2601$, $\lambda = -2{,}895461496024842$ (2 800 m), $t_{\text{gps}} = t_{\text{rec}} = 29\,400$, $n = 3$ |
| 6, 7, 8, 10 y 11 | ninguna |
| 9 | como el 1, con $t_{\text{gps}} = t_{\text{rec}} = 28\,300$ |
| 12 | $\varphi = 43{,}2601$ (11 m al norte de `S0`), $\lambda = -2{,}93$, $t_{\text{gps}} = t_{\text{rec}} = 28\,200$, $n = 2$, $t_{\text{now}} = 28\,210$ |
| 13 | $\varphi = 43{,}26538986704995$ (600 m al norte), mismo $\lambda$ y horas, $n = 2$ |
| 14 y 16 | como el 1, con $t_{\text{now}} = 29\,100$ y $29\,320$ |
| 15 | $\varphi = 43{,}2601$, $\lambda = -2{,}917048061009316$ (1 050 m, 50 m pasada `S2`), $t_{\text{gps}} = t_{\text{rec}} = 29\,020$, $n = 3$, $t_{\text{now}} = 29\,040$ |

Los escenarios 10 y 11 usan la tabla de corrección de la sección anterior; el 11, además, con la fecha marcada como festivo (un calendario de solo domingo que circula ese día por excepción). Los resultados están en la Tabla 9. Las posiciones calculadas son $T_{\text{pos}} = 28\,950$ con $d_{\perp} = 11{,}132000000369544$ m en los escenarios 1 (método «trazado») y 2 (método «tramo recto»); en el 12, $T_{\text{pos}} = t_{\text{gps}} = 28\,800$ con la misma distancia y el método «cabecera»; en el 14 y el 16, la del 1; en el 15, $T_{\text{pos}} = 29\,020$ con el método «trazado»; no hay posición en los demás.

**Del Feed a la Pantalla**

Las reglas de esta sección rodean al emparejador: cómo se lee el feed, a qué viaje del horario se asigna cada bus, qué pasa con un bus que falta en una lectura o con un viaje que pasa de medianoche, qué se guarda de cada paso, qué retraso se muestra y cómo se elige cada noche el percentil de la corrección por viaje. Sus vectores salen del mismo motor y `comprobar.py` los comprueba igual que los anteriores.

***Lectura del Feed***

Un fragmento SIRI con cuatro actividades, leído con fecha 2026-10-28. La referencia `trp_A0651_737_ABC1_28500` da la línea `651` (sin el cero), el viaje `737` y la salida 28 500; su

`Delay PT2M30S` son 150 s; la hora de la posición, `07:02:30Z`, es $t_{\text{gps}} = 28\,950$ en Madrid, y la de la lectura, $t_{\text{rec}} = 28\,960$. La segunda actividad no trae posición, su orden sale de `VisitNumber` y su `Delay -PT45S` son $-45$ s. La tercera se registró el día anterior: entra en el feed, pero sin horas ni posición. La cuarta tiene una referencia de otro formato y no entra.

***Emparejado***

Con la referencia exacta `trp_A9999_101_X_28800` del viaje `J` en la tabla de alias:

| Buses de la línea `9999`, viaje `101` | Viaje pedido | Bus elegido |
|---|---|---|
| Uno con la referencia exacta y otro con otra referencia y la misma salida | `J` | el exacto |
| Sin referencia exacta, salida a 120 s | `J` | ese |
| Sin referencia exacta, salida a 180 s justos | `J` | ese |
| Sin referencia exacta, salida a 200 s | `J` | ninguno |
| Dos sin referencia exacta, a 150 y a 60 s | `J` | el de 60 s |
| Solo el de la referencia exacta de `J` | `K` | ninguno (no se reutiliza) |
| El de `J` y otro sin referencia exacta | `K` | el otro |

***Buses que Faltan***

Con $t_{\text{now}} = 29\,010$, de la lectura anterior se conservan, marcados como retenidos, el bus visto hace 110 s y el que solo tiene hora de lectura de hace 180 s justos. No se conservan: el de hace 190 s ni el de hace 210 s; el de un viaje que en la lectura nueva lleva otro bus; el que aparece en la lectura nueva en otro viaje; el que no tiene hora; el de hora posterior a ahora; ni el que no tiene identificador de vehículo.

***Viajes que Pasan de Medianoche***

A las 00:30 ($t_{\text{now}} = 1\,800$), el viaje que salió a las 23:30 (84 600) y el de las 24:10 del horario de ayer (87 000) son de ayer; uno que sale a las 12:30 (45 000, justo 12 h por delante) es de hoy, y uno a 45 001, de ayer. Para los de ayer, sus horas pasan a contarse desde la medianoche de ayer: un dato de las 00:28:20 (1 700) pasa a 88 100. Un bus sin hora de hoy no cuenta en ningún día.

***Registro de Pasos***

De cada referencia se guarda «orden:hora», con la hora de la lectura o, sin ella, la de la posición: `3:29000` y `2:28900`. No se guardan las entradas sin orden, sin referencia o sin hora. Al leer, por viaje del horario queda el orden mayor de sus referencias (`3:29000` y `5:29100` dan orden 5 a las 29 100); los valores nulos o mal formados se ignoran. Con ese paso, un viaje sin bus en el feed tiene pasadas las paradas de orden menor:

| Paso registrado | $t_{\text{now}}$ | `S0`, `S1`, `S2` | `S3` | `S4` |
|---|---|---|---|---|
| orden 4 a las 29 300 | 29 400 | pasadas, a las 29 300 | horario | capa 3: 29 408 |
| orden 2 a las 29 000 | 28 990 | `S0` pasada a las 28 990 (ahora) | capa 3: 29 310 | capa 3: 29 408 |

En el segundo caso, `S1` queda en el horario (28 890) y `S2` con la capa 3 (29 110).

***El Retraso Mostrado***

Los minutos son los segundos entre 60 redondeados con mitades hacia fuera del cero: 29 s dan 0; 30, 1; 89, 1; 90, 2; $-30$, $-1$; 2 700, 45; y un retraso de más de 2 700 s no se muestra. Del bus del escenario 1, con GPS, se muestra el retraso que llevará en su próxima parada, 10 s, aunque su retraso en la posición sea 0; sin GPS (escenario 4), el de la parada registrada, 10 s; y con un dato de más de 10 min y sin hora de lectura, 0.

***Elección del Percentil***

Con un viaje de la línea `L1` y cinco desvíos en cada uno de cuatro días, se aprende con los tres primeros y se prueba en el último:

| Percentil | 0,30 | 0,35 | 0,40 | 0,45 | 0,50 |
|---|---|---|---|---|---|
| Error medio (s) | 144,4 | 144,4 | 147,4 | 149,2 | 151,2 |
| A $\pm 2$ min (%) | 20 | 20 | 40 | 60 | 60 |

Se elige 0,45: el que más acierta a $\pm 2$ min y, en el empate con 0,50, el de menos error medio, aunque 0,30 tenga el menor. Sin corregir, el error medio es 126 s y el acierto, 60 %: la tabla nueva no bajaría el error medio y no se escribe.

# Apéndice E

## Implementación de Referencia

La implementación de referencia, en Python 3 y sin dependencias externas, se escribió a partir de la especificación de esta memoria y no del código PHP. Reproduce todos los vectores del Apéndice D. Su propósito es servir de punto de partida para portar el método a otro lenguaje: es corta, sigue el orden de los capítulos y usa los mismos nombres que el texto.

### Cómo Comprobarla

Los ficheros van adjuntos a este PDF: `referencia.py`, `comprobar.py`, `vectores.json` y `desvio-horario.py`. La mayoría de los lectores de PDF los muestran en un panel de adjuntos, desde el que se pueden guardar; también se pueden extraer con `pdfdetach -saveall` (de Poppler). Con los cuatro en una carpeta:

```
python3 comprobar.py
# 407 de 407 comprobaciones coinciden con el código de producción
```

La carpeta `referencia` del repositorio público trae además `evaluar.py`, el evaluador público: repite la evaluación de la sección «Evaluación a lo Largo de Varios Días» sobre capturas del feed y escribe los mismos histogramas de 15 s que la de producción, idénticos clase a clase en las 92 686 predicciones de 40 minutos de feed del 7 de noviembre de 2026. Los vectores los generó el autor con el código de producción, que es privado, con el programa `vectores.php`, que necesita ese código y por eso no se publica; `vectores.json` trae además un caso que `comprobar.py` no usa (`k_un_bus`, el factor sin contraer de una sola pista). No hace falta ese código para usarlos: una implementación nueva, en cualquier lenguaje, está bien hecha cuando reproduce todos los vectores. El programa `desvio-horario.py` recalcula las cifras de la sección «El Horario Publicado Se Desvía en los Dos Sentidos»; necesita la base del horario y las muestras de tramo, que se generan con el propio método (capítulos «Fuentes y Modelo de Datos» y «Aprendizaje Continuo»).

**Lo que Hay que Respetar al Portar**

1. **Redondeo.** rnd redondea las mitades alejándose del cero. En Python, `round(2.5)` da 2; hay que usar la función `php_round` de la referencia. En JavaScript, `Math.round(-2.5)` da $-2$ y tampoco sirve para negativos. El redondeo a tres decimales es $\mathrm{rnd}(1000x)/1000$.

2. **Mediana baja.** $x_{(\lfloor p(n-1) \rfloor)}$ sobre los valores ordenados, sin promediar los dos centrales. Las funciones de mediana de las bibliotecas estadísticas suelen interpolar.

3. **División entera.** $t_{\mathrm{mid}} = \lfloor (d_1 + a_{\mathrm{dest}})/2 \rfloor$ y la hora $\lfloor t/3600 \rfloor$ mód 24 se calculan con división entera.

4. **Truncado.** El campo `at` de las muestras de tramo es la parte entera del paso real, no su redondeo.

5. **CRC32 sin signo.** El pliegue de validación usa el CRC32 de IEEE de los bytes UTF-8 de la pista, como entero sin signo; en lenguajes con enteros de 32 bits con signo hay que convertirlo antes del módulo.

6. **Lista u objeto.** `lengthRatios` puede llegar como lista o como objeto (Apéndice C).

7. **Orden de evaluación.** La posición, con su contraste, se evalúa antes que cualquier uso del registro (sección «Contraste Entre el GPS y la Parada Registrada»).

8. **Bucles por horas.** El tiempo esperado recorre los tramos comparando horas, no índices (sección «Tiempo Esperado Entre Dos Instantes del Horario»).

9. **Festivos.** La capa 3 de un festivo entre semana se calcula con el domingo de esa semana, también la franja de la salida con la que se busca la corrección de la línea.

10. **Dos redondeos.** En la capa 3, el tiempo esperado y la corrección por viaje se redondean por separado.

11. **El día del viaje.** La referencia recibe la entrada, $t_{\text{now}}$ y la fecha ya en las horas del día del viaje. Pasada la medianoche, una entrada de un viaje de ayer (35) se calcula con $t_{\text{gps}}$, $t_{\text{rec}}$ y $t_{\text{now}}$ más 86 400 s y con la fecha de ayer; una entrada sin ninguna hora de hoy no se usa (sección «Viajes que Pasan de las 24:00»). El registro de pasos y la memoria del feed, que dependen de un almacén externo, no están en la referencia.

### Código

### Listing E.1

*referencia.py*

```
"""Implementación de referencia del motor de llegadas de Bizkaibus+, escrita solo a partir de la especificación
de la tesis (no del código PHP). Se valida contra los vectores de prueba generados con el código de producción.

Convenciones que hay que respetar al portar a otro lenguaje:
  * Tiempos en segundos desde la medianoche de Europe/Madrid del día de servicio (pueden pasar de 86 400).
  * round() es el redondeo de PHP: mitades hacia fuera del cero (no el «redondeo del banquero» de Python).
  * Las medianas son la mediana BAJA: x_(floor(p*(n-1))) sobre los valores ordenados.
"""
import datetime
import math
import re
import zlib
from xml.etree import ElementTree
from zoneinfo import ZoneInfo

# ---------------------------------------------------------------------- ----- constantes
MADRID = ZoneInfo('Europe/Madrid')
METERS_PER_DEGREE = 111320.0
MAX_LOCATION_AGE = 600            # s: antigüedad máxima del dato del bus
NOT_STARTED_GRACE = 60            # s: el viaje cuenta como empezado 60 s antes de su salida
MAX_WAIT_AT_ORIGIN = 3600         # s: el bus que espera en la primera parada se sitúa hasta 1 h antes de salir
SEGMENTS_BEHIND, SEGMENTS_AHEAD = 3, 2
MAX_FROM_SHAPE = 50.0             # m: pasillo con trazado
```

```
MAX_FROM_ROUTE, MAX_FROM_ROUTE_RATIO = 150.0, 0.15   # m y fracción: pasillo sin trazado
BEYOND_SLACK, AMBIGUITY = 100.0, 20.0                # m
REGISTRATION_SHARE = 0.5
REGISTRATION_MAX_SEGMENT = 1200   # s
CROSSCHECK_MAX_SEGMENT = 360      # s
MAX_GPS_VS_STOP = 360             # s
MAX_PLAUSIBLE_DELAY = 2700        # s
STALE_GRACE = 120                 # s
PASSED_DISTANCE = 150.0           # m: el GPS, más lejos de la parada, la da por pasada aunque el feed no lo diga
ARRIVING_MAX = 180                # s: en «0 min» sin confirmar el paso antes de «puede que ya haya pasado»
SCHEDULE_ORIGIN_OFFSET = 60       # s
WRONG_SCHEDULE_GAP = WRONG_SCHEDULE_PASSED = 600     # s
WRONG_SCHEDULE_LEARNED_SHARE = 0.8
K_MIN, K_MAX = 0.6, 1.2
RATIO_MIN, RATIO_MAX = 0.3, 1.5
LENGTH_CUTS = [300, 600, 1000, 1500, 3000]
BANDS = [(0, 6, 'madrugada'), (6, 9, 'punta_manana'), (9, 13, 'manana'), (13, 16, 'mediodia'),
         (16, 20, 'punta_tarde'), (20, 24, 'noche')]
MIN_TRACKS_PER_SEGMENT = 2
SEGMENT_BAND_PRIOR_TRACKS = 5
SHRINKAGE_TRACKS = 10
MAX_SAMPLES_PER_TRACK = 30
MAX_GAP_BETWEEN_SNAPSHOTS = 300
TRIP_PARTS = 3                    # tercios del recorrido en la corrección por viaje
TRIP_OFFSET_RANK = 0.4            # percentil de los desvíos que toma la corrección por viaje (no la media)
TRIP_OFFSET_CLIP = 900            # s: recorte de cada desvío al calibrar la corrección por viaje
TRIP_OFFSET_LINE_PRIOR = 20       # pasos: peso de la franja en la corrección de la línea
TRIP_OFFSET_RANKS = [0.3, 0.35, 0.4, 0.45, 0.5]   # percentiles entre los que elige la recalibración
TRIP_OFFSET_TRIP_PRIOR_DAYS = 1   # días: peso de la línea en la corrección del viaje


# ------------------------------------------------------------------------- utilidades
def php_round(x):
    """round() de PHP con 0 decimales: las mitades se alejan del cero."""
```

```
    if x >= 0:
        return math.floor(x + 0.5)
    return -math.floor(-x + 0.5)


def percentile(values, p):
    ordered = sorted(values)
    return float(ordered[int(math.floor(p * (len(ordered) - 1)))])


def clamp(x, lo, hi):
    return max(lo, min(hi, x))


# ------------------------------------------------------------------------
    ----- franja y factores
def band_for(date_iso, seconds):
    weekday = datetime.date.fromisoformat(date_iso).isoweekday()   # 1 =
    lunes ... 7 = domingo
    day_type = 'laborable'
    if weekday >= 6:
        day_type = 'fin_semana'
    hour = (seconds // 3600) % 24
    for lo, hi, name in BANDS:
        if lo <= hour < hi:
            return day_type + '.' + name
    return day_type + '.noche'


def line_key(line):
    """El número de línea sin ceros a la izquierda: el feed escribe 0651 y
    el horario, 651."""
    if line is None:
        return None
    key = str(line).lstrip('0')
    if key == '':
        return '0'
    return key


def by_line(table):
    """Una tabla por línea con los números de línea sin ceros a la
    izquierda; si dos coinciden, queda la primera."""
    out = {}
    for line, values in (table or {}).items():
```

```
        out.setdefault(line_key(line), values)
    return out


def pace_factor(pace, line, date_iso, seconds):
    band = band_for(date_iso, seconds)
    k = 1.0
    lines = by_line(pace.get('lines', {}))
    key = line_key(line)
    if key is not None and band in lines.get(key, {}):
        k = lines[key][band]
    elif band in pace.get('global', {}):
        k = pace['global'][band]
    return clamp(float(k), K_MIN, K_MAX)


# ------------------------------------------------------------------------
    ----- tiempos por tramo
def length_bucket(meters):
    for i, cut in enumerate(LENGTH_CUTS):
        if meters < cut:
            return i
    return len(LENGTH_CUTS)


def segment_seconds(segments, from_id, to_id, band):
    seg = segments.get('segments', {}).get(from_id + '>' + to_id)
    if seg is None:
        return None
    if band in seg:
        return float(seg[band])
    if 'all' in seg:
        return float(seg['all'])
    return None


def length_ratios(segments):
    """lengthRatios como diccionario {'0': {...}, ...}. PHP lo escribe
    como lista si tiene todos los cubos 0..5."""
    ratios = segments.get('lengthRatios') or {}
    if isinstance(ratios, list):
        return {str(i): r for i, r in enumerate(ratios)}
    return ratios


```

```
def fallback_seconds(segments, scheduled, meters, band):
    ratios = length_ratios(segments)
    if not ratios or meters is None or meters <= 0:
        return None
    bucket = ratios.get(str(length_bucket(meters)))
    if bucket is None:
        return None
    ratio = bucket.get(band, bucket.get('all'))
    if ratio is None:
        return None
    return clamp(float(ratio), RATIO_MIN, RATIO_MAX) * scheduled


def expected_seconds(stops, segments, t_from, t_target, band, k):
    """Tiempo esperado para ir del instante programado t_from al t_target,
    tramo a tramo."""
    if not segments.get('segments'):
        return k * (t_target - t_from)
    total = 0.0
    for i in range(len(stops) - 1):
        leave, reach = stops[i]['arr'], stops[i + 1]['arr']
        if reach <= t_from:
            continue
        if leave >= t_target:
            break
        scheduled = reach - leave
        tau = segment_seconds(segments, stops[i]['id'], stops[i + 1]['id'],
   band)
        if tau is None:
            meters = None
            if stops[i]['dist'] is not None and stops[i + 1]['dist'] is not
    None:
                meters = stops[i + 1]['dist'] - stops[i]['dist']
            tau = fallback_seconds(segments, float(scheduled), meters, band)

            if tau is None:
                tau = k * scheduled
        share = 1.0
        if scheduled > 0:
            share = (min(reach, t_target) - max(leave, t_from)) / scheduled
        total += max(0.0, share) * tau
    return total


def learned_share(stops, segments, t_from, t_target, band):
```

```
    total = learned = 0
    for i in range(len(stops) - 1):
        leave, reach = stops[i]['arr'], stops[i + 1]['arr']
        if reach <= t_from:
            continue
        if leave >= t_target:
            break
        total += reach - leave
        if segment_seconds(segments, stops[i]['id'], stops[i + 1]['id'],
    band) is not None:
            learned += reach - leave
    if total <= 0:
        return 0.0
    return learned / total


# ------------------------------------------------------------------------
    ----- geometría
# ------------------------------------------------------------------------
    ----- corrección por viaje
def trip_key(line, first_stop, last_stop, departure, count):
    return '%s|%s|%s|%d|%d' % (line_key(line), first_stop, last_stop,
    departure, count)


def same_line(key):
    """La clave de un viaje con su número de línea sin ceros a la
    izquierda."""
    parts = key.split('|', 1)
    parts[0] = line_key(parts[0])
    return '|'.join(parts)


def trip_part(index, count):
    if count <= 0:
        return 0
    return clamp((TRIP_PARTS * index) // count, 0, TRIP_PARTS - 1)


def day_type(date_iso):
    return band_for(date_iso, 0).split('.')[0]


def service_date(date_iso, holiday):
```

```
    """Día con el que calcula la capa 3: en un festivo entre semana, el domingo de esa semana."""
    if not holiday:
        return date_iso
    day = datetime.date.fromisoformat(date_iso)
    return (day + datetime.timedelta(days=7 - day.isoweekday())).isoformat()


def trip_offset(table, key, line, date_iso, departure, index, count):
    """Segundos que se suman a la capa 3: los del viaje en su tercio; si no, los de su línea y franja; si no, la franja."""
    trips = {}
    for k, v in table.get('trips', {}).items():
        trips.setdefault(same_line(k), v)
    parts = trips.get(same_line(key), {}).get(day_type(date_iso))
    if parts is not None:
        return float(parts[trip_part(index, count)])
    band = band_for(date_iso, departure)
    line_bands = by_line(table.get('lines', {})).get(line_key(line), {})
    if band in line_bands:
        return float(line_bands[band])
    if band in table.get('global', {}):
        return float(table['global'][band])
    return 0.0


def project_onto_segment(lat, lon, a, b):
    m_lon = math.cos(math.radians(lat)) * METERS_PER_DEGREE
    ax, ay = (a['lon'] - lon) * m_lon, (a['lat'] - lat) * METERS_PER_DEGREE
    bx, by = (b['lon'] - lon) * m_lon, (b['lat'] - lat) * METERS_PER_DEGREE
    dx, dy = bx - ax, by - ay
    l2 = dx * dx + dy * dy
    f = 0.0
    if l2 > 0:
        f = clamp(-(ax * dx + ay * dy) / l2, 0.0, 1.0)
    px, py = ax + f * dx, ay + f * dy
    return f, math.hypot(px, py), math.sqrt(l2)


def position(journey, live, now):
    """Capa 1: dónde pone el horario al bus (T_pos). None si el GPS no sirve."""
    stops, shape = journey['stops'], journey.get('shape')
```

```
    if live.get('lat') is None or live.get('lon') is None or live.get('locationSeconds') is None or live.get('order') is None:
        return None
    t_gps = live['locationSeconds']
    if now - t_gps > MAX_LOCATION_AGE or len(stops) < 2:
        return None
    if t_gps < stops[0]['dep'] - NOT_STARTED_GRACE:
        return waiting_at_origin(live, stops[0], t_gps)
    nxt = next((i for i, s in enumerate(stops) if s['seq'] == live['order']), None)
    if nxt is None:
        return None
    first, last = max(0, nxt - SEGMENTS_BEHIND), min(len(stops) - 2, nxt + SEGMENTS_AHEAD)
    if shape and all(stops[i]['dist'] is not None for i in range(first, last + 2)):
        lo, hi = stops[first]['dist'], stops[last + 1]['dist']
        limit = stops[nxt]['dist'] + BEYOND_SLACK
        best_exp = best_beyond = None
        for (alat, alon, ad), (blat, blon, bd) in zip(shape, shape[1:]):
            if bd < lo or ad > hi:
                continue
            f, d, _ = project_onto_segment(live['lat'], live['lon'], {'lat': alat, 'lon': alon}, {'lat': blat, 'lon': blon})
            along = ad + f * (bd - ad)
            if along <= limit:
                if best_exp is None or d < best_exp[0]:
                    best_exp = (d, along)
            elif best_beyond is None or d < best_beyond[0]:
                best_beyond = (d, along)
        best = best_exp
        if best_beyond is not None and (best is None or best_beyond[0] + AMBIGUITY < best[0]):
            best = best_beyond
        if best is None or best[0] > MAX_FROM_SHAPE:
            return None
        d, along = best
        along = clamp(along, lo, hi)
        for i in range(first, last + 1):
            s0, s1 = stops[i]['dist'], stops[i + 1]['dist']
            if along > s1 and i < last:
                continue
            f = 0.0
            if s1 > s0:
                f = clamp((along - s0) / (s1 - s0), 0.0, 1.0)
```

```
            sched = php_round(stops[i]['dep'] + f * (stops[i + 1]['arr'] - stops[i]['dep']))
            return {'scheduledSeconds': sched, 'locationSeconds': t_gps, 'offRouteMeters': d, 'method': 'shape'}
        return None
    best = None
    for i in range(first, last + 1):
        f, d, length = project_onto_segment(live['lat'], live['lon'], stops[i], stops[i + 1])
        if best is None or d < best[0]:
            allowed = max(MAX_FROM_ROUTE, length * MAX_FROM_ROUTE_RATIO)
            best = (d, allowed, stops[i]['dep'] + f * (stops[i + 1]['arr'] - stops[i]['dep']))
    if best is None or best[0] > best[1]:
        return None
    return {'scheduledSeconds': php_round(best[2]), 'locationSeconds': t_gps, 'offRouteMeters': best[0], 'method': 'segment'}


def waiting_at_origin(live, first, t_gps):
    """Antes de su hora: el bus que espera en la primera parada (a 150 m o menos) ya se sitúa allí."""
    if first['dep'] - t_gps > MAX_WAIT_AT_ORIGIN:
        return None
    dx = (live['lon'] - first['lon']) * math.cos(math.radians(first['lat']))
    dy = live['lat'] - first['lat']
    meters = math.hypot(dx, dy) * METERS_PER_DEGREE
    if meters > MAX_FROM_ROUTE:
        return None
    return {'scheduledSeconds': min(first['arr'], first['dep']), 'locationSeconds': first['dep'],
            'offRouteMeters': meters, 'method': 'origin'}


DAY_SECONDS = 86400
MATCH_TOLERANCE = 180            # s: emparejado por línea y número de viaje cuando falta la referencia exacta
HOLD_SECONDS = 180               # s: un bus que falta en una lectura del feed se sigue usando con su último dato


# ---------------------------------------------------------------------- el feed
def iso_duration_seconds(text):
```

```
    """«PT2M30S» son 150 s y «-PT45S», -45; lo que no tiene esa forma, 0."""
    m = re.match(r'^(-?)PT(?:(\d+)H)?(?:(\d+)M)?(?:(\d+)S)?$', text or '')
    if not m:
        return 0
    seconds = int(m.group(2) or 0) * 3600 + int(m.group(3) or 0) * 60 + int(m.group(4) or 0)
    return -seconds if m.group(1) == '-' else seconds


def seconds_of_day(text, day_iso):
    """Segundos desde la medianoche de Madrid de «day_iso»; None si la hora es de otro día o no se entiende."""
    if not text:
        return None
    try:
        moment = datetime.datetime.fromisoformat(text.replace('Z', '+00:00'))
    except ValueError:
        return None
    if moment.tzinfo is None:
        moment = moment.replace(tzinfo=MADRID)
    local = moment.astimezone(MADRID)
    if local.date().isoformat() != day_iso:
        return None
    return local.hour * 3600 + local.minute * 60 + local.second


def parse_siri(xml_text, day_iso):
    """Las entradas del feed SIRI Vehicle Monitoring, agrupadas por «línea|número de viaje». La referencia del viaje
    (trp_A0651_737_ABC1_28500) da la línea (sin ceros: 651), el número de viaje y la salida del viaje en segundos."""
    def child(node, *path):
        for name in path:
            if node is None:
                return None
            node = next((c for c in node if c.tag.rsplit('}', 1)[-1] == name), None)
        return node

    def text(node, *path):
        found = child(node, *path)
        return None if found is None else (found.text or '')

```

```
    try:
        root = ElementTree.fromstring(xml_text)
    except ElementTree.ParseError:
        return {}
    out = {}
    delivery = child(root, 'ServiceDelivery', 'VehicleMonitoringDelivery')
    for activity in ([] if delivery is None else delivery):
        if activity.tag.rsplit('}', 1)[-1] != 'VehicleActivity':
            continue
        mvj = child(activity, 'MonitoredVehicleJourney')
        ref = text(mvj, 'VehicleJourneyRef')
        if ref is None:
            continue
        m = re.match(r'^trp_[A-Za-z]+(\d+)_(\d+)_[A-Za-z0-9]+_(\d+)', ref)
        if not m:
            continue
        line, trip_number, departure = line_key(m.group(1)), m.group(2), int
(m.group(3))
        order = text(mvj, 'MonitoredCall', 'Order')
        if order is None:
            order = text(mvj, 'MonitoredCall', 'VisitNumber')
        recorded = seconds_of_day(text(activity, 'RecordedAtTime'), day_iso)
     if text(activity, 'RecordedAtTime') is not None else None
        lat = lon = location = None
        if text(mvj, 'VehicleLocation', 'Latitude') is not None and text(
mvj, 'VehicleLocation', 'Longitude') is not None:
            when = text(mvj, 'LocationRecordedAtTime')
            if when is None:
                when = text(activity, 'RecordedAtTime') or ''
            location = seconds_of_day(when, day_iso)
            if location is not None:
                lat, lon = float(text(mvj, 'VehicleLocation', 'Latitude')),
     float(text(mvj, 'VehicleLocation', 'Longitude'))
        delay = text(mvj, 'Delay')
        out.setdefault(line + '|' + trip_number, []).append({
            'tripRef': ref, 'lineId': line, 'departureSeconds': departure,
            'delaySeconds': iso_duration_seconds('PT0S' if delay is None
else delay),
            'vehicleRef': text(mvj, 'VehicleRef') or '', 'currentStopId':
text(mvj, 'MonitoredCall', 'StopPointRef'),
            'order': None if order is None else int(order), 'lat': lat, '
lon': lon,
            'locationSeconds': location, 'recordedSeconds': recorded})
    return out
```

```
411
412 def has_time(entry):
413     return entry.get('locationSeconds') is not None or entry.get('
    recordedSeconds') is not None
414
415
416 def days_ago(entry, now):
417     """1 si el bus va en un viaje de ayer (su salida queda más de 12 h por
     delante de ahora), 0 si es de hoy."""
418     return int((entry.get('departureSeconds') or 0) - now > DAY_SECONDS / 2)
419
420
421 def vehicles_of_day(vm_map, now, days):
422     """Los buses de viajes de hace «days» días, con sus horas contadas
    desde la medianoche de ese día (para los de
423     ayer, las 00:30 son las 24:30). Un dato sin hora de hoy no cuenta."""
424     out = {}
425     for key, entries in vm_map.items():
426         for entry in entries:
427             if days_ago(entry, now) != days or not has_time(entry):
428                 continue
429             entry = dict(entry)
430             for field in ('locationSeconds', 'recordedSeconds'):
431                 if entry.get(field) is not None:
432                     entry[field] += days * DAY_SECONDS
433             out.setdefault(key, []).append(entry)
434     return out
435
436
437 def with_held(vm_map, previous, now):
438     """El feed nuevo más los buses de la lectura anterior que faltan en él
    : el último dato de cada uno se sigue usando
439     hasta HOLD_SECONDS si su viaje no lo lleva ya otro bus y el bus no
    aparece en otro viaje."""
440     out = {key: list(entries) for key, entries in vm_map.items()}
441     present = {str(e.get('vehicleRef') or '') for entries in vm_map.values()
     for e in entries}
442     for key, entries in previous.items():
443         if key in vm_map or not isinstance(entries, list):
444             continue
445         for entry in entries:
446             if not isinstance(entry, dict) or entry.get('vehicleRef') in (
    None, '', '0', 0) or str(entry['vehicleRef']) in present:
447                 continue
```

```
            seen = entry.get('locationSeconds')
            if seen is None:
                seen = entry.get('recordedSeconds')
            if seen is None or not 0 <= now - seen <= HOLD_SECONDS:
                continue
            held = dict(entry)
            held.setdefault('held', True)
            out.setdefault(key, []).append(held)
            present.add(str(entry['vehicleRef']))
    return out


def lookup(vm_map, aliases, line, trip_number, first_departure, journey_id=None):
    """La entrada del feed de un viaje del horario: la de su referencia exacta (aliases: referencia del feed -> viaje);
    si no, la de la misma línea y número de viaje con la salida más cercana a MATCH_TOLERANCE o menos, sin reutilizar
    una entrada emparejada exactamente con otro viaje."""
    by_journey = {}
    for entries in vm_map.values():
        for entry in entries:
            exact = aliases.get(entry.get('tripRef'))
            if exact is not None:
                by_journey.setdefault(exact, entry)
    if journey_id is not None and journey_id in by_journey:
        return by_journey[journey_id]
    best, best_diff = None, None
    for entry in vm_map.get('%s|%s' % (line, trip_number), []):
        if journey_id is not None and aliases.get(entry.get('tripRef')) is not None:
            continue
        diff = abs(entry['departureSeconds'] - first_departure)
        if diff <= MATCH_TOLERANCE and (best_diff is None or diff < best_diff):
            best, best_diff = entry, diff
    return best


# ---------------------------------------------------------------------- ----- registro de pasos
def passage_fields(vm_map):
    """Lo que se guarda cada minuto por referencia de viaje: «orden de la próxima parada:hora del dato»."""
    out = {}
```

```
    for entries in vm_map.values():
        for entry in entries:
            if not entry.get('tripRef') or entry.get('order') is None:
                continue
            seconds = entry.get('recordedSeconds')
            if seconds is None:
                seconds = entry.get('locationSeconds')
            if seconds is None:
                continue
            out[str(entry['tripRef'])] = '%d:%d' % (int(entry['order']),
    int(seconds))
    return out


def decode_passage(value):
    m = re.fullmatch(r'(\d+):(\d+)', value) if isinstance(value, str) else
    None
    return None if m is None else {'order': int(m.group(1)), 'seconds': int
    (m.group(2))}


def passage_stop_fields(vm_map):
    """El historial por parada: «referencia#orden» -> hora."""
    out = {}
    for ref, value in passage_fields(vm_map).items():
        p = decode_passage(value)
        out['%s#%d' % (ref, p['order'])] = str(p['seconds'])
    return out


def merge_passages(journey_of_field, values):
    """Por viaje del horario, el paso de orden mayor de todas sus
    referencias."""
    out = {}
    for i, value in enumerate(values):
        p = decode_passage(value)
        if p is None or i >= len(journey_of_field):
            continue
        journey = journey_of_field[i]
        if journey not in out or p['order'] > out[journey]['order']:
            out[journey] = p
    return out


def stops_from_hash(fields):
```

```
    trips = {}
    for field, seconds in fields.items():
        m = re.fullmatch(r'(.+)#(\d+)', str(field))
        if not m or not str(seconds).isdigit():
            continue
        trips.setdefault(m.group(1), {})[int(m.group(2))] = int(seconds)
    return {ref: dict(sorted(stops.items())) for ref, stops in sorted(trips.
    items())}


# ---------------------------------------------------------------------------
    ----- el emparejador
class Matcher:
    def __init__(self, journey, live, now, date_iso, segments, pace, trips=
    None, holiday=False, passage=None):
        # passage: el paso registrado del viaje ({'order', 'seconds'},
    merge_passages), que el motor solo consulta
        # cuando ningún bus del feed lleva exactamente ese viaje.
        self.passage = passage
        self.j, self.live, self.now, self.date = journey, live, now,
    date_iso
        self.seg, self.pace = segments, pace
        self.trips = trips
        self.service = service_date(date_iso, holiday)
        self.distrusted = False
        self._pos = None
        self._pos_done = False

    def position(self):
        if not self._pos_done:
            self._pos_done = True
            self._pos = None
            if self.live:
                self._pos = position(self.j, self.live, self.now)
            if self._pos is not None:
                reg = self.registration()
                gps_delay = self._pos['locationSeconds'] - self._pos['
    scheduledSeconds']
                if reg is not None and reg['gap'] <= CROSSCHECK_MAX_SEGMENT
     and abs(gps_delay - reg['delay']) > MAX_GPS_VS_STOP:
                    self.distrusted = True
                    self._pos = None
        return self._pos

    def registration(self):
```

```
        live, stops = self.live, self.j['stops']
        if not live or live.get('order') is None or self.distrusted:
            return None
        order = live['order']
        if order < 2:
            return None
        ref = live.get('locationSeconds')
        if ref is None:
            ref = live.get('recordedSeconds')
        if ref is None or self.now - ref > MAX_LOCATION_AGE:
            return None
        if not stops or ref < stops[0]['dep'] - NOT_STARTED_GRACE:
            return None
        by_seq = {s['seq']: s for s in stops}
        if order not in by_seq or order - 1 not in by_seq:
            return None
        prev, nxt = by_seq[order - 1]['arr'], by_seq[order]['arr']
        if nxt - prev > REGISTRATION_MAX_SEGMENT:
            return None
        band = band_for(self.date, ref)
        k = pace_factor(self.pace, live.get('lineId'), self.date, ref)
        to_next = php_round(REGISTRATION_SHARE * expected_seconds(stops,
    self.seg, prev, nxt, band, k))
        return {'order': order, 'ref': ref, 'previous': prev, 'next': nxt, '
    toNext': to_next,
                'delay': ref + to_next - nxt, 'gap': nxt - prev}

    def registered_eta(self, target, target_seq):
        reg = self.registration()
        if reg is None or target_seq < reg['order']:
            return None
        after = 0.0
        if target > reg['next']:
            band = band_for(self.date, reg['ref'])
            k = pace_factor(self.pace, self.live.get('lineId'), self.date,
    reg['ref'])
            after = expected_seconds(self.j['stops'], self.seg, reg['next'],
     target, band, k)
        return php_round(reg['ref'] + reg['toNext'] + after)

    def origin_eta(self, target, target_seq):
        """Esperando en la primera parada, la llegada es la de la capa 3 (
    no ha recorrido nada todavía)."""
        stops = self.j['stops']
        if not self.seg.get('segments') or target <= stops[0]['dep']:
```

```
            return None
        return max(self.learned_arrival(target) + self.trip_offset(target_seq), self.now)

    def learned_arrival(self, target):
        stops = self.j['stops']
        first = stops[0]['dep']
        mid = (first + target) // 2
        band = band_for(self.service, mid)
        k = pace_factor(self.pace, self.j.get('line'), self.service, mid)
        return first + SCHEDULE_ORIGIN_OFFSET + php_round(expected_seconds(stops, self.seg, first - 1, target, band, k))

    def eta_for_stop(self, target, target_seq):
        pos = self.position()
        if pos is not None:
            if pos['method'] == 'origin':
                eta = self.origin_eta(target, target_seq)
                if eta is not None:
                    return eta, eta - target, True
            if target < pos['scheduledSeconds']:
                passed = pos['locationSeconds'] - (pos['scheduledSeconds'] - target)
                return passed, pos['locationSeconds'] - pos['scheduledSeconds'], True
            band = band_for(self.date, pos['locationSeconds'])
            k = pace_factor(self.pace, self.live.get('lineId'), self.date, pos['locationSeconds'])
            eta = php_round(pos['locationSeconds'] + expected_seconds(self.j['stops'], self.seg, pos['scheduledSeconds'], target, band, k))
            return eta, eta - target, True
        reg_eta = self.registered_eta(target, target_seq)
        if reg_eta is not None:
            return reg_eta, reg_eta - target, False
        return target, 0, False

    def passed_at(self, target, target_seq):
        pos = self.position()
        order = None
        if self.live:
            order = self.live.get('order')
        if pos is not None:
            if pos['method'] == 'origin':
                return None
            feed_past = order is not None and target_seq < order
```

```
            if target < pos['scheduledSeconds'] and (feed_past or self.meters_from_stop(target_seq) > PASSED_DISTANCE):
                return pos['locationSeconds'] - (pos['scheduledSeconds'] - target)
            if feed_past:
                return pos['locationSeconds']
            return None
        if order is not None and target_seq < order:
            reg = self.registration()
            if reg is not None:
                return reg['ref'] - max(0, reg['previous'] - target)
        return None

    def trip_offset(self, target_seq):
        stops = self.j['stops']
        indexes = [i for i, s in enumerate(stops) if s['seq'] == target_seq]
        if not self.trips or not indexes:
            return 0
        key = trip_key(self.j.get('line'), stops[0]['id'], stops[-1]['id'], stops[0]['dep'], len(stops))
        return php_round(trip_offset(self.trips, key, self.j.get('line'), self.service, stops[0]['dep'], indexes[0], len(stops)))

    def scheduled_fields(self, target, target_seq):
        if self.passage is not None and target_seq < self.passage['order']:
            return {'status': 'departed', 'eta': min(self.passage['seconds'], self.now), 'delay': 0}
        out = {'status': 'scheduled', 'eta': target, 'delay': 0}
        stops = self.j['stops']
        if not self.seg.get('segments'):
            return out
        first = stops[0]['dep']
        if target <= first:
            return out
        mid = (first + target) // 2
        band = band_for(self.service, mid)
        corrected = self.learned_arrival(target) + self.trip_offset(target_seq)
        if corrected >= self.now:
            return {'status': 'estimated', 'eta': corrected, 'delay': 0}
        if (corrected < self.now - WRONG_SCHEDULE_PASSED and target - corrected >= WRONG_SCHEDULE_GAP
                and learned_share(stops, self.seg, first - 1, target, band) >= WRONG_SCHEDULE_LEARNED_SHARE):
```

```
            out['status'] = 'likely_departed'
        return out

    def enrich(self, target, target_seq):
        live_ok = self.live is not None and (self.position() is not None or
     self.registration() is not None)
        if not live_ok:
            return self.scheduled_fields(target, target_seq)
        passed = self.passed_at(target, target_seq)
        if passed is not None:
            return {'status': 'departed', 'eta': min(passed, self.now), '
    delay': 0}
        if self.position() is None and self.registered_eta(target,
    target_seq) is None:
            return self.scheduled_fields(target, target_seq)
        eta, delay, positioned = self.eta_for_stop(target, target_seq)
        if abs(delay) > MAX_PLAUSIBLE_DELAY:
            delay = None
        if eta < self.now - STALE_GRACE and not positioned:
            return self.scheduled_fields(target, target_seq)
        if not positioned:
            return {'status': 'estimated', 'eta': eta, 'delay': 0}
        if eta < self.now:
            status = 'live'
            if self.now - eta > ARRIVING_MAX:
                status = 'likely_departed'
            if delay is not None:
                delay = max(delay, self.now - target)
            return {'status': status, 'eta': self.now, 'delay': delay}
        return {'status': 'live', 'eta': eta, 'delay': delay}

    def current_delay(self):
        """Retraso del bus ahora: el de su posición; sin posición, el de
    la parada registrada; si no, 0."""
        if self.live is None:
            return 0
        pos = self.position()
        if pos is not None:
            return pos['locationSeconds'] - pos['scheduledSeconds']
        reg = self.registration()
        return 0 if reg is None else reg['delay']

    def display_delay(self):
        """Retraso que se muestra del bus: con posición, el que llevará en
     su próxima parada; si no, current_delay."""
```

```
        current = self.current_delay()
        if self.live is None or self.live.get('order') is None or self.
    position() is None:
            return current
        order = self.live['order']
        nxt = next((s['arr'] for s in self.j['stops'] if s['seq'] == order),
     None)
        if nxt is None:
            return current
        return self.eta_for_stop(nxt, order)[1]

    def meters_from_stop(self, target_seq):
        for s in self.j['stops']:
            if s['seq'] == target_seq and s.get('lat') is not None and s.
    get('lon') is not None:
                dx = (self.live['lon'] - s['lon']) * math.cos(math.radians(s
    ['lat']))
                dy = self.live['lat'] - s['lat']
                return math.sqrt(dx * dx + dy * dy) * METERS_PER_DEGREE
        return math.inf


# ---------------------------------------------------------------------
    ----- calibración
def fit_segments(samples):
    groups = {}
    for s in samples:
        g = groups.setdefault(s['from'] + '>' + s['to'], {'all': {}})
        g['all'][s['track']] = s['actual']
        g.setdefault(s['band'], {})[s['track']] = s['actual']
    out = {}
    for key, gr in groups.items():
        if len(gr['all']) < MIN_TRACKS_PER_SEGMENT:
            continue
        seg = {'n': len(gr['all'])}
        all_median = percentile(list(gr['all'].values()), 0.5)
        several = len(gr) > 2
        for name, values in gr.items():
            if name == 'all' and not several:
                continue
            if len(values) < MIN_TRACKS_PER_SEGMENT:
                continue
            v = percentile(list(values.values()), 0.5)
            if name != 'all':
                n = len(values)
```

```
                v = (v * n + all_median * SEGMENT_BAND_PRIOR_TRACKS) / (n + SEGMENT_BAND_PRIOR_TRACKS)
            seg[name] = php_round(v)
        if len(seg) > 1:
            out[key] = seg
    return dict(sorted(out.items()))


def fit_trip_offsets(residuals, rank=TRIP_OFFSET_RANK):
    """Tabla de la corrección por viaje a partir de los desvíos (hora real menos capa 3 sin corregir), con el
    percentil «rank» (0,4 de partida; la recalibración lo elige cada noche con choose_trip_offset_rank)."""
    by_band, by_line, by_day, trip_band = {}, {}, {}, {}
    for r in residuals:
        error = clamp(r['error'], -TRIP_OFFSET_CLIP, TRIP_OFFSET_CLIP)
        band = band_for(r['serviceDate'], r['departure'])
        kind = day_type(r['serviceDate'])
        part = trip_part(r['index'], r['count'])
        by_band.setdefault(band, []).append(error)
        by_line.setdefault(r['line'], {}).setdefault(band, []).append(error)
        by_day.setdefault(r['trip'], {}).setdefault(kind, {}).setdefault(part, {}).setdefault(r['date'], []).append(error)
        trip_band.setdefault(r['trip'], {})[kind] = (r['line'], band)
    overall = {band: percentile(v, rank) for band, v in by_band.items()}
    lines = {}
    for line, bands in by_line.items():
        lines[line] = {band: (percentile(v, rank) * len(v) + overall[band] * TRIP_OFFSET_LINE_PRIOR) / (len(v) + TRIP_OFFSET_LINE_PRIOR)
                       for band, v in bands.items()}
    trips = {}
    for trip, kinds in by_day.items():
        for kind, parts in kinds.items():
            line, band = trip_band[trip][kind]
            line_value = lines[line][band]
            values = [php_round(line_value)] * TRIP_PARTS
            for part, days in parts.items():
                day_values = [percentile(v, rank) for v in days.values()]
                values[part] = php_round((sum(day_values) + line_value * TRIP_OFFSET_TRIP_PRIOR_DAYS) / (len(day_values) + TRIP_OFFSET_TRIP_PRIOR_DAYS))
            trips.setdefault(trip, {})[kind] = values
    return {'trips': dict(sorted(trips.items())),
            'lines': {line: {band: php_round(v) for band, v in bands.items()} for line, bands in sorted(lines.items())},
```

```
            'global': {band: php_round(v) for band, v in sorted(overall.items())}}


def display_delay_minutes(seconds):
    """Minutos de retraso que se muestran; un retraso de más de 45 min no se muestra."""
    if seconds is None or abs(seconds) > MAX_PLAUSIBLE_DELAY:
        return None
    return php_round(seconds / 60)


def trip_offset_errors(residuals, table):
    """[error medio sin corregir, error medio corregido, % a ±2 min sin, % a ±2 min con] de unos desvíos."""
    before = after = 0.0
    within_before = within_after = 0
    for r in residuals:
        corrected = r['error'] - trip_offset(table, r['trip'], r['line'], r['serviceDate'], r['departure'], r['index'], r['count'])
        before += abs(r['error'])
        after += abs(corrected)
        within_before += abs(r['error']) <= 120
        within_after += abs(corrected) <= 120
    n = max(1, len(residuals))
    return [before / n, after / n, 100 * within_before / n, 100 * within_after / n]


def choose_trip_offset_rank(train, test, ranks=TRIP_OFFSET_RANKS):
    """El percentil que más acierta a ±2 min en los días de prueba aprendiendo con los anteriores; con empate, el de
    menos error medio; con empate en los dos, el primero de la lista. Devuelve también sus errores
    (trip_offset_errors): la tabla nueva solo se escribe si el error medio corregido es menor que el sin corregir."""
    chosen, best = ranks[0], None
    for rank in ranks:
        errors = trip_offset_errors(test, fit_trip_offsets(train, rank))
        if best is None or (errors[3], -errors[1]) > (best[3], -best[1]):
            chosen, best = rank, errors
    return chosen, best


def shrunk_factor(tracks):
```

```
    ratios = [sum(s['actualRemaining'] for s in t) / sum(s['scheduledRemaining'] for s in t) for t in tracks.values()]
    m = len(ratios)
    raw = percentile(ratios, 0.5)
    k = 1.0 + (raw - 1.0) * m / (m + SHRINKAGE_TRACKS)
    return php_round(clamp(k, K_MIN, K_MAX) * 1000) / 1000


def actual_passing_time(track, arrival):
    for a, b in zip(track, track[1:]):
        if b['sched'] <= a['sched'] or b['t'] - a['t'] > MAX_GAP_BETWEEN_SNAPSHOTS:
            continue
        if a['sched'] <= arrival <= b['sched']:
            return a['t'] + (arrival - a['sched']) / (b['sched'] - a['sched']) * (b['t'] - a['t'])
    return None


def thin(samples_of_one_track):
    n = len(samples_of_one_track)
    if n <= MAX_SAMPLES_PER_TRACK:
        return samples_of_one_track
    return [samples_of_one_track[(i * n) // MAX_SAMPLES_PER_TRACK] for i in range(MAX_SAMPLES_PER_TRACK)]


def fold(track):
    """Pliegue de la validación cruzada: crc32 (IEEE 802.3, sin signo) módulo 2."""
    return (zlib.crc32(track.encode('utf-8')) & 0xFFFFFFFF) % 2
```

**Listing E.2**

*comprobar.py*

```
"""Comprueba la implementación de referencia (referencia.py) contra los vectores del código de producción."""
import json, math, os, sys
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import referencia as R

import os
AQUI = os.path.dirname(os.path.abspath(__file__))
V = json.load(open(os.path.join(AQUI, 'vectores.json'), encoding='utf-8'))
fallos = 0
```

```
total = 0
def igual(nombre, esperado, obtenido, tol=1e-6):
    global fallos, total
    total += 1
    ok = esperado == obtenido
    if isinstance(esperado, float) or isinstance(obtenido, float):
        ok = esperado is not None and obtenido is not None and abs(esperado - obtenido) <= tol
    if not ok:
        fallos += 1
        print('FALLO', nombre, 'esperado', esperado, 'obtenido', obtenido)

for v in V['percentil']:
    igual('percentil', v['resultado'], R.percentile(v['valores'], v['p']))
for v in V['franja']:
    igual('franja', v['franja'], R.band_for(v['fecha'], v['segundos']))
pace = {'global': {'laborable.punta_manana': 0.9, 'laborable.noche': 0.4}, 'lines': {'3414': {'laborable.punta_manana': 1.35}}}
for v in V['factor_k']:
    igual('k', v['k'], R.pace_factor(pace, v['linea'], v['fecha'], v['segundos']))
for v in V['proyeccion']:
    f, d, l = R.project_onto_segment(v['bus'][0], v['bus'][1], v['A'], v['B'])
    igual('proy f', v['f'], f); igual('proy d', v['distancia_m'], d, 1e-6); igual('proy L', v['largo_m'], l, 1e-6)
for v in V['cubo_largo']:
    igual('cubo', v['cubo'], R.length_bucket(v['metros']))
ratios = {'segments': {'X>Y': {'n': 2, 'all': 1}}, 'lengthRatios': {'2': {'all': 0.769, 'laborable.punta_manana': 0.811}, '4': {'all': 2.0}, '5': {'all': 0.1}}}
for v in V['respaldo_largo']:
    igual('respaldo', v['segundos'], R.fallback_seconds(ratios, float(v['horario_s']), v['metros'], v['franja']), 1e-9)
# lengthRatios en forma de lista (como lo escribe PHP con los seis cubos) da lo mismo que en forma de objeto
lista = {'segments': {'X>Y': {'n': 2, 'all': 1}}, 'lengthRatios': [{'all': 1.0}, {'all': 0.9}, {'all': 0.769, 'laborable.punta_manana': 0.811}, {'all': 0.65}, {'all': 0.64}, {'all': 0.53}]}
igual('respaldo con lista', 0.811 * 120, R.fallback_seconds(lista, 120.0, 800, 'laborable.punta_manana'), 1e-9)
igual('respaldo con lista, cubo 5', 0.53 * 300, R.fallback_seconds(lista, 300.0, 5000, 'laborable.manana'), 1e-9)
igual('tiempos por tramo', V['tiempos_tramo']['resultado'], R.fit_segments(V['tiempos_tramo']['muestras']))
```

```
for v in V['k_calibrado']:
    tracks = {'t%d' % i: [{'actualRemaining': 600 * r, 'scheduledRemaining': 600}] for i, r in enumerate(v['cocientes_por_bus'])}
    igual('k calibrado', v['k'], R.shrunk_factor(tracks))
for v in V['paso_real']:
    igual('paso real', v['paso_real'], R.actual_passing_time(v['pista'], v['llegada_programada']), 1e-9)
igual('aclarado', V['aclarado']['indices'], [s['i'] for s in R.thin([{'i': i} for i in range(100)])])

# Corrección por viaje
cv = V['correccion_viaje']
igual('clave del viaje', cv['clave'], R.trip_key('9999', 'S0', 'S4', 28800, 5))
for indice, paradas, tercio in cv['tercios']:
    igual('tercio %d de %d' % (indice, paradas), tercio, R.trip_part(indice, paradas))
for fecha, festivo, dia in cv['dia_servicio']:
    igual('día de servicio ' + fecha, dia, R.service_date(fecha, festivo))
for c in cv['consultas']:
    igual('corrección ' + c['clave'] + ' ' + c['fecha'], c['segundos'], R.trip_offset(cv['tabla'], c['clave'], c['linea'], c['fecha'], c['salida'], c['indice'], c['paradas']))
igual('calibración de la corrección por viaje', V['calibracion_viaje']['resultado'], R.fit_trip_offsets(V['calibracion_viaje']['desvios']))

# Viajes que pasan de medianoche
dv = V['dia_del_viaje']
for c in dv['casos']:
    igual('día del viaje: salida %d a las %d' % (c['salida'], c['ahora']), c['dias_atras'], R.days_ago({'departureSeconds': c['salida']}, c['ahora']))
igual('buses de viajes de ayer', dv['de_ayer'], R.vehicles_of_day(dv['feed'], dv['ahora'], 1))
igual('buses de viajes de hoy', dv['de_hoy'], R.vehicles_of_day(dv['feed'], dv['ahora'], 0))

# Buses que faltan en una lectura del feed
br = V['buses_retenidos']
igual('buses retenidos 180 s', br['resultado'], R.with_held(br['feed'], br['anterior'], br['ahora']))

# Emparejado de cada bus con su viaje
em = V['emparejado']
for c in em['casos']:
```

```
    entrada = R.lookup({'9999|101': c['buses']}, em['alias'], '9999', '101', c['salida'], c['viaje'])
    igual('emparejado: ' + c['caso'], c['bus'], None if entrada is None else entrada['vehicleRef'])

# Lectura del feed SIRI
igual('lectura del feed SIRI', V['lectura_feed']['resultado'], R.parse_siri(V['lectura_feed']['xml'], V['lectura_feed']['hoy']))

# Registro de pasos
rp = V['registro_pasos']
igual('registro de pasos: campos', rp['campos'], R.passage_fields(rp['feed']))
igual('registro de pasos: campos por parada', rp['campos_paradas'], R.passage_stop_fields(rp['feed']))
igual('registro de pasos: lectura', rp['lectura']['resultado'], R.merge_passages(rp['lectura']['viaje_de_cada_campo'], rp['lectura']['valores']))
historial = {ref: {str(k): v for k, v in stops.items()} for ref, stops in R.stops_from_hash(rp['historial']['hash']).items()}
igual('registro de pasos: historial', rp['historial']['resultado'], historial)

# Retraso que se muestra
rm = V['retraso_mostrado']
for c in rm['minutos']:
    igual('minutos mostrados de %s s' % c['segundos'], c['minutos'], R.display_delay_minutes(c['segundos']))

# Elección nocturna del percentil de la corrección por viaje
ep = V['eleccion_percentil']
entreno = [r for r in ep['desvios'] if r['date'] < ep['primer_dia_de_prueba']]
prueba = [r for r in ep['desvios'] if r['date'] >= ep['primer_dia_de_prueba']]
for c in ep['por_percentil']:
    errores = R.trip_offset_errors(prueba, R.fit_trip_offsets(entreno, c['percentil']))
    igual('percentil %.2f: error medio' % c['percentil'], float(c['error_medio_s']), errores[1])
    igual('percentil %.2f: a ±2 min' % c['percentil'], float(c['a_2_min']), errores[3])
elegido, errores = R.choose_trip_offset_rank(entreno, prueba, ep['candidatos'])
igual('percentil elegido', ep['elegido'], elegido)
```

```
igual('error medio sin corregir', float(ep['sin_corregir']['error_medio_s']), errores[0])
igual('a ±2 min sin corregir', float(ep['sin_corregir']['a_2_min']), errores[2])
igual('la tabla nueva se escribe solo si baja el error medio', ep['se_escribe'], errores[1] < errores[0])

# Escenarios del emparejador
vj = V['viaje']
segments_base = {'segments': vj['tramos_aprendidos'], 'lengthRatios': vj['proporciones']}
pace_k = {'global': vj['k']}
for e in V['escenarios']:
    publicadas = [p['hora_publicada'] for p in e['paradas']]
    stops = []
    for i, p in enumerate(vj['paradas']):
        dist = None
        if e['trazado']:
            dist = p['dist_m']
        stops.append({'seq': i + 1, 'id': p['id'], 'arr': publicadas[i], 'dep': publicadas[i], 'lat': p['lat'], 'lon': p['lon'],
                      'dist': dist})
    shape = None
    if e['trazado']:
        shape = [[p['lat'], p['lon'], p['dist_m']] for p in vj['paradas']]
    journey = {'stops': stops, 'shape': shape, 'line': '9999'}
    segs = segments_base
    if 'probablemente' in e['escenario']:
        segs = {'segments': {'S0>S1': {'n': 9, 'laborable.punta_manana': 110}, 'S1>S2': {'n': 9, 'laborable.punta_manana': 140},
                             'S2>S3': {'n': 9, 'laborable.punta_manana': 240}, 'S3>S4': {'n': 9, 'laborable.punta_manana': 100}}}
    trips = None
    if e.get('correccion_viaje'):
        trips = cv['tabla']
    festivo = e.get('festivo', False)
    M = R.Matcher(journey, e['entrada'], e['ahora'], vj['fecha'], segs, pace_k, trips, festivo)
    if e['entrada'] is not None:
        pos = M.position()
        exp = e['posicion']
        esperado = None
        if exp is not None:
            esperado = exp['scheduledSeconds']
        obtenido = None
```

```
        if pos is not None:
            obtenido = pos['scheduledSeconds']
        igual(e['escenario'] + ' · posición', esperado, obtenido)
    for i, p in enumerate(e['paradas']):
        out = R.Matcher(journey, e['entrada'], e['ahora'], vj['fecha'], segs, pace_k, trips, festivo).enrich(p['hora_publicada'], i + 1)
        igual(e['escenario'] + ' · ' + p['parada'] + ' estado', p['estado'], out['status'])
        igual(e['escenario'] + ' · ' + p['parada'] + ' llegada', p['llegada_estimada'], out['eta'])
        igual(e['escenario'] + ' · ' + p['parada'] + ' retraso', p['retraso_s'], out['delay'])

# Escenarios con el registro de pasos: sin bus en el feed
journey_base = {'stops': [{'seq': i + 1, 'id': p['id'], 'arr': p['llegada'], 'dep': p['salida'], 'lat': p['lat'], 'lon': p['lon'],
                           'dist': p['dist_m']} for i, p in enumerate(vj['paradas'])],
                'shape': [[p['lat'], p['lon'], p['dist_m']] for p in vj['paradas']], 'line': '9999'}
for e in V['escenarios_pasos']:
    for i, p in enumerate(e['paradas']):
        out = R.Matcher(journey_base, None, e['ahora'], vj['fecha'], segments_base, pace_k, None, False, e['paso_registrado']).enrich(p['hora_publicada'], i + 1)
        igual(e['escenario'] + ' · ' + p['parada'] + ' estado', p['estado'], out['status'])
        igual(e['escenario'] + ' · ' + p['parada'] + ' llegada', p['llegada_estimada'], out['eta'])
        igual(e['escenario'] + ' · ' + p['parada'] + ' retraso', p['retraso_s'], out['delay'])

# Retraso que se muestra de cada bus
for c in rm['buses']:
    M = R.Matcher(journey_base, c['entrada'], c['ahora'], vj['fecha'], segments_base, pace_k)
    igual(c['caso'] + ' · retraso actual', c['retraso_actual_s'], M.current_delay())
    igual(c['caso'] + ' · retraso mostrado', c['retraso_mostrado_s'], M.display_delay())

print(f'{total - fallos} de {total} comprobaciones coinciden con el código de producción')
if fallos:
    sys.exit(1)
```

```
168 sys.exit(0)
```

# Apéndice F

## Glosario

**Alias de viaje.** Correspondencia entre un `trip_id` original del GTFS y el viaje fusionado que lo contiene (sección «Fusión de Variantes y Alias de Viaje»).

**Cabecera.** Primera parada de un viaje; su hora de salida es $d_1$.

**Capa.** Cada una de las tres fuentes de la predicción: GPS, parada registrada y horario corregido (capítulo «Predicción en Tres Capas»).

**Captura.** Conjunto de versiones del feed descargadas durante un bloque de captura (hasta una hora) (sección «Captura del Feed»).

**Contracción.** Media ponderada entre una estimación propia y un valor común, con un peso que crece con los datos propios (secciones «Contracción y Modelos Jerárquicos» y «Mediana y Contracción Hacia el Tramo a Cualquier Hora»).

**Corrección por viaje.** Lo que suele apartarse cada viaje del horario corregido, por tipo de día y tercio del recorrido (sección «Corrección por Viaje»).

**Cubo de longitud.** Intervalo de longitudes de tramo para el respaldo por longitud (ecuación (13)).

**Día del viaje.** Día de servicio al que pertenece un viaje del horario. Sus horas se cuentan desde su medianoche y pueden pasar de las 24:00 (sección «Viajes que Pasan de las 24:00»).

**Desfase.** Diferencia $\Delta(s) = R(s) - T(s)$ entre la hora real y la programada en un punto del recorrido.

**Feed.** Fuente de datos en tiempo real publicada por el operador; aquí, SIRI Vehicle Monitoring.

**Festivo entre semana.** Día en que el horario hace circular por excepción los calendarios de solo domingo; en la capa 3 cuenta como fin de semana.

**Franja.** Combinación de tipo de día y periodo del día (definición 4).

**GTFS.** *General Transit Feed Specification*, formato abierto de horarios de transporte público.

**Horario corregido.** Horario recalculado desde la cabecera con los tiempos aprendidos (capa 3).

**Horizonte.** Tiempo que falta para la llegada en el momento de la predicción.

**Interpolación del horario.** Cálculo de las horas de las paradas intermedias repartiendo el tiempo entre puntos de control a velocidad constante (`timepoint=0`).

**Mediana baja.** Elemento central de una lista ordenada; con un número par de elementos, el menor de los dos centrales.

**Muestra de ritmo.** Par (tiempo programado, tiempo real) que faltaba para llegar a una parada desde una posición (sección «Muestras de Ritmo»).

**Muestra de tramo.** Tiempo real que tardó un autobús en ir de una parada a la siguiente (sección «Muestras de Tramo»).

**Pasillo.** Distancia máxima al recorrido para aceptar una posición GPS.

**Paso real.** Instante, reconstruido, en que un autobús pasó por una parada (definición 10).

**Pista.** Recorrido de un vehículo en un viaje en una fecha; unidad de independencia de las muestras.

**Pliegue.** Mitad de las pistas en la validación cruzada.

**Proporción por longitud.** Mediana de tiempo real entre tiempo programado de los tramos de un cubo de longitud.

**Registro.** Último dato del feed sobre la próxima parada de un autobús y su hora, usado por la capa 2.

**Registro de pasos.** Almacén que guarda el último orden de parada visto de cada viaje, por día del viaje (sección «Registro de Pasos»).

**SIRI-VM.** *Service Interface for Real Time Information, Vehicle Monitoring*: norma europea para publicar la actividad de los vehículos.

**Tramo.** Par de paradas consecutivas de un viaje.

**Trazado.** Polilínea geográfica que sigue el recorrido de un viaje (`shapes.txt`).

$T_{\text{pos}}$**.** Hora a la que el horario pone al autobús en el punto del recorrido donde está.

**Ventana.** Conjunto de tramos alrededor de la próxima parada donde se busca la proyección del GPS.